Croqtile
Programming Guide

A guide to the Croqtile language

Version 1.0 — November 2026

LancerLab

Preface

As large models have become widely used, efficient operator implementations have become a growing part of the development work. Model architectures may share a few common designs, but the computations within them continue to vary. Attention mechanisms, sparsity patterns, positional encodings and quantisation schemes all introduce new implementation requirements. Even the same operator may need different kernels for different input shapes or numerical formats. A shared model design therefore leaves substantial work to be done at the operator level.

That work also grows with the variety of computing platforms. A program may need to run on several kinds of machine, each with its own memory organisation and execution model. Within a single family of machines, new instructions and programming interfaces can require an existing kernel to be reconsidered and retuned. Operator fusion adds another source of variation: combining several operations can reduce intermediate data transfers and launch costs, but each combination brings its own implementation choices. Supporting these workloads calls for a continuing supply of kernels and the means to develop them efficiently.

Writing those kernels is demanding because performance depends on decisions that interact: tile sizes, data layouts, memory staging, pipeline depth and cooperation between parallel regions. A change in one can require changes in several others. Higher-level kernel languages have emerged to improve author productivity by expressing more of this work with less code. Their central challenge is to simplify programming while keeping the decisions needed to reach high performance available to the programmer. When an abstraction hides those decisions, it also limits the implementations that can be explored.

Croqtile is a language for high-performance heterogeneous computing built around this challenge, with human productivity as its starting point. It gives shapes, tile selection, data movement and coordination direct language forms, so a programmer can describe how a kernel works in one compact program. Related sizes and indices are derived from shaped data, reducing the bookkeeping that otherwise accompanies a structural change. Symbolic dimensions allow the same source to handle different input sizes, while compile-time checks identify inconsistent shapes and indices before execution. These abstractions add no runtime overhead. Together, they make kernels easier to read and revise while leaving performance decisions explicit.

The same design also supports AI-assisted kernel development and tuning. Compact source leaves room in an agent's context for profiling results and previous attempts. Independently expressed operations let the agent change a pipeline or a parallel arrangement through a small number of edits, and compiler diagnostics help it locate and correct invalid combinations. Croqtile is agent-ready in this practical sense: the language supports repeated cycles of proposing a change, checking it and measuring its effect. Its AI-native design carries the goal of efficient kernel development into an automated workflow.

Croqtile is embedded in C++. In a `.co` file, ordinary C++ prepares shaped buffers and calls a `__co__` function; that function describes the iterations, tiles, data movement and coordination of the parallel work. This guide introduces those constructs through examples and shows how to combine them into complete programs.

Reading guide

This guide is for developers who want to read and write Croqtile kernels. It assumes familiarity with C++ functions, pointers and arrays, and with the shape of a tensor. The examples begin with basic language constructs and show how to organise parallel computation, select and move tiles, and coordinate execution. Working through them will help you read a `.co` program and combine these constructs into kernels of your own.

If you are new to Croqtile, begin with Chapter 1 to build the compiler and run a first program. Then read Chapter 2 in order. Its short examples start with function declarations and shapes, then introduce iteration, tiling, data movement and coordination. Look first at the structure of each program, then use the explanation to understand the role of each statement. Focus on how the data is divided, moved and used in computation.

Once you are comfortable with the basic syntax, consult Chapter 3 as needed. It groups the language by function and explains the semantics, constraints and design choices behind the constructs. Complete kernels appear alongside the features they use, so you can study a particular topic or return to it while writing a program. Appendix A collects the syntax for quick lookup; the corresponding sections explain the meaning of a construct and the conditions for using it.

Conventions

Croqtile code is set in a monospaced font. Longer examples appear as numbered listings, collected in the *List of Code Listings*; short fragments appear inline, for example `s32 [4, 8] output;` or `dma.copy src => shared`. Three kinds of callout box carry the running commentary: a **Chapter roadmap** opens each chapter with what it covers and where to go afterwards, a **Key point** marks an idea worth carrying forward, and a **Common pitfall** records a rule that is easy to get wrong. Examples use the `croq::` namespace for host-side helpers and `__co__` for the function that describes parallel work; the two forms appear together in almost every complete program.

Contents

Preface iii

Contents v

List of Code Listings vii

1 Getting Started 1
- 1.1 What is Croqtile . . . 1
- 1.2 Setting Up: Requirements, Build, and Verification . . . 1
- 1.3 The Toolchain: Command Line and Build Utilities . . . 3
- 1.4 Inspecting and Debugging a Build . . . 5

2 Language Basics 8
- 2.1 The Two Parts of a Croqtile Program . . . 8
- 2.2 Reading a Kernel Line by Line . . . 9
- 2.3 The Host-Side API: Inputs and Outputs . . . 10
- 2.4 Shapes and mdspan . . . 14
- 2.5 Integers, I-Tuples, and Bounded Values . . . 17
- 2.6 Spanned Data and Storage Specifiers . . . 20
- 2.7 Iteration: parallel-by, with-in, and foreach . . . 24
- 2.8 Tile Selection: chunkat, subspan, and view/from . . . 33
- 2.9 Data Movement: the DMA Statement and Futures . . . 38
- 2.10 Coordination: inthreads, Events, and Async Blocks . . . 46
- 2.11 Branching and Early Return . . . 49
- 2.12 The Four-Step MMA Syntax . . . 50
- 2.13 C++ Embedding, Calls, Macros, and the Preprocessor . . . 50
- 2.14 Printing, Assertions, and Compile-Time Inspection . . . 54
- 2.15 Syntax Quick Reference . . . 56

3 Programming Guide 58
- 3.1 Program Structure and Compilation Models . . . 58
- 3.2 Syntax Design and the Type System . . . 61
- 3.3 Symbolic Shapes, Shape Evaluation, and Value Numbering . . . 66
- 3.4 Parallelism, Space Specifiers, and Masking . . . 72
- 3.5 Tile Selection and Data Expressions . . . 79
- 3.6 Data Movement: Operations, Configuration, and Async Patterns . . . 84
- 3.7 Contraction and Layout Contracts . . . 91
- 3.8 Coordination, Pipelines, and Persistent Scheduling . . . 96
- 3.9 C++ Interoperability and the Preprocessor . . . 109
- 3.10 Tileflow Patterns and Design Rationales . . . 113
- 3.11 Diagnostics and Methodology . . . 117

A Syntax Quick Reference **118**
A.1 Declarations and shapes 118
A.2 Iteration 118
A.3 Tile selection and movement 118
A.4 Statement modifiers and contraction 118
A.5 Coordination, interop and diagnostics 120

References **121**

Bibliography **121**

List of Code Listings

1.1 Clone and prepare . . . 2
1.2 Build the compiler . . . 2
1.3 Run the test suite . . . 2
1.4 A minimal Croqtile program . . . 2
1.5 Compile and run the check . . . 2
1.6 A complete element-wise addition . . . 3
1.7 Compile and run the example . . . 3
1.8 Inspecting intermediate output and compiler help . . . 4
1.9 Testing sample operators . . . 5
1.10 Verbose mode: -v . . . 5
1.11 Runtime checks: -rtc . . . 5
1.12 Debug RTTI and debugger . . . 5
1.13 Debug RTTI and debugger . . . 5
1.14 Guarding debug prints . . . 6
1.15 Debug and production builds . . . 6
2.1 The three regions of one source file . . . 8
2.2 A kernel signature (fragment) . . . 9
2.3 Allocating the output . . . 9
2.4 The parallel body . . . 9
2.5 Returning the result . . . 10
2.6 Creating the shaped inputs . . . 10
2.7 Calling the kernel . . . 10
2.8 Host Code . . . 11
2.9 Parameter Passing between Croqtile and C Function . . . 11
2.10 Input Parameters . . . 12
2.11 Passing a shaped buffer . . . 12
2.12 The host-side buffer wrapper . . . 12
2.13 A call built from runtime dimensions . . . 12
2.14 Return Type and Type Deduction . . . 13
2.15 Returning a shaped buffer . . . 13
2.16 The host-side result wrapper . . . 13
2.17 span and span(i) . . . 14
2.18 Declaring a shape . . . 15
2.19 A rank mismatch . . . 15
2.20 Omitting the mdspan keyword . . . 15
2.21 The original shape . . . 15
2.22 A derived shape . . . 15
2.23 Rebinding a shape . . . 16
2.24 A kernel with an unknown dimension . . . 16
2.25 The caller supplying that dimension . . . 16
2.26 Step Further: Symbolic Dimension . . . 17

2.27 Scalar Types (fragment) . . . 17
2.28 I-Tuple Types . . . 17
2.29 Operations over mdspan and i-tuple . . . 17
2.30 An initialized integer . . . 18
2.31 Assignment to an integer . . . 18
2.32 Several integers at once . . . 18
2.33 An integer parameter . . . 18
2.34 A declaration without initialization . . . 19
2.35 An i-tuple literal . . . 19
2.36 An inconsistent rank . . . 19
2.37 Assigning a new tuple . . . 19
2.38 Indexing an mdspan with an i-tuple . . . 20
2.39 An invalid shape expression . . . 20
2.40 Complete Types . . . 20
2.41 The Storage Qualifier . . . 21
2.42 A spanned-data declaration . . . 21
2.43 A standalone shape . . . 21
2.44 A rejected fundamental type . . . 22
2.45 Storage Specifier . . . 22
2.46 Initialization . . . 22
2.47 An explicit return shape . . . 22
2.48 A deduced return shape . . . 23
2.49 1. global . . . 23
2.50 2. shared . . . 23
2.51 3. local . . . 23
2.52 A declaration without a specifier . . . 24
2.53 Choosing local storage explicitly . . . 24
2.54 Example Code: Using Storage Qualifiers . . . 24
2.55 Croqtile's Two-Layer Parallelism Model . . . 25
2.56 : block — parallel blocks . . . 25
2.57 : thread — Threads Within a Block . . . 26
2.58 Nesting Specifiers . . . 26
2.59 Parallel Region: the parallel-by Block . . . 26
2.60 Indexing a with-in block . . . 27
2.61 A tuple of indices . . . 27
2.62 Indices derived from a tuple . . . 27
2.63 Nested bounded groups . . . 27
2.64 The foreach Block . . . 27
2.65 A named parallel index . . . 28
2.66 An unnamed parallel index . . . 28
2.67 One parallel level . . . 28
2.68 Two parallel levels . . . 29
2.69 Sub-Level Parallelism . . . 29
2.70 A foreach loop . . . 29
2.71 The equivalent C++ loop . . . 30
2.72 A bounded variable . . . 30
2.73 A tuple of bounded variables . . . 30
2.74 A variable derived from a tuple . . . 30
2.75 Indexing one shape . . . 31
2.76 A tuple of indices . . . 31

2.77 The generated C++ loop . . . 31
2.78 Several bounded groups . . . 31
2.79 The generated inner loop . . . 31
2.80 A foreach loop . . . 32
2.81 The with-in form it expands to . . . 32
2.82 A tuple of bounded variables . . . 32
2.83 The derived C++ loop . . . 32
2.84 The bounded-variable syntax . . . 32
2.85 chunkat(...): equal partitioning . . . 33
2.86 subspan(...).at(...): explicit tile extents on a tile grid . . . 33
2.87 view(...).from(...): explicit tile extents with explicit element offsets . . . 33
2.88 Compose: outer inner . . . 34
2.89 Extent: name . . . 35
2.90 Bounded-ituple/integer and chunkat Operation . . . 35
2.91 Ubound Operation . . . 36
2.92 Use chunkat Expression to Tile Data . . . 37
2.93 Use Multiple Bounded Variables . . . 38
2.94 The general DMA form . . . 38
2.95 A concrete copy . . . 38
2.96 Tiled Addition Through Shared Storage . . . 39
2.97 A future from a copy . . . 39
2.98 Reading data and span . . . 40
2.99 A synchronous copy . . . 40
2.100 An asynchronous copy . . . 40
2.101 An asynchronous copy . . . 40
2.102 dma.transp 41
2.103 dma.pad 41
2.104 The tileflow program . . . 42
2.105 The equivalent C++ loop . . . 42
2.106 Basic Syntax . . . 44
2.107 Synchronous and Asynchronous . . . 44
2.108 Data Expression . . . 44
2.109 Two asynchronous copies . . . 45
2.110 Waiting on both futures . . . 45
2.111 Parallel Perspective of Tileflow Programs . . . 46
2.112 Implicit Masking . . . 47
2.113 Declaring a shared event . . . 47
2.114 Declaring an event array . . . 47
2.115 Triggering an event array . . . 47
2.116 Waiting on an event . . . 48
2.117 Event arrays for multi-stage pipelines . . . 48
2.118 Bootstrap protocol . . . 48
2.119 Predicated execution with if . . . 49
2.120 Early return from parallel . . . 49
2.121 Tile geometry as macros . . . 51
2.122 Compile-time assertions with error . . . 51
2.123 Conditional code paths . . . 51
2.124 Calling C++ Functions . . . 51
2.125 An object-like macro . . . 52
2.126 A macro with parameters . . . 52

2.127Conditional Compilation . . . 53
2.128The general call syntax . . . 53
2.129An illegal call site . . . 53
2.130The Convention: Allowed Argument Types . . . 54
2.131A compile-time shape print . . . 55
2.132The printed shape . . . 55
2.133A runtime inspect kernel . . . 55
2.134A print inside a parallel block . . . 55
2.135assert: runtime invariant checks . . . 55
3.1 Transpilation and Compilation . . . 59
3.2 A device helper called from tileflow . . . 59
3.3 The cok wrapper for separate sources . . . 59
3.4 Heterogeneity: Transpiling to program launch . . . 60
3.5 Embedding Croqtile within C++ . . . 61
3.6 An mdspan declaration . . . 63
3.7 A shape literal . . . 63
3.8 A shape with one dimension changed . . . 63
3.9 The same shape again . . . 63
3.10 Values of Bounded Variables . . . 65
3.11 A kernel with anonymous dimensions . . . 67
3.12 The transpiled signature . . . 67
3.13 The generated shape check . . . 67
3.14 Cost and Limitations . . . 68
3.15 a. Improved Readability and Clarity . . . 68
3.16 b. Type Safety and Compile-Time Checking . . . 69
3.17 c. Expressiveness and Flexibility . . . 69
3.18 A shape expression . . . 70
3.19 Its value-number graph . . . 70
3.20 A constant expression . . . 71
3.21 The simplified graph . . . 71
3.22 The folded graph . . . 71
3.23 Explicit Masking/Condition . . . 74
3.24 Implicit Masking . . . 75
3.25 Experimental: Optional Where-Binding for within . . . 75
3.26 Masking for Multi-Levels . . . 76
3.27 Scalar Matmul (No DMA) . . . 77
3.28 Tile and Flow . . . 79
3.29 A window at a constant offset . . . 80
3.30 A window at a runtime offset . . . 80
3.31 Strided tiles: .subspan, .step, and .at . . . 81
3.32 A flat strip reinterpreted as a matrix . . . 82
3.33 A movement statement using the reinterpretation . . . 82
3.34 An asynchronous DMA declaration . . . 85
3.35 The same declaration with storage . . . 85
3.36 Operation Type . . . 86
3.37 Full Non-Blocking DMA Mode (Chain Mode in Croqtile) . . . 86
3.38 DMA Matmul: Tiles in Shared Storage . . . 87
3.39 Double buffering with swap . . . 88
3.40 Multi-Buffering Optimization with Croqtile . . . 89
3.41 Partial Tiles, Guarding, and .zfill . . . 92

3.42 .swiz N : Shared-Memory Layout for the Consumer . . . 93
3.43 A swizzled copy . . . 93
3.44 A swizzled mma load . . . 93
3.45 A sparse contraction . . . 94
3.46 A scaled contraction . . . 94
3.47 Another contraction layout . . . 94
3.48 One group, one tile: the simplest MMA matmul . . . 95
3.49 Structured concurrent regions with inthreads.async . . . 97
3.50 Inter-region signaling with events . . . 98
3.51 Make it Asynchronous . . . 100
3.52 Event Instances . . . 100
3.53 A single-instance deadlock . . . 101
3.54 A credit-count deadlock . . . 101
3.55 The canonical pattern: 1 producer 1 consumer . . . 101
3.56 A 1P1C matmul skeleton . . . 102
3.57 Persistent scheduling and the if guard . . . 103
3.58 parallel.async: host-level concurrency . . . 104
3.59 Asynchronous program launch and Stream Binding . . . 105
3.60 An asynchronous launch . . . 105
3.61 The synchronous form . . . 105
3.62 Stream Binding . . . 105
3.63 Stream Binding without .async . . . 106
3.64 The full 1P1C event pipeline . . . 106
3.65 Differences from C++ Preprocessing . . . 110
3.66 A templated device helper . . . 110
3.67 A call that instantiates it . . . 110
3.68 A cok block around device code . . . 111
3.69 The block after factoring target-specific details . . . 111
3.70 device functions: target alongside Croqtile . . . 111
3.71 Example Code: Matrix Multiplication with Tiling . . . 114
3.72 Example Code . . . 116

Chapter One

Getting Started

Chapter roadmap

This chapter takes you from installing Croqtile to a program you have built and run yourself. After the language's motivation, set up and verify the compiler, run a complete element-wise addition example, and use the command-line options to inspect its build. Finally, learn the tools for checking compilation commands, runtime values, and incorrect results.

1.1 What is Croqtile

Much of the development effort in a high-performance kernel goes into organising the computation. Beyond implementing the arithmetic, the programmer has to arrange data storage, divide parallel work and coordinate execution. These choices interact, so a change can require updates in several places. Different input sizes or target machines call for another round of validation and tuning. Implementing, maintaining and revising these arrangements accounts for much of the cost.

Croqtile is a C++ embedded DSL for high-performance kernel programming, designed to make this work easier while preserving control over performance. It provides a direct way to describe the organisation of computation, data movement and coordination in a compact program. The programmer chooses how the kernel works; the compiler derives related information, checks consistency and generates the underlying code. This reduces the repetitive work involved in expressing and maintaining those choices.

The resulting programs are easier to read as a whole and revise as requirements change. They can accommodate different input sizes, and the abstractions introduce no additional runtime overhead. Compact source and useful compiler feedback also make the same process accessible to AI-assisted development and tuning.

1.2 Setting Up: Requirements, Build, and Verification

Croqtile is built from source rather than installed as a prebuilt package. The compiler is itself a C++ program whose parser is generated from a grammar, so the first thing to check is the toolchain that builds it. The table below gives the versions that are known to work: a C++17-capable compiler and two parser-generator tools.

Dependency	Minimum Version
GCC	8.1+ (or Clang 6+)
Bison	3.8+
Flex	2.6.4+

Most Linux distributions ship Flex but an older Bison. If the Bison on your system is below 3.8, the setup step below fetches a compatible version on its own, so the distribution package does not have to be replaced by hand.

The version floor is not arbitrary. GCC 8.1 and Clang 6 are the first releases with complete C++17 support, and Bison 3.8 and Flex 2.6.4 are the versions whose generated code supports the C++ features the compiler uses. An older Bison is the usual obstacle, and it is the one the setup step resolves for you.

Clone the repository and run the automated setup:

```
git clone https://github.com/LancerLab/croqtile.git
cd croqtile
make setup-core
```

Listing 1.1: Clone and prepare

`make setup-core` pulls git submodules and downloads the required versions of Flex, Bison, FileCheck, and GoogleTest if they are not already present. Once setup finishes, build the compiler:

```
make
```

Listing 1.2: Build the compiler

Then run the test suite; it is the check that the freshly built compiler is sound:

```
make test
```

Listing 1.3: Run the test suite

If all tests pass, the `croq` binary is ready in your build directory. Add it to your `PATH` or invoke it by its full path. The next step compiles a minimal program to confirm the installation end to end.

The smallest useful check is a kernel that copies four integers. Save it as `verify.co`:

```
__co__ s32 [4] identity(s32 [4] input) {
  s32 [input.span] output;
  parallel i by 4
    output.at(i) = input.at(i);
  return output;
}
int main() {
  auto input = croq::make_spandata<croq::s32>(4);
  input[0] = 1; input[1] = 2; input[2] = 3; input[3] = 4;
  auto result = identity(input.view());
  for (int i = 0; i < 4; ++i)
    if (input[i] != result[i]) { std::cerr << "FAIL\n"; return 1; }
  std::cout << "OK\n";
}
```

Listing 1.4: A minimal Croqtile program

Compile and run:

```
croq verify.co -o verify
./verify
```

Listing 1.5: Compile and run the check

You should see `OK`. If you do, the compiler, linker, and runtime are all working. Section 1.3 lists the options that change what the command produces.

1.3 The Toolchain: Command Line and Build Utilities

The shortest program that shows the shape of the language is an element-wise addition of two small matrices. It has no tiling, no explicit data movement and no coordination between regions; those are the constructs every later example adds. What it does have is the full skeleton of a Croqtile program: a `__co__` function that describes the parallel work, and an ordinary C++ `main()` that prepares the inputs, calls it and checks the result. Every program in the guide is built from these two halves.

A Complete Example: Element-Wise Addition

Here is the full program. It adds two `[4, 8]` matrices of 32-bit integers element by element:

```
__co__ s32 [4, 8] ele_add(s32 [4, 8] lhs, s32 [4, 8] rhs) {
  s32 [lhs.span] output;
  parallel {i, j} by [4, 8]
    output.at(i, j) = lhs.at(i, j) + rhs.at(i, j);
  return output;
}
int main() {
  auto lhs = croq::make_spandata<croq::s32>(4, 8);
  auto rhs = croq::make_spandata<croq::s32>(4, 8);
  lhs.fill_random(-10, 10);
  rhs.fill_random(-10, 10);
  auto res = ele_add(lhs.view(), rhs.view());
  for (int i = 0; i < 4; ++i)
    for (int j = 0; j < 8; ++j)
      croq_assert(lhs[i][j] + rhs[i][j] == res[i][j],
                  "values are not equal.");
  std::cout << "Test Passed\n" << std::endl;
}
```

Listing 1.6: A complete element-wise addition

Save this as `ele_add.co`, compile, and run:

```
croq ele_add.co -o ele_add
./ele_add
```

Listing 1.7: Compile and run the example

You should see `Test Passed`. The chapters that follow take this same program apart, one construct at a time.

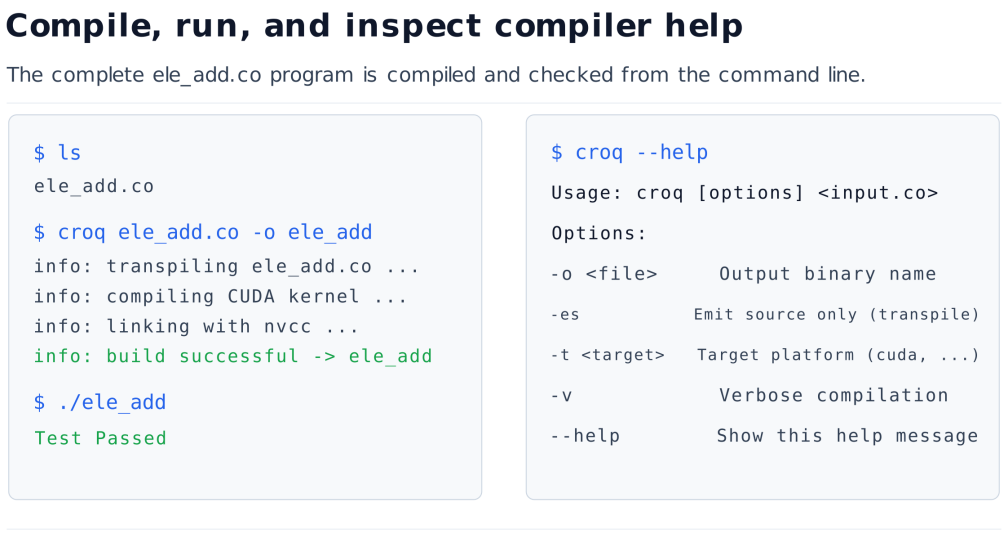


Figure 1.1: Compiling and running ele_add, followed by compiler help

Compiler Options

After running `ele_add`, you can use the same command pattern for other `.co` files. Croqtile first generates target source code, then invokes the target toolchain to compile and link it. By default the output is `a.out`; `-o` gives it an explicit name.

The following options select the output form or expose a step in this process:

Option	Purpose
`-o <file>`	Set the output filename.
`-t <platform>`	Select the target platform; list supported targets with `-help-target`.
`-es`	Emit target source without compiling it further.
`-E`	Emit preprocessed Croqtile source only.
`-c`	Compile to an object file without linking.
`-S`	Emit assembly when supported by the target toolchain.
`-gs`	Generate a script for subsequent compilation and execution.
`-v` / `-verbose`	Show the external commands invoked by the compiler.
`-help` / `-help-hidden`	List standard or advanced compiler options.

For example, you can inspect the generated source or the result of preprocessing before building an executable:

```
croq -t target -es ele_add.co -o ele_add.target.cpp
croq -E ele_add.co
croq --help
```

Listing 1.8: Inspecting intermediate output and compiler help

Replace `target` with a name reported by `croq --help-target`. Available output forms depend on the selected toolchain. Use `-v` when you need to see the actual compilation commands; Section 1.4 covers further build diagnostics.

Development Utilities

The commands above build your `.co` program. The repository's Makefile commands are useful when testing sample operators or changing the compiler itself:

```
make help
make sample-test
make sample-test-operator OPERATOR=add
```

Listing 1.9: Testing sample operators

`make help` lists the available targets. `make sample-test` runs all sample operators; `OPERATOR=add` restricts the run to addition. Other operator names include `mul`, `relu`, `sigmoid`, `softmax`, and `tanh`. With the program built, Section 1.4 explains how to inspect the compilation commands and check its execution.

1.4 Inspecting and Debugging a Build

Verbose mode: `-v`

Pass `-v` (or `-verbose`) to the compiler to see the external commands it invokes — which preprocessor, which code generator, which `c++` invocation:

```
croq kernel.co -v -o kernel
```

Listing 1.10: Verbose mode: -v

This is useful when you suspect the compiler is passing wrong flags to `c++` or when you need to see the exact generated file paths for manual inspection.

Runtime checks: `-rtc`

The compiler supports graduated runtime checking with `-rtc` (or `--runtime-check`):

```
croq kernel.co -rtc=high -o kernel
```

Listing 1.11: Runtime checks: -rtc

Levels: `none`, `entry` (default), `low`, `medium`, `high`, `all`. Each level is a superset of the previous. Use `high` or `all` during development, `none` for production.

Debug RTTI and `debugger`

When `print` / `println` is not enough, Croqtile supports debug RTTI (Runtime Type Information) that makes its types visible to `debugger`.

Compile with `-g` to enable debug symbols:

```
croq kernel.co -g -o kernel_debug
```

Listing 1.12: Debug RTTI and debugger

The generated code includes RTTI structs for Croqtile types:

```
debugger -q ./kernel_debug
(gdb) break my_kernel
```

Croqtile type	GDB type	Fields
Shaped tensor (`s32 [M, N]`)	`croq::rtti::spanned<int, 2>`	`.span.data[]` (dimensions), `.stride.data[]` (strides), `.data` (pointer)
Index tuple	`croq::rtti::bounded_ituple<N>`	`.data[]` (values), `.ub[]` (upper bounds)
Integer tuple	`croq::rtti::ituple<N>`	`.data[]` (values)
Multi-dim span	`croq::rtti::mdspan<N>`	`.data[]` (extents)

```
(gdb) run
(gdb) ptype __dbg_lhs
type = struct croq::rtti::spanned<int, 2>
(gdb) print __dbg_lhs.span.data[0]
$1 = 32
(gdb) print __dbg_lhs.span.data[1]
$2 = 64
(gdb) print __dbg_lhs.data != 0
$3 = true
```

Listing 1.13: Debug RTTI and debugger

The `__dbg_` prefix on variable names is generated by the compiler — it makes Croqtile variables visible alongside the generated C++ intermediates.

Putting it together

Choose the first check from the symptom. If a tile has the wrong dimensions, inspect its shape with `println!` before running the program. If the shape is right but the values are wrong, print one tile or add an assertion near the suspect operation. A stalled pipeline calls for event and iteration counts; a regular pattern of incorrect matrix elements calls for a layout check. Use a debugger when these checks point to an address or lifetime problem. Keep temporary output behind a preprocessor guard so it can be removed from a normal build:

```
#ifdef DEBUG_PRINT
  println("tile_k=", iv_k, " accum=", mc);
#endif
```

Listing 1.14: Guarding debug prints

```
croq kernel.co -DDEBUG_PRINT -o kernel    # debug build
croq kernel.co -o kernel                   # production build
```

Listing 1.15: Debug and production builds

Debugging MMA-heavy kernels

When the wrong answer comes from a tensor-core path, suspect **layout first** — row versus column major, and whether the RHS is `[N, K]` versus `[K, N]` in memory. Then check **indexing** (which `block_m`, `block_n`, and K slice you attached with `.at` / `chunkat`).

A common mistake is mislabeling `mma.row.row` when the staged data is actually column-major, or using `chunkat` indices that do not align with the MMA tile geometry. If you use swizzled loads (`dma.copy.swiz` / `mma.load.swiz`), a mismatch shows up as a regular pattern in the error (e.g., every sixteenth element correct, the rest shifted).

Debugging tip

Before reaching for the compiler's inspection flags, look at the program itself. If something looks wrong after editing a pipeline, verify **event order and trip counts** before chasing MMA layout bugs: producer and consumer must use the same `cdiv(K, TILE_K)` loop bound, and too few stages shifts pressure to `wait full` when the consumer outruns the producer.

Tool	Level	Cost
`print!` / `println!`	Compile-time	Free
`assert(expr, "msg")`	Runtime	Low — aborts on violation
`print` / `println`	Runtime	Medium — serialized `printf`
`-rtc=high`	Runtime	Medium — bounds checks
`-v` / `-verbose`	Compiler	Free — shows subprocess invocations
`-g` + `debugger` + RTTI	Runtime	High — debug symbols, no optimization

In short These tools observe different stages of the same program. Compiler output explains the shapes and types Croqtile inferred; runtime checks expose the values used in an actual call. Verbose mode is useful when the build itself fails, because it shows which external command failed and how to reproduce it. A successful build and a correct result are separate checks.

Once the small example passes, apply the same checks to a kernel you want to change. Keep a reference result, change one part of the program, and compare the new output before measuring performance. This makes it possible to connect a regression to a specific change.

Where to go next

With a program running, continue to Chapter 2 to learn what each construct in it means. If you already know the syntax, Chapter 3 is arranged for lookup.

Chapter Two

Language Basics

Chapter roadmap

You will learn to read and write Croqtile programs by working through short examples. Start with the structure of a complete program and the roles of its functions, then follow how shapes, iteration, tile selection, data movement and coordination are expressed. Each example adds a construct and explains its role, so you can build up a working understanding of the syntax.

2.1 The Two Parts of a Croqtile Program

At its simplest a program has two parts. A `__co__` function describes computation on shaped tensors, and the compiler translates that function into target code. The host program is standard C++ that prepares the data, calls the function and checks the result. Both live in a single `.co` file, and the `__co__` prefix is all the compiler needs to tell them apart.

Host, Device and Tileflow

Both of those parts are present here, but the ordinary C++ side is worth splitting once more: a `.co` file may also carry a `__device__` helper. The listing is written from the inside out — a device helper, the `__co__` function that calls it, and the host `main()` that calls the `__co__` function.

```
__device__ void device_function(...) {
  // High-performance device kernel implementation
}
// Tileflow program: orchestrating data movement
__co__ void croqtile_function(...) {
  // ... croqtile code ...
  device_function(...);
  // ...
}
// Host program: typically runs on CPU
void main() {
  // ... prepare data ...
  croqtile_function(...);
  // ...
}
```

Listing 2.1: The three regions of one source file

Execution starts at the bottom of the listing, in the host program. Written in standard C++ and run on the CPU, it is the entry point of the module and the caller of the Tileflow program — the `__co__` functions — and it manages the overall workflow of the heterogeneous application. A typical host body prepares the input data, invokes a Croqtile function, handles its return value and advances to the next step.

The Tileflow program called from there is the part Croqtile adds. Built from `__co__` functions, it is the core of a `.co` program: it orchestrates data movement between host and device and between storage levels within a single device. A typical Tileflow body moves data into a buffer at the appropriate storage level, calls a device function to compute on it, and moves the result back to the host. It is the only region the compiler translates.

Device functions sit at the other end of that call chain. A `__device__` function defines computation-intensive operations that run on the target device; the prefix marks it as device code. Like the host program, it passes through Croqtile compilation unchanged. The compiler leaves both sides of ordinary C++ alone; only the `__co__` function between them is rewritten.

2.2 Reading a Kernel Line by Line

The Croqtile Function

Start with the signature, which declares more than a C++ function prototype would: every parameter and the result carry a shape.

```
__co__ s32 [4, 8] ele_add(s32 [4, 8] lhs, s32 [4, 8] rhs) {
```

Listing 2.2: A kernel signature (fragment)

The `__co__` prefix marks a Croqtile function. The declaration `s32 [4, 8]` combines signed 32-bit elements with a two-dimensional shape: four rows and eight columns. Both inputs and the result follow the form `type [shape] name`. These declarations give the compiler the information needed to check ranks and reason about index bounds. Constraints that depend on caller-supplied dimensions may also require runtime checks.

Croqtile supports these element types: `s32` (signed 32-bit int), `f16` (half-precision float), `bf16` (brain float), `f32` (single-precision float), and `f8_e4m3` / `f8_e5m2` (8-bit floats). In the examples that follow, `s32` is the simplest to reason about.

```
s32 [lhs.span] output;
```

Listing 2.3: Allocating the output

This allocates the output tensor. The expression `lhs.span` copies the full shape from `lhs`, so `output` automatically has shape `[4, 8]`. If you later change the input shape, the output adjusts — a pattern that makes Croqtile functions easier to generalize.

```
parallel {i, j} by [4, 8]
  output.at(i, j) = lhs.at(i, j) + rhs.at(i, j);
```

Listing 2.4: The parallel body

The statement `parallel {i, j} by [4, 8]` declares 32 independent instances, one for each element position. An instance with `(i, j) = (1, 3)` reads that position from both inputs and writes the sum to the same position in `output`. No two instances write the same

element, and none needs a result produced by another instance. That independence is what permits parallel execution.

Without a space specifier, the compiler chooses how to map the instances to execution resources. The statement establishes their independence; it does not require one physical thread per element or all instances to run at once.

```
return output;
```

Listing 2.5: Returning the result

A `__co__` function returns its result tensor. The return type in the signature (`s32 [4, 8]`) must match what you actually return. The caller on the host side receives a `croq::spanned_data` — an owning buffer with shape metadata that you can index into with `[i][j]`.

`__co__ auto` return type

Use `auto` when the result shape is easiest to describe inside the function. In `__co__ auto matmul(...)`, the compiler infers the return type from the buffer named by `return output`. The result still has an element type and a shape; `auto` saves repeating that information in the signature. An explicit return type is useful when you want the signature itself to state the required output shape.

The return convention is the same for element-wise arithmetic, scalar matrix multiplication, and MMA kernels that return a buffer. It does not depend on how the function computes the elements.

2.3 The Host-Side API: Inputs and Outputs

The Host Program

The host supplies both storage and shape information. In the addition example, it first creates two owning buffers, fills them, and passes views to the kernel. Keeping ownership on the host lets the same input buffers remain available for checking the result after the call.

```
auto lhs = croq::make_spandata<croq::s32>(4, 8);
auto rhs = croq::make_spandata<croq::s32>(4, 8);
lhs.fill_random(-10, 10);
rhs.fill_random(-10, 10);
```

Listing 2.6: Creating the shaped inputs

The call `croq::make_spandata<T>(dims...)` allocates an owning host buffer. The template parameter selects the element type, and the arguments supply its dimensions. The calls to `fill_random` then initialize its elements within the specified range.

```
auto res = ele_add(lhs.view(), rhs.view());
```

Listing 2.7: Calling the kernel

Calling a `__co__` function from C++ looks like a normal function call. The `.view()` method produces a non-owning `spanned_view` from an owning `spanned_data` — this is how you pass host tensors into a Croqtile function without transferring ownership. The return value `res` is a `croq::spanned_data` that owns its buffer.

The rest of `main` verifies the result by iterating over every element and asserting equality. The host program handles setup and verification; the computation takes place in the `__co__` function.

Host Code

The host side uses the same `make_spandata` / `.view()` pattern used by the host code above:

```
int main() {
  auto lhs = croq::make_spandata<croq::s32>(128, 256);
  auto rhs = croq::make_spandata<croq::s32>(256, 256);
  lhs.fill_random(-10, 10);
  rhs.fill_random(-10, 10);
  auto res = matmul(lhs.view(), rhs.view());
  for (size_t i = 0; i < res.shape()[0]; ++i)
    for (size_t j = 0; j < res.shape()[1]; ++j) {
      int ref = 0;
      for (size_t k = 0; k < lhs.shape()[1]; ++k)
        ref += lhs[i][k] * rhs[k][j];
      croq_assert(ref == res[i][j], "values are not equal.");
    }
  std::cout << "Test Passed\n" << std::endl;
}
```

Listing 2.8: Host Code

The loop after the call checks every element against a plain C++ reference computation, so an incorrect result is reported before the program prints anything.

Parameter Passing between Croqtile and C++ Function

Passing arguments between Croqtile and C++ requires the `choreo.h` header. Combine a raw C++ pointer with its dimensions to construct Croqtile spanned data, as shown below.

```
#include "choreo.h"
void bar(const float* data, unsigned size) {}
__co__ void foo(f32 mdspan<2> d) {
  parallel p by 6 {
    call bar(d, |d|);     // Call the C++ function
  }
}
void foobar(float* a) {
  foo(croq::make_spanview<2>(a, {1, 2}));
}
```

Listing 2.9: Parameter Passing between Croqtile and C Function

The `make_spanview` utility template wraps the data without copying it. In the Croqtile function `foo`, the corresponding parameter `d` has type `f32 mdspan<2>`. An implicit conversion allows `d` to be passed as the first argument to the C++ function `bar`; `|d|` supplies its total element count.

Scalar values can also pass between C++ and Croqtile functions. An `ituple`, however, is used only inside a Croqtile function.

Input Parameters

A Croqtile function accepts only two kinds of input parameter, the *spanned data* and the *integer*.

Similar to any C++ program, the parameter may either be named or unnamed:

```
__co__ void foo(f32 [7] a, int) {...}
```

Listing 2.10: Input Parameters

The second parameter is unnamed, so the function body cannot refer to it. The declaration is still valid.

C++ Host: Data Argument with Shape

Croqtile spanned data carries a shape in addition to its data pointer. Before passing a C/C++ pointer or array to a Croqtile function, wrap it with the required shape information.

One direct method is to use the Croqtile API provided in `choreo.h`. A program does not have to include `choreo.h` explicitly in a Croqtile compilation environment; the compiler includes it for every Croqtile program.

```
__co__ void foo(f32 [7, 16] input, f32 [7, 16] output) {...}
void entry(float * input, float* output) {
  // foo(input, output); <-- result in error since missing shape
  foo(croq::make_spanview<2>(input, {7, 16}),
      croq::make_spanview<2>(output, {7, 16}));
}
```

Listing 2.11: Passing a shaped buffer

The `croq::make_spanview` function template takes the rank as a template argument and the typed pointer and shape list as function arguments. It returns a `croq::spanned_view` that wraps the pointer and dimension list, making them a valid Croqtile argument. Its simplified definition is:

```
template <typename T, size_t Rank>
class spanned_view {
  T* ptr;                 // typed pointer to the data
  size_t dims[Rank];      // ranked dimensions
};
```

Listing 2.12: The host-side buffer wrapper

Dimensions do not have to be constants; runtime values work as well:

```
__co__ void foo(f32 [7, 16] input) {...}
void entry(float * input, float* output, int M, int N) {
  foo(croq::make_spanview<2>(input, {M, N})),
}
```

Listing 2.13: A call built from runtime dimensions

Here, `input` has a runtime shape. On entry to the Tileflow program, Croqtile checks that `M == 7` and `N == 16`. **If either condition fails, the program terminates immediately**, because the function's shape assumptions do not hold.

Spanned-data inputs are **references to the original data or buffers**. Calling a Croqtile function does not copy the input data, even though the parameter syntax may resemble a declaration with value semantics.

Return Type and Type Deduction

Similar to the input, *spanned data* and *integer* are valid output types. The *void* type is also allowed if nothing is returned.

In addition to the explicit return type annotation, Croqtile supports type inference on returns. For example:

```
__co__ auto foo() {
  f32 [7, 16] result;
  // ...
  return result;
}
```

Listing 2.14: Return Type and Type Deduction

Here, `auto` is inferred as `f32 [7, 16]` from `result`. This is useful when the output shape is derived inside the function: changing that derivation does not require a matching edit to the return annotation. Use an explicit type when a fixed return shape is part of the interface you want to enforce.

C++ Host: Shape Return

The C++ host code that receives the output of Croqtile function actually receives a buffer. The return has value semantics: the caller receives the buffer itself, not a reference to an object allocated inside the called function. For example:

```
__co__ auto foo() {
  f32 [7, 16] output;
  // ...
  return output;  // semantically copy
}
void entry() {
  auto result = foo(); // move the 'spanned_data' to caller
  // ...
}
```

Listing 2.15: Returning a shaped buffer

The example returns `output` with value semantics, but the implementation avoids copying the buffer. The returned C++ type is `croq::spanned_data`, whose simplified definition is:

```
template <typename T, size_t Rank>
class spanned_data {
  std::unique_ptr<T[]> ptr; // move only pointers
  size_t dims[Rank];        // ranked dimensions
};
```

Listing 2.16: The host-side result wrapper

Unlike `croq::spanned_view`, `croq::spanned_data` owns the buffer it points to. **Returning it does not copy the buffer**; the implementation uses return value optimization.

Use member functions such as `.rank()` and `.shape()` to query the returned buffer's shape. The host can then work with the result without depending on the implementation of the Croqtile function.

2.4 Shapes and mdspan

`span` and `span(i)`

The member `tensor.span` returns the whole shape, while `tensor.span(i)` returns the extent of dimension i. For matrix multiplication, the output takes its row count from the left input and its column count from the right input:

```
1 s32 [lhs.span(0), rhs.span(1)] output;
```

Listing 2.17: span and span(i)

The example builds a result whose dimensions come from two different operands. Reusing those dimensions keeps the output declaration tied to its inputs. The same approach is useful for a staging buffer that must match the shape of a moved tile.

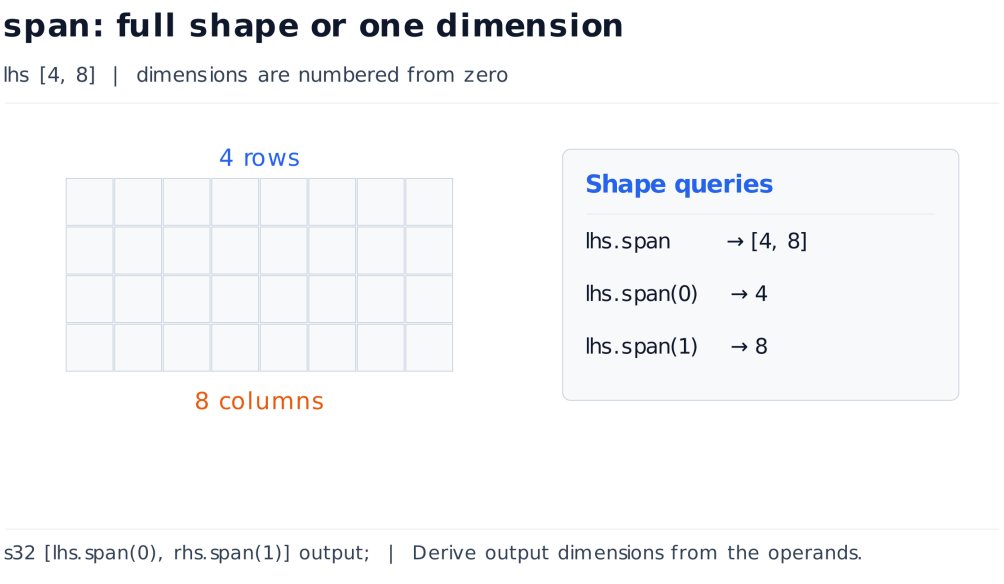


Figure 2.1: For a [4, 8] tensor, .span returns [4, 8], while .span(0) and .span(1) return 4 and 8.

A shape in Croqtile is a value a program can name and compute with, not just an annotation on a declaration. The examples below define one, derive another from it, and leave a dimension open for the caller to fill in.

Defining Shapes with `mdspan`

The keyword `mdspan` stands for multidimensional span. It describes the extents of a shape without allocating its elements. For example, a shape `[7, 8]` has rank 2 and describes 56

positions; choosing `f32` or `s32` for those positions is a separate decision.

You can explicitly define **`mdspan`** variables as follows:

```
mdspan s0 : [7, 8]; // Defines a 2D shape with dimensions [7, 8]
mdspan<1> s1 : [3]; // Defines a 1D shape with dimensions [3]
```

Listing 2.18: Declaring a shape

The colon introduces the shape initializer, and square brackets enclose its dimensions. Thus `s0` has seven rows and eight columns, while `s1` has one dimension of size 3. A named shape can be reused in later buffer declarations.

You can specify the **rank** by placing **`<rank>`** after **`mdspan`**. The compiler checks that this rank matches the initialization expression and reports a compilation error if they differ:

```
mdspan<3> s2 : [64, 32]; // error: the rank of mdspan is inconsistent
```

Listing 2.19: A rank mismatch

Because the initializer identifies a shape, the compiler can infer the `mdspan` type when the keyword is omitted:

```
s3 : [7, 8, 9]
```

Listing 2.20: Omitting the mdspan keyword

Here `s3` has rank 3 and dimensions 7, 8, and 9. Every named shape requires an initializer, so the inferred form is often sufficient.

Deriving *mdspan*s

New shapes often derive from existing ones. **Tiling** or **blocking** divides dimensions, while **padding** increases selected dimensions by a specified amount.

Shape arithmetic expresses these derivations directly:

```
shape : [128, 64]; // initial shape
new-shape0 : shape [(0) / 2, (1) / 4, 1];  // tile and reshape: [1, 64, 16]
new-shape1 : shape [(1) + 2, (0) / 16];    // pad and reshape: [66, 8]
```

Listing 2.21: The original shape

The first derived shape divides dimension 0 by 2 and dimension 1 by 4, then adds a dimension of size 1. The expression determines the new dimension order. It changes the shape description; it does not move any elements.

In Croqtile, the definition of **`new-shape0`** is equivalent to:

```
new-shape0: [shape(0) / 2, shape(1) / 4, 1];
```

Listing 2.22: A derived shape

This form lists the dimensions explicitly inside **`[]`**. The **element-of** operation **`()`** retrieves a dimension from the existing shape. Both forms produce the same result; the earlier form is syntactic sugar for the complete initialization expression.

The second derived shape adds 2 to the original dimension 1 and divides dimension 0 by 16, then places them in that order.

Furthermore, you may use the **`mdspan`** as a whole for derivations:

```
shape : [32, 72]
new-shape0 : shape;  // [32, 72]
new-shape1 : shape + 1;  // [33, 73]
new-shape2 : shape / 4;  // [8, 18]
new-shape3 : [shape, 6]; // [32, 72, 6]
```

Listing 2.23: Rebinding a shape

Arithmetic on a whole shape applies to each dimension. Adding 1 to `[128, 64]` therefore gives `[129, 65]`; it does not append a dimension. The two expressions below spell out that same calculation:

The expression `new-shape1 : shape + 1;`
is equivalent to
The expression `new-shape1 : shape [(0) + 1, (1) + 1];`

The derived definition of `new-shape3` demonstrates that using an *mdspan* variable in an *mdspan initialization expression* results in **concatenation** behavior. Thus, the declaration

The expression `new-shape3 : [shape, 6];`
is equivalent to
The expression `new-shape3 : shape [(0), (1), 6];`
The shorter form is syntactic sugar for the complete shape definition.

mdspan with Anonymous Dimension

Croqtile supports shape dimensions with unknown values, similar to many high-level machine learning languages. The simplest method is to use **Anonymous Dimension** in `mdspan`:

```
__co__ auto foo(s32 [?, 1, 2] input) { ... }
```

Listing 2.24: A kernel with an unknown dimension

Here, the question mark `?` represents a compile-time unknown value. Any positive integer could be assumed here. However, the value must be provided at runtime, as the following example shows:

```
void bar(int * data, int dim0) {
  foo(croq::make_spanview<3>(data, {dim0, 1, 2}));
}
```

Listing 2.25: The caller supplying that dimension

The first dimension of `input` comes from the spanned input passed to `foo`; `bar` obtains the value from its caller at runtime. For example, it may come from a configuration file or a shape calculation in a machine learning framework.

An anonymous dimension is sufficient when only its size matters within one input. It does not give two inputs a shared name for a dimension that must agree. Matrix multiplication needs precisely that relationship: the left operand's column count must equal the right operand's row count.

Step Further: Symbolic Dimension

In Croqtile, **symbolic dimensions** give names to dimensions whose values are unknown at compile time. For example:

```
__co__ auto Matmul(s32 [M, K] lhs, s32 [K, N] rhs) {
  int tile_m = 32, tile_m = 8, tile_k = 16;
  s32 [M / tile_m, K / tile_k] tiled_lhs;
  s32 [K / tile_k, N / tile_n] tiled_rhs;
  // ...
}
```

Listing 2.26: Step Further: Symbolic Dimension

Each input dimension has a symbolic name: `M`, `K`, or `N`. Unlike anonymous dimensions, these names make relationships between shapes explicit, improving readability and maintenance. They also give the compiler more information for consistency checks.

2.5 Integers, I-Tuples, and Bounded Values

Scalar Types

Croqtile provides **integer** and **Boolean** scalar types. Its integer type is a signed 32-bit value, like C++ `int` or `int32_t`, with range -2^{31} to $2^{31} - 1$. The following example uses it in variable and function declarations.

```
int a;
__co__ int foo(int b);
```

Listing 2.27: Scalar Types (fragment)

Integers support arithmetic, shifts, and logical operations with C++ syntax. Croqtile does not provide unsigned or 8-, 16-, or 64-bit scalar control integers: a signed 32-bit integer is sufficient for the program-control tasks this type serves.

The Boolean type corresponds to C++ `bool`. Boolean operations and conversions between Boolean and integer values follow C++ rules.

I-Tuple Types

An integer tuple is an ordered collection of integers, commonly used as a multidimensional index.

To define an i-tuple, simply enclose its elements within '{' and '}' braces. That is,

```
ituple index = {5, 4, 3, 2, 1};  // It defines a tuple of 5 elements
index = {a, b};  // 'a' and 'b' are existing integers
```

Listing 2.28: I-Tuple Types

Operations over *mdspan* and *i-tuple*

Croqtile supports operations between `ituple` and `mdspan` values. The following example applies fixed tiling factors to a shape:

```
sp : [6, 8];
tiling_factor = {3, 2};
spn : sp / tiling_factor;   // spn is defined as [2, 4];
```

Listing 2.29: Operations over mdspan and i-tuple

In Croqtile, mdspan can be defined with such **Tuple-Span Operations**. The supported operations include:

- *mdspan* / *i-tuple*
- *mdspan* + *i-tuple*
- *mdspan* % *i-tuple*
- *mdspan* ∗ *i-tuple*
- *mdspan* − *i-tuple*

Each operation applies to corresponding dimensions. Tuple-span notation collects those calculations into one expression, so the relationship between a shape and its tiling factors remains visible.

Integers in a kernel control loops and index arrays, and an *i-tuple* groups several of them into one index. Both are declared with an initializer and both are checked against the shape they are used with.

Integers for Program Control

Control integers describe loop ranges, indices, and related calculations. Croqtile uses a signed 32-bit `int` for these tasks. Numerical data stored in a tensor uses the separate fundamental element types described in Section 2.6.

In Croqtile, an **integer**-typed variable is defined similar to C/C++:

```
int a = 1;
```

Listing 2.30: An initialized integer

Croqtile can also infer the type from the *initialization expression*, so an integer variable can be declared in another style:

```
a = 1;
```

Listing 2.31: Assignment to an integer

Similar to C/C++, you can define multiple integers in a single line:

```
int a = 1, b = 2, c = 3;
d = 4, e = 5;
```

Listing 2.32: Several integers at once

In Croqtile, the *mdspan*-typed shape cannot be used as the function parameter. However, integers can be passed from the host to the Tileflow program, and from the Tileflow program to the device:

```
__co__ void foo(int a) {
  // ...
  call kernel(a, 3);
}
```

Listing 2.33: An integer parameter

Unlike C/C++, Croqtile does not allow the declaration of integers without initialization or the reassignment of an integer. For example:

```
int a;  // error: declaration without initialization
int b = 1;
b = 3;  // error: re-assignment
```

Listing 2.34: A declaration without initialization

This enforces that **integers must be initialized at declaration and cannot be re-assigned**. In some contexts, these integers are referred to as "**immutable**" or "**constant**".

Defining I-Tuples

An integer tuple, written `ituple`, groups several integer values in a fixed order. Its initializer uses braces:

```
ituple a = {1, 2, 3};
b = {4, 5, 6};  // utilize the type inference
```

Listing 2.35: An i-tuple literal

Since Croqtile can infer types from the *initialization expression*, programmers can often omit the `ituple` keyword. However, without explicit type annotation, an *ituple* variable definition might look similar to an *mdspan* definition if you are not yet familiar with Croqtile. To distinguish them:

- The *initialization expression* of an *ituple* follows an assignment operation `=` (like integers), whereas an *mdspan* is initialized after `:`;
- The *initialization expression* of an *ituple* is enclosed by `{}`, not `[]` as for *mdspan*.

Similar to `mdspan`, you may enforce rank check for *ituple*s at compile time:

```
ituple<3> a = {1, 2};  // error: inconsistent rank
```

Listing 2.36: An inconsistent rank

Operations on I-Tuples

Operations on *i-tuple*s are similar to those on *mdspan*. You can either use the *element-of* operation `()` to retrieve the element values or use *ituple* as a whole:

```
a = {3, 4};
b = {a(0), 1, a(1)};  // '()' to retrieve the element value
c = a {(0), (1), 2};  // syntax sugar
d = {a, 5, 6};        // concatenate
e = a + 1;            // addition is applied elementwise
```

Listing 2.37: Assigning a new tuple

Similar to *integer* and *mdspan*, an *ituple* variable must be initialized and cannot be reassigned.

In practice, *ituple*s are often used with *mdspan*s. For example:

```
shape : [7, 18, 28];
tiling_factors = {1, 2, 4};
tiled_shape : shape / tiling_factors;
padded_shape : shape + {2, 0, 2};
```

Listing 2.38: Indexing an mdspan with an i-tuple

Here, dividing an `mdspan` shape by an `ituple` of tiling factors produces `tiled_shape`. Adding an anonymous `ituple` to the original shape produces `padded_shape`. The shape and tuple must have matching ranks; otherwise, the compiler reports an error.

```
shape : [7, 8, 9] + {1, 2}; // error: inconsistent rank
```

Listing 2.39: An invalid shape expression

In short Use an integer for one control value and an `ituple` for an ordered group of values. A shape such as `[7, 14]` describes ranges, while a tuple such as `{1, 2}` can describe a position or a pair of tiling factors. The matching ranks let shape arithmetic apply each factor to the corresponding dimension.

Both integers and ordinary tuples must be initialized and cannot be reassigned. This restriction concerns the names used for control and shape calculations. A buffer element is different: the addition example writes to `output.at(i, j)`, and a matrix multiplication updates its accumulator. Keeping these roles distinct avoids treating an immutable index as though it were a mutable accumulator.

2.6 Spanned Data and Storage Specifiers

Fully-Typing

A buffer declaration needs both an element type and a shape. The shape determines how many positions the buffer contains, while the fundamental type determines what each position stores. The examples below reuse a named shape for half-precision data and use a literal shape for single-precision data.

```
ndims : [20, 15];
f32 [10, 10] d0;
f16 [ndims] d1;
```

Listing 2.40: Complete Types

The basic element types include:

- Unsigned Integers: *u8/u16/u32*
- Signed Integers: *s8/s16/s32*
- Floating-points: *f16/bf16/f32*

The types `s32` and `int` serve different purposes in Croqtile. The fundamental element type `s32` must be combined with a shape to declare a fully typed buffer.

The Storage Qualifier

A tile may be shared by cooperating threads or used privately by one thread. Its element type and shape do not express that distinction; a storage specifier does.

The three storage specifiers used in these examples are:

- **global**,
- **shared**,
- and **local**.

For example:

```
ndims : [20, 15];
local f32 [10, 10] d0;
shared f16 [ndims] d1;
```

Listing 2.41: The Storage Qualifier

Choose the storage specifier to match the code that uses the buffer. Defaults depend on the execution context and target; use an explicit specifier when a transfer or a cooperating group requires a particular storage location.

A Croqtile Tileflow program describes data movement, so buffer declarations are fundamental. Each declaration specifies an element type, a shape, and, when needed, a storage location.

Defining a *Spanned Data*

Defining a *spanned data* requires both the element type and its shape. For example,

```
f32 [32, 16] data;
```

Listing 2.42: A spanned-data declaration

This defines a buffer with a shape of `[32, 16]`, where the element type is `f32`, which is the *IEEE-754* single-precision float. It is also possible to use a named `mdspan` to define the buffer:

```
shape : [72, 14];
s32 [shape] data0;              // using the named mdspan 'shape'
u32 [shape(0), shape(1)] data1; // same shape as the above
u16 [shape + 6] data2;          // shaped as [78, 20]
s16 [shape, 8] data3;           // shaped as [72, 14, 8]
```

Listing 2.43: A standalone shape

This example uses the named `mdspan` `shape` to declare the two-dimensional buffer `data0` with 32-bit integer elements. Place the shape inside `[]`; element access, arithmetic, and concatenation can also appear in the shape expression.

In Croqtile terminology, the element type is referred to as the **fundamental type**. Croqtile supports multiple fundamental types, including:

- **Unsigned Integers**: `u8`, `u16`, `u32`
- **Signed Integers**: `s8`, `s16`, `s32`

- **Floating Points**: `f16`, `bf16`, `f32`

The prefixes `u` and `s` mean unsigned and signed, and the numeric suffix gives the bit width. The type `f16` is IEEE 754 half precision; `bf16` is the 16-bit brain floating-point format commonly used in machine learning.

The element type `s32` and the control type `int` have different roles, even though both describe 32-bit integers. An `s32` declaration needs a shape. The following standalone declaration is rejected:

```
s32 a;  // error: the fundamental type can not be used for variable definition alone
```

Listing 2.44: A rejected fundamental type

This results in a compile-time failure. Consequently, `s32` cannot be used for program control like `int`, and type conversion between them is not possible.

In practical code, some buffer declarations must specify a **storage specifier**, like so:

```
global f32 [32, 7, 2] a;
shared u8 [512, 144] b;
local u8 [72, 1024] c;
```

Listing 2.45: Storage Specifier

Since Croqtile handles storage in a heterogeneous context, a buffer definition without a storage specifier defaults to the storage type of the host program, i.e., CPU memory. Other storage specifiers are defined by the target.

- *global*: Refers to the device's global storage.
- *shared*: Refers to the device block's shared storage
- *local*: Refers to the thread-private storage.

Other targets may have different definitions. Furthermore, buffers with different storage types have limitations in their declarations. You will explore this in later chapters.

Initialization

It is possible to initialize a buffer at the declaration site. For example:

```
local s32 [17, 6] b1 {0};       // elements are initialized to 0
shared f32 [128, 16] b2 {3.14f}; // elements are initialized to 3.14f
```

Listing 2.46: Initialization

The braces after the buffer name contain one initial value, applied to every element. This form fills the whole buffer; it is not an element-by-element initializer list.

Declaring the Parameters and Return Values

Spanned data can appear in the parameter and return declarations of a Croqtile function:

```
__co__ f16 [7, 8] foo(f32 [16, 17, 5] input) {...}
```

Listing 2.47: An explicit return shape

Parameter and return declarations resemble variable declarations, but they do not allow operations on an existing `mdspan`, storage specifiers, or initializers.

One useful built-in member function for the *spanned* parameters is `.span`, which provides the associated *mdspan* of the *spanned data*. The following code demonstrates how to use it for a buffer declaration:

```
__co__ auto foo(f32 [16, 17, 5] input) {
  f32 [input.span / {4, 1, 5} ] buffer;  // declare a buffer with tiled shape
  // ...
}
```

Listing 2.48: A deduced return shape

In short A declaration determines the element type and shape, an initializer fills the buffer, and a storage specifier selects its location. The compiler manages the buffer's lifetime. Dynamic shapes extend these declarations to dimensions supplied at runtime.

1. `global`

A `global` declaration places the buffer in the target's global storage. It is useful for data that must remain accessible beyond one thread or one cooperating block. Declaring the storage does not initialize its contents or copy data into it:

```
global f32 [100, 200] matrix;
```

Listing 2.49: 1. global

2. `shared`

A `shared` buffer provides a common staging area for threads in a block. In a tiled matrix multiplication, a block can load an operand tile once and let several threads read it. The reads must follow completion of the load, and the next load must wait until those reads finish.

```
shared f32 [10, 10] tile_data;
```

Listing 2.50: 2. shared

3. `local`

It is particularly useful when each thread requires a small amount of data, such as scalar values or small arrays that do not need to be shared with other threads. Example:

```
local f16 [5] thread_local_data;
```

Listing 2.51: 3. local

Default Behavior and Usage

Omitting the specifier leaves storage selection to the target and the surrounding execution context. This is convenient for introductory code, but a name without a specifier does not

by itself tell you which threads can share its contents. When following a data path, check the declaration together with the parallel region that contains it.

```
f32 [100, 100] d0;
```

Listing 2.52: A declaration without a specifier

If you want to place data in a different memory space (shared or local), you need to specify the appropriate storage qualifier.

```
local f32 [10, 10] d0;      // Small data, local to the thread
shared f16 [ndims] d1;      // Intermediate data, shared between threads in a
    block
```

Listing 2.53: Choosing local storage explicitly

Example Code: Using Storage Qualifiers

The following sketch combines the storage declarations:

```
ndims : [20, 15];
local f32 [10, 10] d0;
shared f16 [ndims] d1;
global f32 [100, 200] matrix;
// Perform computations on these arrays, utilizing the appropriate storage
    space
dma.copy d0 => d1;
dma.copy d1 => matrix;
```

Listing 2.54: Example Code: Using Storage Qualifiers

The declarations illustrate three storage locations. The copy lines sketch the direction of movement between them. To turn this sketch into a working transfer, select source and destination views with compatible shapes: the full arrays declared here have different sizes. A storage specifier chooses a location; it does not make differently shaped buffers interchangeable.

2.7 Iteration: parallel-by, with-in, and foreach

Croqtile's Two-Layer Parallelism Model

Croqtile gives you two separate knobs:

1. `parallel` — declares that iterations are independent and *may* run concurrently. This is a logical statement.

2. **Space specifiers** (`: block`, `: thread`, `: group`, `: thread`) — tell the compiler where each parallel axis maps. This is a physical statement.

There is also `foreach`, which is the opposite of `parallel`: iterations run **sequentially**, in order. Use `foreach` when later iterations depend on earlier ones — an accumulation loop, a K-dimension reduction, a pipeline that loads tile k before computing with it.

```
parallel {px, py} by [8, 16] : block
  foreach {tile_k} in [16]                // sequential -- K-dimension loop
    parallel {qx, qy} by [16, 16] : thread  // concurrent -- mapped to threads
      foreach k in [16]                      // sequential -- inner accumulation
        output += lhs * rhs;
```

Listing 2.55: Croqtile's Two-Layer Parallelism Model

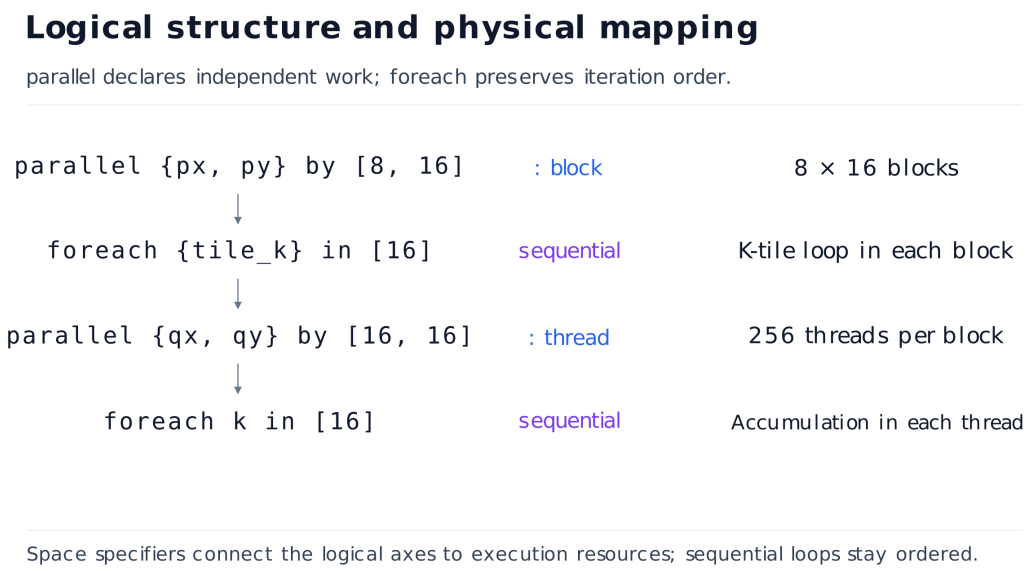


Figure 2.2: Space specifiers map parallel axes to blocks and threads; foreach loops remain sequential within that structure.

If you omit the space specifier, as in `parallel p by 8`, the compiler chooses a default mapping. The introductory examples use this form. Explicit specifiers let you control the mapping when performance requires it.

Both `parallel` and `foreach` can use multidimensional indices. Once the outer and inner ranges are established, the compose operator `#` connects their coordinates to positions in the full tensor. The bounds are part of that calculation, which is why changing the iteration geometry also changes tile selection.

`: block` — parallel blocks

```
parallel {px, py} by [8, 16] : block
```

Listing 2.56: : block — parallel blocks

This declares an 8 by 16 grid, giving 128 blocks. Each block must be able to complete its assigned output tile independently; the example does not provide a communication or synchronization protocol between blocks.

Use `: block` for the outermost tiling of your output. If your output is `[128, 256]` and each block handles a `[16, 16]` tile, you need `128/16 x 256/16 = 8 x 16 = 128` blocks.

: thread Threads Within a Block

```
parallel {qx, qy} by [16, 16] : thread
```

Listing 2.57: : thread — Threads Within a Block

This creates $16 \times 16 = 256$ threads within the enclosing block.
Use : thread for the finest-grain parallelism — the per-element work within a tile.

Nesting Specifiers

Real kernels nest these levels. A common pattern:

```
parallel {px, py} by [8, 16] : block {
  parallel {qx, qy} by [16, 16] : thread {
    // 128 blocks x 256 threads = 32,768 threads total
  }
}
```

Listing 2.58: Nesting Specifiers

The outer `parallel` creates the block grid. The inner `parallel` creates threads within each block. The braces `{px, py}` introduce multi-dimensional indices — shorthand for two nested `parallel` lines.

with tile_k in 16

The statement `with tile_k in 16` gives `tile_k` the range from 0 through 15, with upper bound `#tile_k = 16`. It does not run the body 16 times. A later `foreach tile_k` performs that iteration. Separating the binding from the loop lets a pipeline select tile 0 before the loop and then iterate over the remaining tiles.

foreach tile_k(1:): sliced iteration

The range expression `(1:)` starts iteration at index 1 and keeps the original exclusive upper bound. With 16 tiles, it visits 1 through 15. This is useful after a prologue has loaded tile 0: the loop issues each remaining load once, while computation consumes the previously loaded tile. The final loaded tile is handled after the loop.

Parallel Region: the 'parallel-by' Block

Systems such as CPUs can use asynchronous threads for parallel execution. Croqtile expresses parallel work with the Single Program, Multiple Data (SPMD) model.

The syntax of constructing a parallel region is quite different. It employs the C-style bracket and encloses the code for parallel execution within the 'parallel-by' block.

```
parallel p by 6 {
  // SPMD code
}
```

Listing 2.59: Parallel Region: the parallel-by Block

The `parallel` and `by` keywords declare six execution instances. Each runs the same SPMD code with a different value of `p`. You can think of `p` as the index of a six-iteration loop whose iterations are independent and may execute in any order.

The declaration also associates `p` with the range `[0, 6)`, making it a **bounded integer**. Operations such as `chunkat` use both its current value and its bound.

The 'with-in' Block and 'where' Clause

A `with-in` statement binds indices to a range without launching parallel instances. For a two-dimensional range, it can bind one tuple:

```
with index in [10, 10] {
  // index is ituple with 2 elements
}
```

Listing 2.60: Indexing a with-in block

Here, `index` is an `ituple` with two elements. You can also name the elements individually:

```
with {x, y} in [10, 10] {...}
```

Listing 2.61: A tuple of indices

The tuple and its components can also be named together:

```
with index = {x, y} in [10, 10] {...}
```

Listing 2.62: Indices derived from a tuple

You name 'index' as a **bounded ituple** in such scenarios.

A `with-in` statement creates bounded indices but does not introduce parallel execution. Its body executes sequentially unless it contains another construct that introduces parallel work.

In addition, programmers could append a 'where' clause. For example,

```
with {m, n} in [M, N], {n_p, k} in [N_P, K] where n_p <-> n {
  // matmul implements with m,n,K. n_p is no long useful.
}
```

Listing 2.63: Nested bounded groups

The relation `n <-> n_p` requires the two indices to have the same value in every iteration. Within the `with-in` block, either name can be used wherever the other is required. Such bindings are useful when expressing AI kernels.

The 'foreach' Block

Once the *bounded-ituple* is defined by the 'with-in' clause, programmers can loop over the bounded-ituples/bounded-integers. In Croqtile, this is simple.

```
with x in [10] {
  foreach x {
    // do something with each x
  }
}
```

Listing 2.64: The foreach Block

Each iteration gives the bounded variable a current value for the duration of the loop body. The bound stays fixed. Use that current value to select data, and use the bound when deriving how many pieces the data is divided into. Declaring a parallel index instead would assign one value to each independent instance rather than visit the values in order.

Basic Syntax: `parallel-by`

The basic syntax for a `parallel-by` block is as follows:

```
parallel p by 6 {
  // SPMD (Single Program, Multiple Data) code
}
```

Listing 2.65: A named parallel index

The braces enclose the body executed by each parallel instance. In this example:

- `parallel`: This keyword initiates the *parallel execution block*.
- `p`: It is the **parallel variable**. It contains an integer value identifying the current *parallel thread*. Each *parallel thread* runs the same code but with a different value of *parallel variable* `p`.
- `by 6`: It indicates that there are 6 *parallel thread*s. Thus, the *parallel variable* `p` ranges from `0` to `5` across different threads.

In some scenarios, programmers may not require an explicit *parallel variable* for a simple parallel construct. It is possible to omit it:

```
parallel by 2 { ... }
```

Listing 2.66: An unnamed parallel index

This creates two execution instances. Without a named index, the code cannot use that index to select different data for each instance, as SPMD programs commonly need to do.

Multiple-Level Parallelism

In Croqtile, you can define multiple levels of parallelism. Here's an example:

```
parallel p by 2 {
  // parallel-level-0
  parallel q by 12 {
    // parallel-level-1
  }
}
```

Listing 2.67: One parallel level

This code defines two levels of parallelism with values `2` and `12`, respectively. This means a total of `2 x 12 = 24` parallel threads are invoked for the task. Each thread is identified

by a unique combination of `p` and `q`. In Croqtile, you refer to the level of `p` as *parallel-level-0* and the level of `q` as *parallel-level-1*.

The nesting determines which instances belong to the same enclosing region. Shared data and synchronization must follow that structure. Put cooperating work at a level whose instances can use the required storage and synchronization operations.

In Croqtile syntax, you can define multiple-level parallelism in a single line:

```
parallel p by 2, q by 12 { ... }
```

Listing 2.68: Two parallel levels

This comma-separated form expresses the same two parallel levels as the nested version, provided there is no intervening code between those levels.

Sub-Level Parallelism

Some targets allow a single parallel level to be divided into up to three coordinate axes. Croqtile expresses these axes using `ituple` and `mdspan`:

```
parallel {px, py, pz} by [1, 1, 2] {
  // higher synchronization cost
  // ...
  parallel t_index = {qx, qy} by [3, 4] {
    // lower synchronization cost
    // ...
  }
  // ...
}
```

Listing 2.69: Sub-Level Parallelism

In this example, two levels of parallelism are defined. The first level (*level-0*) is subdivided into three sub-levels, annotated by the `{px, py, pz} ituple`. The second level (*level-1*) is subdivided into two sub-levels, annotated by the `{qx, qy} ituple`, or `t_index` as a whole. For each sub-level, the parallelization count corresponds to the respective value inside the `mdspan`.

The first element inside the `ituple` represents the **least significant parallel variable**. Croqtile adopts this ordering to align with developers' conventions, rather than for internal consistency.

A `with-in` statement declares a *bounded variable*; a `foreach` statement iterates over its range. Together they describe a loop without spelling out its bounds in C++.

Loops in Croqtile

In C++, loops are typically constructed using `for`, `while`, and `do-while` blocks. These loops include conditions for termination and usually involve variable manipulations related to loop control. Programmers have the freedom to define exit conditions, loop stepping, and other aspects, allowing for flexible loop structures.

Croqtile uses bounded ranges for these loops. The range is defined once and then visited by `foreach`:

```
foreach index {
  // loop body
}
```

Listing 2.70: A foreach loop

Here, `index` is a *bounded variable* (introduced in the previous section). If `index` has an upper bound of `6`, the above code is equivalent to:

```
for (int index = 0; index < 6; ++index) {
  // loop body
}
```

Listing 2.71: The equivalent C++ loop

This form suits tile traversal: the loop visits the valid coordinates in a declared range, and the same bounds can be used to derive each tile's shape.

In Croqtile, *bounded variables* defined within a `parallel-by` statement are immutable and cannot be used in `foreach` statements. Instead, programmers should use the `with-in` statement to define the *bounded variables* suitable for loops.

Defining *Bounded Variable*s:

A `with-in` statement allows you to define either a *bounded integer* or a *bounded ituple*. Syntactically, it resembles the `parallel-by` statement but only defines *bounded variable*s without invoking multiple instances for execution. Below are examples:

```
with x in 128 {
  // 'x' is a bounded integer
}
with y in [512] {
  // 'y' is a bounded ituple
}
with index in [10, 10] {
  // 'index' is a bounded ituple
}
```

Listing 2.72: A bounded variable

Here, `x` represents a bounded integer with an upper bound of `128`. Both `y` and `index` represent bounded ituples whose upper bounds are `512` and `10, 10` respectively. Similar to the `parallel-by` statement, you can name the elements of the ituple for clarity or declare both the bounded ituple and the associated bounded integers, as shown below:

```
with {x, y} in [10, 10] {
  // x and y are explicitly named elements of the ituple
}
with index = {x, y} in [10, 10] {
  // 'x', 'y', and 'index' can be used within the block
}
```

Listing 2.73: A tuple of bounded variables

Like the `parallel-by` statement, multiple `with-in` declarations can be combined using a comma-separated syntax, as demonstrated below:

```
with index = {x, y} in [10, 10], idx in [100, 10] { }
```

Listing 2.74: A variable derived from a tuple

In this way, two bounded ituples are defined. Note that either `index` or `idx` can only be referenced within the following `with-in` block.

The `with-in` statement defines *bounded variable*s, which can then be iterated over using the `foreach` statement. The basic syntax is as follows:

```
with index in [6] {
  foreach index {
    // Perform operations with each index
  }
}
```

Listing 2.75: Indexing one shape

In this example, the `foreach` block iterates 6 times, with the value of the **iteration variable** `index` ranging from `0` to `5` incrementally.

It is also possible to iterate over *bounded ituple*s. For example:

```
with index = {x, y} in [6, 17] {
  foreach index { }
}
```

Listing 2.76: A tuple of indices

This code defines a *bounded ituple* `index`. Iterating over `index` is equivalent to a nested loop structure:

```
for (int x = 0; x < 6; ++x)
  for (int y = 0; y < 17; ++y) { }
```

Listing 2.77: The generated C++ loop

The left-to-right order of indices in a bounded tuple determines the outer-to-inner nesting of the loops. Keep this ordering in mind when the order of iteration matters.

The same outer-to-inner ordering applies to a comma-separated list of bounded variables:

```
with index = {x, y} in [6, 17], iv in [128] {
  foreach iv, index { }
}
```

Listing 2.78: Several bounded groups

In this code, two bounded variables are defined. The following `foreach` block is equivalent to a multi-level nested loop:

```
for (int iv = 0; iv < 128; ++iv)
  for (int x = 0; x < 6; ++x)
    for (int y = 0; y < 17; ++y) { }
```

Listing 2.79: The generated inner loop

Syntactic Sugar

```
foreach x in 128 { }
foreach idx in [10, 20] { }
foreach {y, z} in [8, 16] { }
```

Listing 2.80: A foreach loop

The code above is equivalent to the following code:

```
with x in 128 {
  foreach x { }
}
with idx in [10, 20] {
  foreach idx { }
}
with {y, z} in [8, 16] {
  foreach y, z { }
}
```

Listing 2.81: The with-in form it expands to

Deriving the Loop From a Bounded Integer

In certain scenarios, such as pipelining data movement, it may be necessary to modify loop iterations. In Croqtile, this can be achieved by deriving a loop from a *bounded integer* within the `foreach` statement. For example:

```
with {x, y} in [6, 17] {
  foreach x, y(1::) { }
}
```

Listing 2.82: A tuple of bounded variables

In this case, the **Range Expression** `y(1::)` is used to derive the loop. This results in equivalent C/C++ code:

```
for (int x = 0; x < 6; ++x)
  for (int y = 1; y < 17; ++y) { }
```

Listing 2.83: The derived C++ loop

The `y` loop starts at `1`. A range expression consists of a bounded variable followed by three colon-separated integer fields in parentheses:

```
  bounded-variable(lower-offset:upper-offset:stride)
```

Listing 2.84: The bounded-variable syntax

This derives a loop from the `bounded-variable`, where the `bounded-variable` serves as the *iteration variable* of the loop. Specifically:

- The initial value of the *iteration variable* is the `lower-offset` plus the lower bound of the `bounded-variable`. Since all bounded integers have a lower bound of `0` in Croqtile, the `lower-offset` sets the initial value of the loop.

- The loop terminates when the *iteration variable* is equal to or greater than the upper bound of the `bounded-variable` plus `upper-offset`. Negative values are typically used for `upper-offset`.

- The *iteration variable* increments by `stride` at the end of each iteration.

Thus, `y(1:-1:2)` results in a loop like `for (y = 0 + 1; y < 17 - 1; y += 2)` in the example above. If any field of the *range expression* is not specified, it results in default values: `0` for `lower-offset` and `upper-offset`, and `1` for `stride`.

Note that *range expression*s only apply to the *bounded variable*s. *Range expression* over the *bounded ituple* triggers an error at compile time.

In short Bounded variables define iteration ranges, and range expressions adjust the starting point, ending point, and stride. Reading these loops requires distinguishing a variable's current value from its upper bound.

Deriving a loop this way is useful for multi-buffered data movement, where the loop may skip a tile loaded before iteration begins. Section 3.6.2 develops this pattern.

2.8 Tile Selection: chunkat, subspan, and view/from

`chunkat(...)`: equal partitioning

```
1 lhs.chunkat(tr, tc)
```

Listing 2.85: chunkat(...): equal partitioning

The `chunkat` expression divides each dimension according to the corresponding index bound. With input shape `[64, 128]` and `parallel {tr, tc} by [4, 8]`, it selects a `[16, 16]` tile. The current values of `tr` and `tc` select which tile, while their bounds determine its size.

That makes `chunkat` the natural tool when your tiling follows the parallel decomposition directly.

`subspan(...).at(...)`: explicit tile extents on a tile grid

```
1 lhs.subspan(64, 64).at(block_m, iv_k)
```

Listing 2.86: subspan(...).at(...): explicit tile extents on a tile grid

The `subspan` expression supplies the tile extents explicitly, and `.at(...)` supplies tile coordinates. For a `[16, 16]` tile, coordinate `(2, 3)` begins at element `(32, 48)`. This form is useful when the tile size is a fixed algorithmic choice rather than derived from the number of parallel instances.

`view(...).from(...)`: explicit tile extents with explicit element offsets

```
1 lhs.view(TILE_M, TILE_K).from(offset_m, offset_k)
```

Listing 2.87: view(...).from(...): explicit tile extents with explicit element offsets

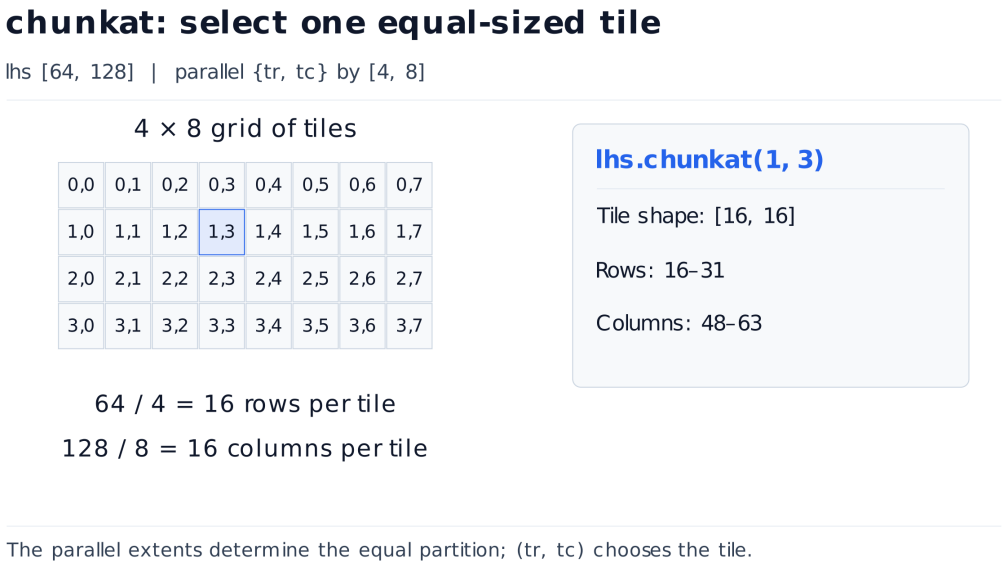


Figure 2.3: chunkat(1, 3) selects rows 16–31 and columns 48–63 from a [64, 128] tensor partitioned into [4, 8] tiles.

The form `view(...).from(...)` gives a window an explicit shape and an origin measured in elements. A `[16, 16]` window starting at `(37, 50)` keeps that exact origin; it does not round it to the nearest tile boundary.

This is the right tool when the tile origin is already available as element offsets rather than tile indices, or when the region is not most naturally described as an equal chunk. It is especially useful for dynamic and irregular access patterns where `chunkat(...)` would be awkward or impossible.

You can think of the three forms like this:

- Use `chunkat(...)` when the surrounding parallel decomposition already defines an equal tiling.
- Use `subspan(...).at(...)` when you want an explicit tile shape and tile coordinates.
- Use `view(...).from(...)` when you want an explicit tile shape and element-space starting offsets.

Compose: `outer # inner`

```
output.at(tr # i, tc # j)
```

Listing 2.88: Compose: outer inner

This combines a tile coordinate with an in-tile coordinate to produce a coordinate in the full tensor.

For an outer tile index `tr` and an inner index `i`, `tr # i` has value `tr * #i + i`. If a tile has 16 rows, tile 2 and inner row 5 identify global row 37. Its bound is the product of the two bounds. Operand order matters because the inner bound supplies the stride.



Figure 2.4: With 16 elements per tile, tile = 2 and i = 3 compose to global position 2 × 16 + 3 = 35.

Extent: `#name`

```
foreach i in [128 / #tile]
```

Listing 2.89: Extent: name

The notation `#tile` means the extent of the `tile` axis: how many tiles exist along that dimension. If `parallel tile by 8`, then `#tile` is 8.

Bounded-ituple/integer and 'chunkat' Operation

A `chunkat` expression selects a view of existing spanned data. It uses each bounded argument twice: the upper bound determines the number of chunks, and the current value selects one chunk. Selection alone does not move the elements; the surrounding DMA statement performs the transfer.

```
global f32 [6, 10, 100] data;
parallel p by 6 {
  with index = {x, y} in [10, 10] {
    // for every data move, the stride into 'data' is
    //    stride = p*1000 + x * 100 + y * 10
    //
    // for each chunk, the dimensioned size for the movement is {1, 1, 10}
    f = dma.copy data.chunkat(p, index) => local;
  }
}
```

Listing 2.90: Bounded-ituple/integer and chunkat Operation



Figure 2.5: Prefix #tile gives the axis extent, 8; infix tile # i composes a tile coordinate and an in-tile offset.

The example above shows one typical use of `chunkat`. Here the spanned data has type `f32 [6, 10, 100]`. The operation takes two parameters — the *integer* `p` and the *ituple* `index` — and divides the data into $6 \times 10 \times 10$ pieces, one for each combination of their ranges. Each piece holds 10 consecutive elements along the least significant dimension and is copied into local storage.

A *chunkat expression* names one tile of a larger buffer in a DMA statement, so the movement code never has to compute an offset by hand.

Ubound Operation

A bounded variable declared by `parallel-by` or `with-in` is an integer or tuple with associated bounds. A `foreach` loop uses its current value as the index and its upper bound to determine the iteration range.

In Croqtile, a program can use the **ubound operation** to explicitly retrieve the upper bound of a bounded variable. *Ubound* is a unary operation that uses the prefix symbol `#` before a bounded variable. For example:

```
with size in [32] {
  local u8 [#size, 16] a; // buffer shape: [32, 16]
  local u8 [size, 16] b; // error in shape: [0, 16]
}
```

Listing 2.91: Ubound Operation

In this code, the local buffer `a` is shaped as `[32, 16]`, since `#size` evaluates to the upper-bound of `size`. However, the declaration of `b` uses the *current value* of `size`, resulting in an incorrect shape definition.

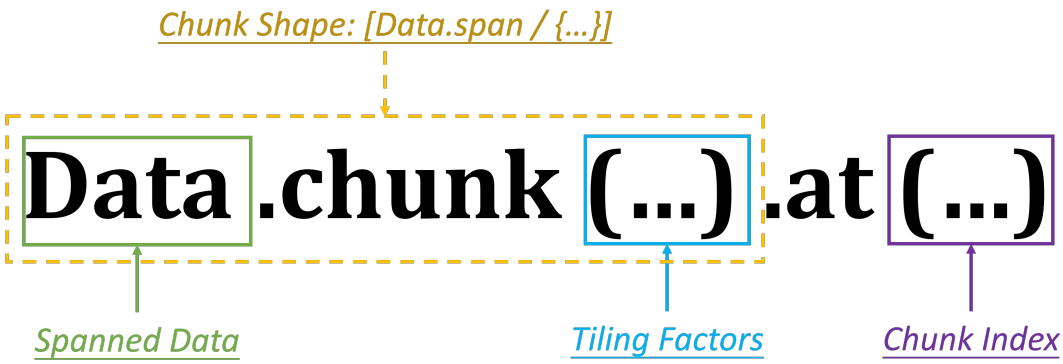
Use `chunkat` Expression to Tile Data

A transfer can use a whole buffer, but a tiled algorithm usually needs only the portion assigned to the current instance. Put `chunkat` in the source expression to describe that portion directly:

```
__co__ void foo(f32 [36] input) {
  parallel p by 6 {
    dma.copy input.chunkat(p) => shared;
  }
}
```

Listing 2.92: Use chunkat Expression to Tile Data

In this example, the built-in member function `.chunkat` is used to create a tile or block for the source expression of the DMA statement. Each tiled block is called a **chunk** in Croqtile. The `.chunkat(...)` data expression can be considered a simpler form of `.chunk(..).at(...)`, where the semantics of each part are illustrated in the figure below:



Considering the expression `input.chunkat(p)`, which is equivalent to `input.chunk(#p).at(p)`:

- `input` is the *spanned data* to be tiled.
- `#p` indicates the **tiling factor**. It also specifies the total **chunk count** for each dimension to process. In this case, the 1D input is tiled by `6`, resulting in `6` chunks to be processed.
- `.chunk(#p)` determines the **chunk shape**. The chunk is a 1D data segment tiled from `input`, with its size being the data *mdspan* divided by the *upper-bound* of the bounded variable. In this example, the chunk shape is calculated as `[input.span / #p] = [6]`.
- `.at(p)` determines the **chunk index**. Here, `p` ranges from `[0, 6)`, allowing parallel threads to process different chunks of `input`. Specifically, parallel thread `1` processes a chunk with elements `[6, 12)` from `input`, while parallel thread `5` processes a chunk with elements `[30, 36)`, and so on.

In many scenarios, it is unnecessary to use separate `.chunk` and `.at` expressions, as `.chunkat` takes bounded variables as parameters. The upper-bounds of these variables act as the *tiling factors*, and their current values act as the *chunk index*.

Use Multiple Bounded Variables

The example below uses several bounded variables at once:

```
__co__ void foo(f32 [36, 14, 8] input) {
  parallel p by 6 {
    with index in [7, 4] {
      foreach index {
        dma.copy input.chunkat(p, index) => shared;
      }
    }
  }
}
```

Listing 2.93: Use Multiple Bounded Variables

In this example, there are two bounded variables: `p` is the *bounded integer* defined by the `parallel-by` statement, while `index` is the *bounded ituple* with two integers. The `chunkat` expression concatenates them to form a *bounded ituple* with upper-bounds `{#p, #index}` and current values `{p, index}`. Thus, the shape of each chunk is calculated as `[36, 14, 8] / {#p, #index} = [36, 14, 8] / {6, 7, 4} = [6, 2, 2]`.

In addition, since `p` is the bounded variable for parallelization, there are `6` threads working in parallel, each handling a large chunk of `[6, 14, 8]`. Within a single parallel thread, it iterates `7x4` times, each time moving a `[6, 2, 2]` small chunk to a compiler-allocated `shared` storage.

The same bounds now describe both the iteration space and the selected data. If the partition changes, the compiler derives the new chunk dimensions from those bounds. To review the transfer, check the input shape, divide by the bounds, and then multiply the chunk coordinates by the resulting chunk extents to recover the element origin.

In short The `chunkat` expression divides a buffer into smaller chunks for a DMA statement, and the shape of each chunk follows from the bound used to select it. You explored how `chunkat` can be applied to either the source or destination of a DMA statement to manage data movement between memory and processors.

Section 3.6.1 combines these constructs in a complete Tileflow program.

2.9 Data Movement: the DMA Statement and Futures

The DMA Statement Family

All of Croqtile's tile-movement operations use the same basic form:

```
future = engine.operation.modifiers source => destination;
```

Listing 2.94: The general DMA form

The important pieces are:

Minimal examples:

```
f = dma.copy input.chunkat(blk) => shared;
g = dma.transp<1, 0> matrix => local;
h = dma.pad<{2, 1}, {3, 2}, {0, 0}, 0> tile => shared;
```

Listing 2.95: A concrete copy

Part	Meaning
`future`	Optional handle to the transfer result. Use `future.data` to access the moved tile and `future.span` to query its shape.
`operation`	`copy`, `transp<...>`, or `pad<...>`.
`modifiers`	Optional flags such as `.async`, `.zfill`, or `.swiz<N>`.
`source => destination`	The source view and the destination memory or destination view.

The key shift from Section 1.3 is that these statements move shaped regions, not individual elements.

First Example: Tiled Addition Through shared storage

```
__co__ s32 [64, 128] tiled_add_2d(s32 [64, 128] lhs, s32 [64, 128] rhs) {
  s32 [lhs.span] output;
  parallel {tr, tc} by [4, 8] {
    lhs_load = dma.copy lhs.chunkat(tr, tc) => shared;
    rhs_load = dma.copy rhs.chunkat(tr, tc) => shared;
    foreach {i, j} in lhs_load.span
      output.at(tr # i, tc # j) =
          lhs_load.data.at(i, j) + rhs_load.data.at(i, j);
  }
  return output;
}
```

Listing 2.96: Tiled Addition Through Shared Storage

The arithmetic is still element-wise addition. What changed is the data path:

1. `chunkat(tr, tc)` selects one logical tile from each input.
2. `dma.copy ... => shared` stages that tile into block-visible fast memory.
3. `tr # i` and `tc # j` map tile-local coordinates back to global output coordinates.

This is the core Croqtile pattern: select a tile, move it, compute on the moved tile.

Futures, `.data`, and `.span`

The result of a DMA statement is a future-like handle:

```
f = dma.copy input => shared;
```

Listing 2.97: A future from a copy

That handle gives you two important things:

- `f.data`: the moved tile, as a spanned tensor.
- `f.span`: the shape of that tile.

Example:

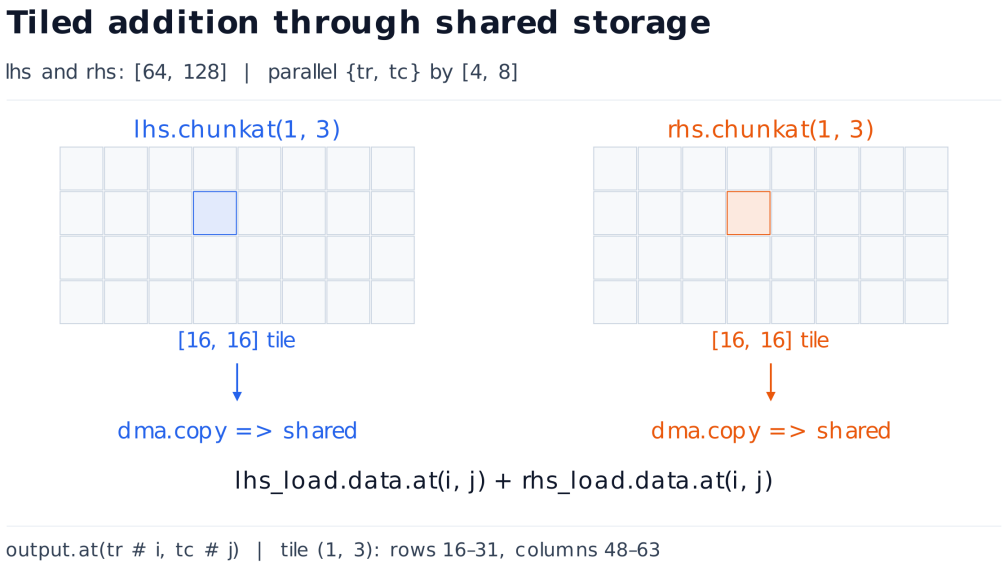


Figure 2.6: For [64, 128] inputs and a [4, 8] tile grid, tile (1, 3) stages [16, 16] operands before element-wise addition.

```
f = dma.copy input => shared;
foreach idx in [f.span]
  output.at(idx) = f.data.at(idx);
```

Listing 2.98: Reading data and span

Even synchronous copies are worth naming when you want to pass the moved tile to later code.

Synchronous and Asynchronous Copies

By default, copies are synchronous:

```
f = dma.copy input => shared;
dma.copy f.data => output;
```

Listing 2.99: A synchronous copy

Add `.async` when you want the transfer to proceed in the background:

```
f = dma.copy.async input => shared;
wait f;
dma.copy f.data => output;
```

Listing 2.100: An asynchronous copy

The same pattern applies to asynchronous copies:

```
f = dma.copy.async input => shared;
wait f;
dma.copy f.data => output;
```

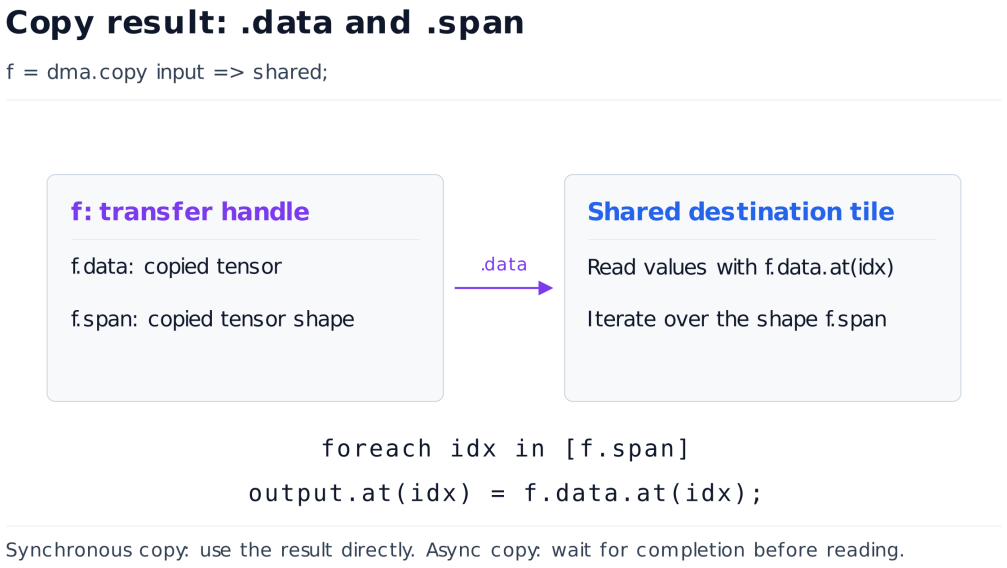


Figure 2.7: The copy result exposes the destination tensor through .data and its shape through .span; asynchronous reads require completion first.

Listing 2.101: An asynchronous copy

A future identifies the destination before an asynchronous transfer necessarily finishes. Access through `f.data` does not mean the elements are ready. Place `wait f` before the first operation that reads those elements, and keep the source and destination valid while the transfer is in flight.

Later chapters use async copies to overlap load and compute. In this part of the guide, the important part is the contract: `.async` creates an in-flight transfer, and `wait` makes it safe to consume.

dma.transp<...>

```
1 fa = dma.transp<1, 0, 2> a => local;
2 dma.copy fa.data => o;
```

Listing 2.102: dma.transp ...

This permutes dimensions during the movement itself. You do not first copy and then transpose. The transfer produces data in the transposed layout.

dma.pad<...>

```
1 f = dma.pad<{2, 1}, {3, 2}, {0, 0}, V>.async input => shared;
2 wait f;
3 dma.copy f.data => output;
```

Listing 2.103: dma.pad ...

Padding creates a larger destination tile. The template arguments describe low padding, high padding, internal stride-style padding, and the fill value. Use `pad` when you want explicit border values, not just out-of-bounds protection.

Tileflow Program: Orchestrating the Data Movement

The *Tileflow Program* consists of *Croqtile functions*. As described earlier, it manages the movement of data between the host and the target device, ensuring that data is copied correctly across different storage locations.

For convenience, let's revisit the code:

```
__co__ s32 [6, 17, 128] ele_add(s32 [6, 17, 128] lhs, s32 [6, 17, 128] rhs) {
  s32 [lhs.span] output; // Use same shape as lhs
  parallel p by 6 {
    with index in [17, 4] { // Tiling factors
      foreach index {
        lhs_load = dma.copy lhs.chunkat(p, index) => local;
        rhs_load = dma.copy rhs.chunkat(p, index) => local;
        local s32 [lhs_load.span] l1_out;
        // Call kernel with loaded data
        call kernel(lhs_load.data, rhs_load.data, l1_out, |lhs_load.span|);
        // Store result back to output
        dma.copy l1_out => output.chunkat(p, index);
      }
    }
  }
  return output;
}
```

Listing 2.104: The tileflow program

The `__co__` function accepts `lhs` and `rhs`, both with shape `[6, 17, 128]` and element type `s32` (signed 32-bit integer). The output has the same shape and element type.

The `parallel p by 6 {...}` block indicates that the enclosed code runs in parallel. Specifically, six instances of the code are executed concurrently. This implies a transition in the execution environment from the host to the device.

Inside the `parallel-by` block, `with-in` binds `index` to the bounds `[17, 4]`. This two-dimensional bounded tuple supplies the indices for the `foreach` loop.

The `foreach index {...}` statement is equivalent to the following C code:

```
for (int x = 0; x < 17; x++)  // assume 'x' represents the 1st element of '
    index'
  for (int y = 0; y < 4; y++) { ... }  // and 'y' represents the 2nd element
    of 'index'
```

Listing 2.105: The equivalent C++ loop

Within the `foreach` block, the `dma.copy` statement describes how data movement occurs. For example, consider the statement `lhs_load = dma.copy lhs.chunkat(p, index) => local;`:

- The symbol `lhs_load` is called **future** of the DMA operation, which contains information about the DMA destination.

- `dma.copy` invokes a direct DMA data transfer without transforming the shape of the data. The expression on the left-hand side of `=>` represents the DMA source, while the right-hand side represents the destination.
- In this case, the destination is specified as a `local` buffer, which is automatically allocated by the Croqtile compiler.
- The source expression `lhs.chunkat(p, index)` selects a chunk of `lhs`. The bounds of `p` and `index`, namely `[6, 17, 4]`, determine the partition. Dividing the input shape `[6, 17, 128]` by these bounds gives chunk shape `[1, 1, 32]`. The current values select the chunk: instance `p = 1` and iteration `index = {16, 2}` select chunk `(1, 16, 2)`, whose element origin is `(1, 16, 64)`.

The following figure shows how the bounds determine the chunk shape and the indices select a chunk.

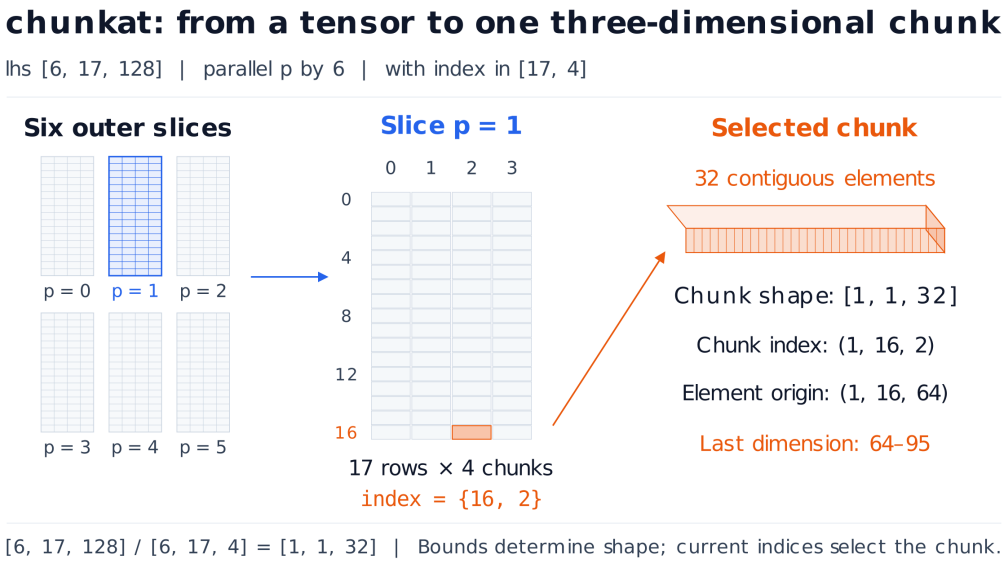


Figure 2.8: Dividing shape [6, 17, 128] by bounds [6, 17, 4] gives chunks of shape [1, 1, 32]. Chunk (1, 16, 2) starts at element (1, 16, 64).

With the DMA statement, different chunks of data tiled from `lhs` are moved iteratively and in parallel from the host to the device's `local` storage. Similarly, the DMA statement manages `rhs` by moving it in small chunks to `local` storage for processing.

The declaration `local s32 [lhs_load.span] l1_out;` creates a buffer for each parallel instance, with the same shape as the loaded chunk. The subsequent `call` invokes the device function `kernel`, which writes its result to that buffer. A final DMA statement copies the result back to the host output, completing the iteration.

In this code, each parallel thread runs `17x4` iterations, with each iteration handling a `1x1x32`-sized chunk of data. The *croqtile program* terminates when all `6` parallel threads have completed their iterations. It then returns the output buffer to its caller, the host program.

The DMA statement describes data movement and can name its result with a future. Its syntax specifies the operation, source, and destination.

A DMA statement can execute asynchronously, allowing subsequent code to run while the transfer is in progress. An asynchronous transfer requires explicit synchronization before its result is used.

To support these features, the syntax is organized as follows:

```
future = dma-op src-expr => dst-expr;
```

Listing 2.106: Basic Syntax

The name on the left of `=` denotes a future. It identifies the transfer result and, for an asynchronous operation, provides the handle used by `wait`. A synchronous transfer may omit the name when later code does not need a handle to its destination.

The right-hand side of the **DMA operation** includes the **operation type** (`dma-op`), the **data expression** for the operation source (`src-expr`), and the *data expression* for the destination (`dst-expr`). The source and destination are separated by the symbol `=>`, which indicates the direction of data flow.

Synchronous and Asynchronous

An asynchronous copy is non-blocking: subsequent code can execute while the transfer is pending. A synchronous copy completes before execution proceeds past the statement. These terms describe when the issuing code can continue, not whether it may safely read an unfinished transfer's destination.

In Croqtile, the `.async` suffix in a **DMA operation** indicates that the operation is asynchronous. For example:

```
f0 = dma.copy.async data0 => shared; // non-blocking
f1 = dma.copy data1 => shared;       // blocking
...
wait f0;                             // synchronization
```

Listing 2.107: Synchronous and Asynchronous

In this example, the DMA operation that produces the future `f0` is asynchronous, allowing the program to continue executing while the data transfer occurs in the background. The DMA operation that produces `f1` is synchronous, meaning the program will pause until the data is fully transferred from `data1` to `shared` storage. Meanwhile, the data copy of `data0` might still be in progress.

The statement `wait f0;` waits until the transfer from `data0` to `shared` storage is complete, synchronizing subsequent execution with that transfer. Ensure that every asynchronous result is awaited before it is used.

Data Expression

The data expressions used so far have been simple, either a defined **spanned data** or a **storage location**. The **storage location** is used solely for the **destination buffer** declaration, as it requires the Croqtile compiler to allocate memory for it. For example:

```
global f32 [32, 16] input;
shared f32 [32, 16] output;
dma.copy input => output;  // Copy to a user-declared buffer
dma.copy input => shared;  // Same, but to a compiler-allocated buffer
```

Listing 2.108: Data Expression

The second DMA statement names `shared` as its destination, so the compiler allocates the buffer with the inferred shape. It is equivalent to the first statement's explicit buffer declaration. Naming a storage location is convenient when the compiler can infer the destination shape.

The same source and destination positions can contain tile expressions. For example, `input.chunkat(p)` restricts the copy to the portion selected by `p`; an explicit destination view restricts where the result is written. Section 2.8 explains how those views are formed.

Future and Wait

Both *Sync-DMA* and *Async-DMA* operations can be used to define a *future*. As seen in earlier examples, a *future* acts as a handle for asynchronous operations and is typically used as a parameter of the `wait` statement to synchronize an *Async-DMA*.

Croqtile allows the `wait` statement to take multiple *future*s. However, using a *future* from a **Sync-DMA** in a `wait` statement will result in an error. For example:

```
f0 = dma.copy.async input0 => shared;
f1 = dma.copy.async input1 => shared;
f2 = dma.copy input2 => shared;
...
wait f0, f1;  // Multiple wait
wait f2;      // Error: cannot wait on a sync-dma
```

Listing 2.109: Two asynchronous copies

In Croqtile, another use of a *future* is to retrieve the destination buffer of the associated DMA statement. This is also the reason why **Sync-DMA** is allowed to define a future. Croqtile provides two built-in member functions for *future* variables:

- `.span` to retrieve the **mdspan** of the DMA destination buffer.
- `.data` to retrieve the reference of the DMA destination buffer (spanned data).

For example:

```
f0 = dma.copy.async input0 => shared;
f1 = dma.copy input2 => shared;
...
local f32 [f0.span] buffer;
call device_kernel(f1.data, |f1.span|);
```

Listing 2.110: Waiting on both futures

Here, `.span` retrieves the shape of the destination buffer associated with `f0`, and `.data` retrieves the destination data associated with `f1`. Applying the element-count operation to `f1`'s destination shape returns its total number of elements.

In short DMA statements support synchronous and asynchronous transfers and several movement operations. Futures name the results, expose destination buffers, and provide handles for waiting on asynchronous work.

Section 2.8 explains how data expressions select tiles or blocks within a buffer.

2.10 Coordination: inthreads, Events, and Async Blocks

Parallel Perspective of Tileflow Programs

In Croqtile, a *divergent* code region is generated with the keyword `inthreads`, followed by a comparison expression.

Let us have a look at the following example:

```
__co__  void foo() {
  // sequential code
  // start of the SPMD code region
  parallel p by 6 {
    // some uniform code for all threads
    // ...
    inthreads (p <= 2) {
      // divergent code for thread 0, 1, 2 (thread-mask: 000111)
    }
    // threads are forced to be synchronized
    // some uniform code for all threads
    // ...
  }
  // end of the SPMD code region
  // sequential code
}
```

Listing 2.111: Parallel Perspective of Tileflow Programs

You can examine Croqtile tileflow programs from the perspective of parallelism. It consists of

- *Sequential Code*
- *SPMD-Styled Parallel Code* enclosed in `parallel-by` block, which consists of:
  - *Uniform Code*
  - *Divergent Code* enclosed in `inthreads` block.

In this example, `p <= 2` initiates the *divergent code block*, where from the implementation perspective, it creates a thread mask of `000111`, which allows only thread `0`, `1` and `2` to execute the code inside. And thread `3`, `4` and `5` will skip this path. **All threads are forced synchronized after the `inthreads` block**. In this case, it implies thread `3`, `4`, and `5` waits for the completion of the others before stepping forward.

An `inthreads` block resembles a C++ conditional, but it selects participants in an SPMD region. Its placement and predicate must therefore refer to that parallel structure:

- Can only appear in the SPMD code region, which is enclosed in the `parallel-by` block.
- Can only have a comparison result related to thread identifiers.

Croqtile compiler has the checks for both the comparison expression and `inthreads`-block placement.

Implicit Masking

In Croqtile, code between multiple `parallel-by` levels can be considered as a type of implicit masking. For example:

```
__co__  void foo() {
  parallel p by 6 {
    // implicit masking: 'inthreads (q == 0) { ... } '
    parallel q by 2 {
      // code here is for all threads
    }
    // implicit masking: 'inthreads (q == 0) { ... } '
  }
}
```

Listing 2.112: Implicit Masking

In this example, there exists two levels of parallelization. The code inside `parallel p by 6` block but outside `parallel q by 2` blocks is conceptually for the outer parallelization only. This means, only one thread from the sub-level parallelization execute the code, which is equivalent to `inthreads (q == 0) { ... }`. Programmers must be aware there is only single thread instance to execute the code part. Only the code inside the inner `parallel-by` is executed by all threads.

An asynchronous region can advance independently, but a dependency on another region still needs to be expressed. An event marks that dependency: one region triggers it after completing the required work, and another waits before using the result. The event carries readiness information; the data remains in the buffer shared by the regions.

Event Basics

The meaning of an event varies between programming languages. In Croqtile, an **event** coordinates asynchronous code blocks at different execution levels.

A Croqtile event has two states, **SET** and **UNSET**. A newly declared event starts in the UNSET state. For example:

```
shared event e;
```

Listing 2.113: Declaring a shared event

A `shared` event is declared inside a `parallel-by` block and used by its parallel instances. Events can also use `global` or `local` storage. The target platform determines the constraints on their declarations and references.

In addition, you may declare an event array following C-style syntax:

```
shared event e[4], e1;
```

Listing 2.114: Declaring an event array

A comma-separated expression can declare both an event and an event array on one line.

The `trigger` statement changes an event from **UNSET** to **SET**:

```
shared event e[4], e1;
trigger e, e1;
```

Listing 2.115: Triggering an event array

The comma-separated statement triggers every event in array `e` and the single event `e1`.

The `wait` statement blocks the current instance while the event is **UNSET**. Execution resumes once the event becomes **SET**.

Triggering an event is what unblocks the code waiting on a *wait statement*. For example,

```
parallel p by 2 {
  shared event e;                              // init 'e' to 'UNSET'
  inthreads.async (p == 1) { wait e; }         // wait-thread
                                               // once unblocked, change 'e' to '
    UNSET'
  inthreads.async (p == 0) { trigger e; }      // trigger-thread
                                               // convert 'e' to 'SET'
  sync.shared;                                 // sync point
}
```

Listing 2.116: Waiting on an event

In this example, the triggering instance sets `e`, allowing the waiting instance to continue. Both then reach `sync.shared`. The event establishes a dependency between the two asynchronous regions: the trigger must occur before the wait can finish.

In Croqtile semantics, **an event is always auto-reset by its `wait` statement**. That means when a `wait` statement is unblocked, the events that are waited are changed to *UNSET* state.

Event arrays for multi-stage pipelines

Consider a producer that loads tiles and a consumer that computes on them. Before reading a slot, the consumer needs a signal that its data is ready; before overwriting it, the producer needs a signal that the previous read has finished. Call these signals `full` and `empty`. A pipeline with several reusable slots gives each slot its own pair of events:

```
shared event full[STAGES], empty[STAGES];
```

Listing 2.117: Event arrays for multi-stage pipelines

Each physical buffer slot gets its own `full`/`empty` pair. The ring index `stage = iv_k % STAGES` maps the unbounded K iteration to a fixed number of physical slots. With four stages, the producer can run several tiles ahead before blocking on `wait empty`.

Bootstrap protocol

The consumer must **seed** the `empty` credits before the K-loop starts:

```
foreach {s} in [STAGES] {
  trigger empty[s];
}
```

Listing 2.118: Bootstrap protocol

Without this bootstrap, the producer's first `wait empty[0]` blocks forever — a deadlock, not a mysterious MMA bug. This is a common pitfall: every bounded buffer protocol requires initial credits.

Section 3.8 develops the full double-buffered and multi-stage pipeline kernels that put these primitives to work. The examples there compose `inthreads.async`, events, `swap` into complete, runnable matmul pipelines.

In short Events order asynchronous `inthreads` regions. A `trigger` sets an event or event array to **SET**. A `wait` blocks until the event is triggered, then resets it to **UNSET**.

Pair event triggers and waits carefully. Incorrect ordering can cause deadlocks that are difficult to diagnose.

2.11 Branching and Early Return

Predicated execution with `if`

Croqtile's `if` behaves like its C counterpart:

```
if (tile_id < total_tiles) {
  // body executes only when the condition is true
}
```

Listing 2.119: Predicated execution with if

All threads in scope evaluate the condition. Threads where it is false skip the body. Threads on the skipped side sit idle while the taken side runs, then vice versa. This is the price of runtime flexibility.

Use `if` for runtime decisions such as bounds checks and conditional accumulation. The condition may depend on the current tile coordinate or input values. Keep it uniform across a cooperating group when the body contains an operation that requires the group to participate together.

Whenever possible, keep `if` conditions uniform across the execution group: in this example, all 32 threads should agree.

Early return from `parallel`

Use `yield` to return early from a `parallel` block. It generates a `return;` instruction that ends device execution for the current instance.

```
parallel p by BLK_CNT: block {
  parallel q by THR_CNT : thread {
    if (cond) yield;
    ...
  }
}
```

Listing 2.120: Early return from parallel

Common pitfall

The `yield` keyword currently will **not** do any compile-time nor runtime checks. Be careful about synchronization safety.

Check an early exit against the remaining synchronization in the region. An instance that leaves before triggering an event can strand its consumer; an instance that skips a required collective operation can strand its peers. A tail guard is easiest to reason about when all instances responsible for the same tile make the same decision.

2.12 The Four-Step MMA Syntax

Every contraction kernel follows the same rhythm, and the spelling makes each step visible:

1. `mma.fill 0.0`
2. `mma.load`
3. `mma.row.row mc, ma, mb` — issue the contraction: $\mathbf{C} \mathrel{+}= \mathbf{A} \times \mathbf{B}$ into `mc`.
4. `mma.store mc, dst`

You loop steps 2–3 over K (loading the next K-slice each iteration, accumulating into the same `mc`), then run step 4 once to flush the completed output tile. You do not declare per-lane layouts; the compiler derives them from the target and your layout choice (`row.row` here).

The `.row.row` suffix is a **layout contract** — Both operands are row-major. The full set of layout combinations is `row.row`, `row.col`, `col.row`, and `col.col`; in practice, `row.row` and `row.col` cover most kernels. Choosing the wrong variant produces incorrect results.

For two K slices, initialize `mc` once, load the first pair of operands and accumulate, then load the second pair and accumulate into the same `mc`. Only then store the output. The load handles describe operand fragments; the accumulator holds the partial result across iterations. This distinction explains why fill and store sit outside the repeated load-and-multiply sequence.

2.13 C++ Embedding, Calls, Macros, and the Preprocessor

`__cpp__`: verbatim C++ injection

The `__cpp__` form inserts a string as C++ source at the point where it appears in a Croqtile function. Use it for a short target operation that needs to sit inside a particular parallel or asynchronous region. The inserted text must be valid in that scope, with all referenced names visible to the target compiler. Croqtile passes the contents through without parsing or rewriting them.

- `__cpp__("...")` — Ordinary string; best for short one-liners.
- `__cpp__(R"(...)")` — Raw string literal; use for `asm volatile` where escaping every `"` would be painful.

Croqtile's preprocessor runs before the main compiler pass. Macros expand in both `__co__` regions and host C++ in the same `.co` file, so one definition keeps tile geometry and host-side checks aligned.

Directive	Role
`#define NAME value`	Object-like macro: textual replacement
`#if` / `#elif` / `#else` / `#endif`	Conditional inclusion
`#ifdef` / `#ifndef`	Shorthand for whether a macro is defined
`#error message`	Force a compile-time failure with a message

Function-like macros (`#define MAX(a, b) ...`) are not supported. Use `constexpr` helpers in ordinary C++.

Tile geometry as macros

Production matmul sources centralize tile dimensions at the top:

```
#define MATMUL_WARP_M 64
#define MATMUL_WARP_N 128
#define MATMUL_TILE_K 64
#define MATMUL_WARP_K 16
#define MATMUL_STAGES 4
```

Listing 2.121: Tile geometry as macros

The same names appear in `parallel`, shared-memory declarations, `foreach` bounds, and host-side verification.

Compile-time assertions with `#error`

```
#if MATMUL_SWIZ != (2 * MATMUL_TILE_K)
#error "MATMUL_SWIZ must equal 2 * MATMUL_TILE_K for f16 kernel"
#endif
#if MATMUL_WARP_M != 64
#endif
```

Listing 2.122: Compile-time assertions with error

The preprocessor evaluates this condition before the Croqtile function is parsed. An incompatible set of constants therefore stops the build with the supplied message. Use this for relationships among configuration macros; a property of input data supplied at runtime belongs in a runtime assertion instead.

Conditional code paths

```
#define PATH0
__co__ auto foo() {
  #ifdef PATH0
    // path 0 code
  #else
    // path 1 code
  #endif
}
```

Listing 2.123: Conditional code paths

The preprocessor keeps one branch and discards the other before Croqtile parses the `__co__` body. Command-line defines: `croq kernel.co -DMATMUL_TILE_K=128`.

Calling C++ Functions

Croqtile functions are not allowed to call another Croqtile function. However, inside a Croqtile function, it is normal to have function calls to C++ kernels.

```
void bar() {...}    // C++ kernel function
__co__ void foo() {
  parallel p by 6 {
```

```
    call bar();     // Call the C++ function
  }
}
```

Listing 2.124: Calling C++ Functions

The example uses the Croqtile `call` keyword to invoke the existing C++ function `bar`.

A shared configuration constant often appears in both the host allocation and the kernel declaration. Defining it once as an object-like macro keeps those uses together. The examples below show how replacement text and conditional compilation affect the source before Croqtile checks its shapes and types.

Object-Like Macros

Croqtile preprocessor supports C/C++ **object-like macros** to connect both host code and tileflow code. The *object-like macro* performs pure text replacement and accepts no parameters. The following code gives an example:

```
#define M 256
#define N 32
#define K 64
__co__ auto matmul(f32 [M, K] lhs, f32 [N, K] rhs) { /*...*/ }
void foo() {
  croq::f32 a[M][K];
  croq::f32 b[N][K];
  // ...
  auto res = matmul(croq::make_spanview<2>(a, {M, K}),
                    croq::make_spanview<2>(b, {N, K}));
}
```

Listing 2.125: An object-like macro

In the code snippet, the inputs of croqtile function `matmul` are not dynamically shaped. Instead, Croqtile pre-processor substitutes `M`, `N`, `K` with values `256`, `32`, `64` ahead of Croqtile compilation, which produces tileflow function with statically shaped inputs. In this way, programmers make different code consistent and easy to manipulate.

The preprocessor currently supports only macros without parameters, so code like:

```
#define MAX(a, b) (((a) > (b)) ? (a) : (b))
```

Listing 2.126: A macro with parameters

would not work.

Comments

Croqtile supports both C-style block comments (`/*...*/`) and line comments (`//...`) through its preprocessor.

Conditional Compilation

Croqtile supports conditional compilation with directives including `#if`, `#ifdef`, `#ifndef`, `#else`, and `#endif`:

```
#define PATH0
// some host code
__co__ auto foo() {
#ifdef PATH0
// some code related to PATH0
#else
// other code
#endif
}
// host control
#ifdef PATH0
// ...
#else
// ...
#endif
```

Listing 2.127: Conditional Compilation

The same preprocessing conditions can control both host C++ and Croqtile code.

The preprocessor is still being extended, and full C preprocessing support is not planned: a C feature is picked up only when a program actually needs it.

Croqtile uses device functions to perform target computations. The `call` statement specifies where to invoke a device function and which arguments to pass.

Call Device Functions

To invoke a device function from a Croqtile function, use a statement beginning with `call`.

Basic Syntax of Call Statements

The general syntax for a `call` statement in Croqtile is:

```
call func-name <optional-template-args> (arguments);
```

Listing 2.128: The general call syntax

After `call`, write the function name and, optionally, comma-separated template arguments in angle brackets. Ordinary arguments follow in parentheses, separated by commas as in C/C++.

The target compiler checks that the called function exists and that its signature accepts the generated arguments. Croqtile does not perform these C++ checks during transpilation. A call error may therefore mention generated source; compare it with the helper declaration and the original call site.

In the following example, `call bar();` in `mou` is illegal code since it tries to call device function in a code location which is assumed to be host code area.

```
__co__ void foo() {
  parallel p by 1 {
    call bar();  // ok
  }
}
__co__ void mou() {
  call bar();  // error: not able to call device function from host
}
```

Listing 2.129: An illegal call site

The Convention: Allowed Argument Types

In current implementation, the argument must be either:

- *Spanned Data* type, or
- Scalar *Integer* type, or
- Scalar *Floating-Point* type.

Here, the scalar *integer* type only includes `int`, whereas the scalar *floating-point type* includes `float`, `double`, `half`, `bfp16`, and `half8`. However, since the target-platform differs in floating-point support, the *floating-point type* parameter also varies for different target platforms.

Croqtile does not perform arithmetic on these floating-point scalar values in Tileflow code. Instead, the Tileflow program passes them to device functions for computation.

For example:

```
__device__ void bar(float *p, int m, int n) {}
__device__ void foo(float *, int, unsigned, double, float) {}
__co__ void foobar(f32 [M, 24] input, int N, float padding) {
  parallel p by 1 {
    shared f32 [14, 7] buffer;
    call bar(input.data, input.span(0), buffer.span(0));
    call foo(buffer.data, N, 3, 3.14, padding);
  }
}
```

Listing 2.130: The Convention: Allowed Argument Types

Here the spanned-data arguments become pointers to their element types. The device helper receives those pointers without shape metadata; pass any required dimensions separately. The declared pointer element type must match the data supplied by the call.

For element formats without a native target type, use the corresponding Croqtile C++ type, such as `croq::f16` or `croq::bf16`, where supported.

In short A `call` statement invokes a device function, with or without template arguments. Care is needed since no check is applied at transpilation time, which may result in more confusing error reporting since the check is applied at target compile time with generated functions.

2.14 Printing, Assertions, and Compile-Time Inspection

`print!` and `println!`: compile-time inspection

The bang variants (`print!`, `println!`) run at **compile time**, not at runtime. They print to the compiler's output and are useful for inspecting shapes, extents, and type information without launching a kernel:

```
__co__ void check_shapes(f32 [3, 2] b) {
  print!("shape of b: ");
  println!("b.span = ", b.span);
}
```

Listing 2.131: A compile-time shape print

When you compile this file, the compiler emits:

```
shape of b: b.span = [3, 2]
```

Listing 2.132: The printed shape

String literals are concatenated (`"wor" "ld!"` becomes `"world!"`).

`print` and `println`: runtime device output

```
__co__ void inspect(s32 [4, 8] data) {
  foreach i in [data.span] {
    println("element ", i, " = ", data.at(i));
  }
}
```

Listing 2.133: A runtime inspect kernel

Each call takes a comma-separated mix of string literals and expressions. The output order across threads is **nondeterministic**.

To inspect one execution path, guard the print with a condition on its index:

```
parallel {px, py} by [8, 16] : block
  parallel {qx, qy} by [16, 16] : thread {
    // ... compute ...
    if (px == 0 && py == 0 && qx == 3 && qy == 5) {
      println("partial sum = ", accum);
    }
  }
```

Listing 2.134: A print inside a parallel block

Without the guard you get one line per thread — thousands of lines for a large grid.

`assert`: runtime invariant checks

Croqtile's `assert` builtin checks an invariant at runtime and aborts the kernel with a message if it fails:

```
assert(stage < MATMUL_STAGES, "stage index out of bounds");
```

Listing 2.135: assert: runtime invariant checks

On the device, this compiles to a `printf` of the message followed by an abort. Use assertions to catch index-out-of-range, null pointer, and other "this should never happen" conditions early.

2.15 Syntax Quick Reference

Checklist

The syntax collected here means the following:

Syntax	Meaning
`dma.copy.async ...`	Start a non-blocking DMA transfer.
`wait f`	Wait until future `f` is ready to consume.
`future.data`	Access the moved tile.
`future.span`	Access the moved tile's shape.
`dma.transp<...>`	Permute dimensions during transfer.
`dma.pad<...>`	Pad during transfer with an explicit fill value.
`.zfill`	Zero-fill out-of-bounds positions in partial tiles.
`.swiz<N>`	Choose a shared-memory swizzle layout.
`chunkat(...)`	Select one equal tile from a regular tiling.
`subspan(...).at(...)`	Select a tile by explicit tile extents and tile coordinates.
`view(...).from(...)`	Select a tile by explicit extents and explicit element offsets.
`tile # i`	Compose tile and in-tile coordinates into a global coordinate.
`#tile`	Ask for the extent of the `tile` axis.

When reading a movement statement, first identify the selected region and its destination, then check when the destination becomes ready. These questions remain the same in a pipeline; the extra buffers simply allow different tiles to be at different points in that sequence.

Summary of New Syntax

Syntax	Meaning
`parallel p by N`	N-way parallelism, index `p` runs 0..N-1 concurrently
`parallel {px, py} by [M, N]`	Multi-dimensional parallel level (`p` and `q` correspond to one parallel level)
`parallel p by A, q by B`	Comma-separated parallel levels (`p` and `q` correspond to different parallel levels)
`parallel q by N : thread`	Map to threads within a block
`foreach index = {m, n, k} in [a, b, c]`	Named tuple destructuring in a `foreach`
`lhs.span(0)`	Extract one dimension of a tensor's shape
`p # m`	Compose outer tile index `p` with inner offset `m`

Section 3.7 replaces scalar arithmetic in the inner loop with matrix contraction operations.

Syntax at a glance

Syntax at a glance

Syntax	Meaning
`if (expr) { ... }`	Predicated execution — runtime conditional, divergent threads masked
`inthreads.async (condition)`	Structured concurrent region — compile-time thread partitioning
`shared event name[N]`	Declare N synchronization tokens
`trigger name`	Signal that a condition is met
`wait name`	Block until the corresponding `trigger` fires
`tile_id = tile_iter # block_id`	Compose indices for tile striping
`int total_tiles = expr`	Local integer variable
`yield`	Early return from `parallel` block

Syntax	Meaning
`dma.copy.async src => dst`	Non-blocking copy (returns immediately)
`dma.any`	Placeholder future (no transfer in flight yet)
`swap(f0, f1)`	Exchange two future handles without copying data
`rotate(f0, f1, f2)`	Cycle three future handles
`with tile_k in N { ... }`	Scoped tile axis binding with extent N
`foreach tile_k(1:)`	Iterate starting from index 1
`__co__ auto fn(...)`	Return type inferred from `return` statement

Where to go next

You can now read most `.co` files. Chapter 3 explains why the constructs are shaped this way and covers the advanced forms; Appendix A is a compact lookup table.

Chapter **Three**

Programming Guide

Chapter roadmap

This chapter organises the same language by function. It explains the structure of a Croqtile program, its type system, symbolic shapes, parallelism, tile selection, data movement, contraction and coordination, and the reasoning behind them. Complete kernels sit beside the feature they demonstrate, so each section also stands on its own.

3.1 Program Structure and Compilation Models

A `.co` file brings the application and its parallel work into one source file. Ordinary host C++ prepares inputs and calls a `__co__` function. That function describes the parallel work and can invoke device helpers for specialized computations. Understanding which compiler handles each part makes both integration and diagnostics easier.

Construct a Croqtile Program

A typical *Croqtile* program is composed of multiple parts, depending on the target platform.

- The Device Program
- The Host Program
- The Tileflow Program

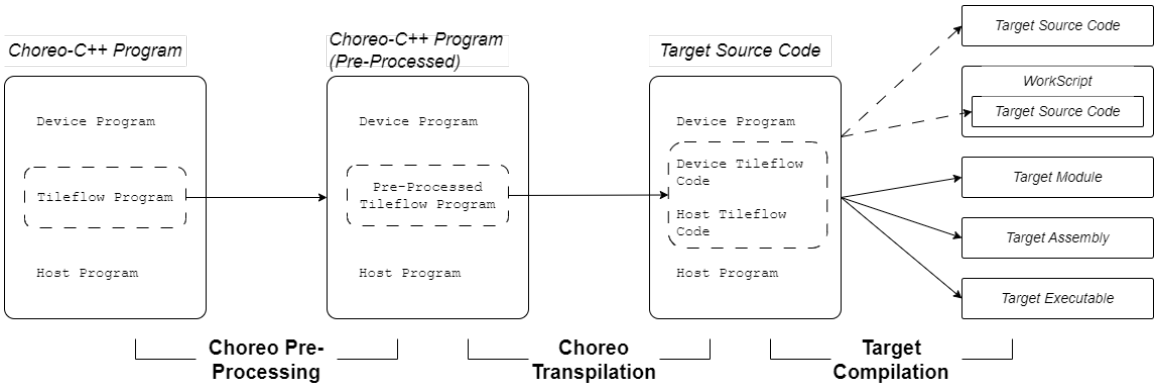
Transpilation and Compilation

The Croqtile compilation process typically involves three major steps: **Pre-processing**, **Transpilation**, and **Target Compilation**. To better understand how different parts of a Croqtile program work together, the full compilation workflow is illustrated below:

As shown in the figure, immediately after *pre-processing*, Croqtile *transpiles* (*source-to-source* compilation) the *tileflow program* into target code, leaving the user-provided *host program* and *device program* unchanged. The *tileflow program* is *transpiled* into host and device source code, which you refer to as *croqtile-generated code*. The compiler then combines the user-provided code and *croqtile-generated* code to perform the *target compilation* process. This process can generate various outputs, such as transpiled source code, workscripts, target modules, target assembly, and executables.

The Croqtile compiler functions as an end-to-end compiler, with the key step being the transpilation of the *tileflow program* into *croqtile-generated code*.

The target determines whether host and device sources are compiled together or separately:



- **Single Source Compilation Model**: The target compiler allows device and host programs to be in a single source file for target compilation.
- **Separate Source Compilation Model**: Host and device code must be compiled separately.

The code shown above naturally supports the *Single Source Compilation Model*. However, to support the *Separate Source Compilation Model*, Croqtile requires wrapping the Device Program with the `__cok__` block, as shown below:

```
__cok__ {
  void device_function(...) { ... }
} // end of __cok__
__co__ void croqtile_function(...) { ... }
void foo() { ... }
```

Listing 3.1: Transpilation and Compilation

The `__cok__` wrapper marks device code that Croqtile must separate from host code before invoking the target compilers. It is needed when the target uses separate host and device compilation. The source can still be maintained as one `.co` file; the wrapper tells the build process where to split it.

Device Program: Parallel Computation

A device helper receives the data selected and staged by its caller. Its job is to compute on that data using the target language. The following addition helper illustrates this boundary: Croqtile arranges the chunks and transfers, while the helper operates on the pointers and element count passed to it.

The device helper from the earlier example is repeated below:

```
__device__ void kernel(int * a, int * b, int * c, int n) {
  for (int i = 0; i < n; ++i) c[i] = a[i] + b[i];
}
```

Listing 3.2: A device helper called from tileflow

For targets that only support the _Separated Source Programming Model_, the code may need to be wrapped within a `__cok__ {}` block. The equivalent code is shown below:

```
__cok__ {
```

```
  extern "C" void kernel(int * a, int * b, int * c, int n) {
    for (int i = 0; i < n; ++i) c[i] = a[i] + b[i];
  }
} // end of __cok__
```

Listing 3.3: The cok wrapper for separate sources

In this separate-source example, `extern "C"` replaces `__device__` because the target requires C linkage for device functions.

The helper runs within the parallel execution arranged by its caller. In this example, the Tileflow function performs the transfers and the helper computes on the selected chunk. Keeping those responsibilities separate lets the transfer structure change without rewriting the helper's arithmetic.

Heterogeneity: Transpiling to program launch

For targets that separate host and device execution, `parallel-by` specifies both the parallel instance count and the boundary between host and device code:

- Code **outside** the `parallel-by` block is transpiled into **host code**.
- Code **inside** the block is transpiled into **device code**, as illustrated below:

```
__co__ void foo(...) {
  // Generate host code
  // ...
  parallel p by 6 {
    // Generate device code
    // ...
  }
  // Generate host code
  // ...
}
```

Listing 3.4: Heterogeneity: Transpiling to program launch

The `parallel-by` statement therefore also marks a device-program launch. Croqtile represents this boundary through the same SPMD parallelism model used to describe the work.

This reference covers every aspect of programming with Croqtile:

1. **Program structure**: How Croqtile programs are organized — the `__co__` function, host-side APIs, and C++ interoperability.
2. **Shaped data**: Croqtile's type system for multi-dimensional tensors — shapes, MDSpan, spanned data, symbolic dimensions, and dynamic shapes.
3. **Loop and parallelism**: SPMD parallelism, loop control, tiling, and iteration constructs.
4. **Data movement**: DMA statements, tiling with `chunkat`, swizzle, and advanced data movement patterns.
5. **C++ embeddings**: Input/output conventions, `__cpp__` inline blocks, macros, and preprocessor integration.
6. **Optimization patterns**: Tileflow optimizations, async DMA patterns, multi-buffering, and performance tuning techniques.

In short A Croqtile program consists of host code, Tileflow code, and device code. Compilation translates the Tileflow functions into target source. The call chain typically runs from the host to a Tileflow function, then to a device function.

Croqtile combines higher-level DMA operations with iteration and tiling, so a compact program can express the full data flow. The following sections explain the syntax and semantics of these constructs.

3.2 Syntax Design and the Type System

Type System Aside: `mdspan` and `ituple`

Consider the information needed to select one tile: the tensor's shape, the number of pieces along each axis, and the coordinates of the chosen piece. Croqtile represents these explicitly with shapes, tuples, and bounded indices. They let a tile expression retain the relationships that would otherwise be spread across offset calculations.

A shaped declaration such as `s32 [128, 256] lhs` fixes both the element type and the rank. Here the extents are constants, so the compiler can evaluate them before execution. It can reject a three-coordinate access to this two-dimensional tensor and check compatible tile shapes. A declaration with symbolic dimensions keeps the same rank information even when some extents come from the caller.

The `span(i)` syntax extracts one dimension: `lhs.span(0)` is 128, `rhs.span(1)` is 256. The shorthand `lhs.span` copies the entire shape.

An `ituple` groups integer values. In `parallel {px, py} by [8, 16]`, the coordinates are also bounded: their current values identify one instance, while their bounds describe the grid. The compiler can simplify the bounds and derived expressions without knowing one fixed current index for all instances. Index calculations that depend on execution still take place at runtime.

Sections 2.4 and 2.5 introduce the declarations. Here the useful distinction is between information known from the program and information supplied by a particular call. Compile-time checks use the former; entry checks and runtime assertions cover constraints that require the latter.

Croqtile aims to simplify the everyday work of developing high-performance kernels, making it easier to express and optimize their computation and data movement.

Embedding Croqtile within C++

Croqtile is a DSL embedded in C++. Its compiler translates Croqtile functions into C++ and implicitly includes `choreo.h` so the generated code can interoperate with ordinary C++. For example:

```
// some C++ code
__co__ void croqtile_function() {
  // croqtile code
}
// another C++ code
void foo() {
  croqtile_function();
}
```

Listing 3.5: Embedding Croqtile within C++

The `__co__` prefix identifies the function body Croqtile translates. Ordinary C++ calls that function by its declared name. The host-facing interface uses shaped buffer wrappers so the call carries dimensions as well as an address.

In addition, certain Croqtile-specific types are introduced to ensure consistency between Croqtile function arguments and their respective callers. Considering that there are some important programming elements missing, you will defer the introduction of such details until the data types are explained.

Variables and Types

Croqtile introduces four fundamental type categories: scalar-type, spanned-type, integer-tuple-type (ituple-type) and bounded-type. These types serve for different purposes.

- **Scalar Type**. It is designed to fulfill the requirement of program control. It consists of *Integer Type* and *Boolean Type*.
- **Spanned Type**. It represents the data type (normally the tensor) for computation. Apart from referencing the raw data, a *spanned type* also associates data with a shape represented by multi-dimensional ranges. The 'multi-dimensional ranges' is named 'mdspan'. It is useful for tiling, etc, which is introduced soon.
- **Integer Tuple (I-Tuple) Type**. It represents a group of integer values. A common usage of *i-tuple* is to introduce the bound of multi-dimensional data.
- **Bounded (Integer/ITuple) Type**. This is the special type that is used to simplify data (sub-zone) reference. And it works with loop construct to iteratively process data.

Scalar and `ituple` values resemble familiar constructs in general-purpose languages. Spanned and bounded types are specific to Croqtile's domain; the following sections explain their roles.

Spanned Types

A spanned type combines a **fundamental element type** with a **multidimensional span (`mdspan`)**. Neither component alone is a complete type for data with storage; Croqtile calls them **partial types**.

Separating these components lets a program manipulate shapes independently of element types. This is useful for multidimensional machine learning data: tiling divides it into smaller blocks, and data movement transfers those blocks between storage levels. Shape calculations describe these operations.

The following examples define these partial types and combine them into complete types for multidimensional data declarations.

The Partial Type: mdspan

An `mdspan` is a partial type: it describes a shape but cannot by itself declare a data buffer. Shape values can nevertheless be defined and manipulated independently. Tiling depends on multidimensional shape rather than element type, so this separation simplifies shape calculations.

The compiler also checks `mdspan` expressions to detect errors as early as possible, before execution when the necessary information is available.

In Croqtile, a mdspan is braced by '[' and ']'. The upper-bound of dimensions inside are separated by ','. That is,

```
mdspan sp : [7, 8];           // defined a mdspan of 2-dimensions
mdspan<4> mds : [a, 3, 4, 1]; // 'a' is an existing integer
d : [c, 4, 28];               // 'd' is not explicit annotated. Type is
    deduced.
```

Listing 3.6: An mdspan declaration

An `mdspan` can be declared explicitly, with an optional rank in angle brackets, or inferred from its initializer, as in the declaration of `d`. The types of `mds` and `d` depend on the values of `a` and `b`. If these values are supplied at runtime, some checks must also occur at runtime.

For a C++ programmer, an `mdspan` describes the ranges of a multidimensional array. In the example, `sp` has two dimensions, with indices from 0 to 6 and from 0 to 7, respectively.

When a mdspan is defined, it is possible to get the integer dimension value using operator '()' over the mdspan.

```
sp : [7, 8];
int b = sp(0) + sp(1);   // 'b' equals to 15 (7 + 8)
```

Listing 3.7: A shape literal

The expression `sp(0)` returns the first extent of `sp`. Extents are integer values and can participate in integer arithmetic.

You can therefore derive a new `mdspan` from an existing one:

```
sp : [6, 8];
spn : [1, sp(0)/2, sp(1)/4]; // define a new mdspan from the existing one.
```

Listing 3.8: A shape with one dimension changed

This makes tiling multidimensional shapes convenient. Because tiling is common in high-performance kernels, Croqtile provides a shorter form for the same calculation:

```
sp : [6, 8];
spn : sp [1, (0)/2, (1)/4];  // spn is defined as [1, 3, 2]
```

Listing 3.9: The same shape again

This version computes the same shape with less code, following Croqtile's aim of expressing common operations concisely.

A named `mdspan` must be initialized when declared and cannot be modified afterwards. To describe a different shape, derive a new name from the existing one. This preserves the relationship between shape expressions for the compiler's analysis.

An `mdspan` can also be defined using operations on whole tuples. The `ituple` discussion explains those forms.

Bounded Types

A bounded integer carries a value and an exclusive upper bound. In `parallel p by 6`, its possible values are 0 through 5 and its upper bound is 6. A tile selection can therefore determine both which piece `p` identifies and how many pieces exist. The `parallel-by` and `with-in` constructs establish this association.

A bounded tuple extends the same idea to several dimensions. Each component has its own bound, and an `mdspan` describes the combined range. For example, a tuple bounded by

`[4, 8]` can identify one of 32 positions. Section 2.7 shows how to declare and iterate over these indices.

In short Croqtile expresses SPMD programs in a C++-style DSL embedded in C++. It reduces the low-level work of moving data between storage levels, particularly through DMA, and is therefore also described as a dataflow DSL. Its purpose is to support everyday kernel development by letting programmers focus on the organization of computation rather than repetitive implementation details.

Feedback from using these constructs helps guide further improvements to the language.

Group of Ranges and Group of Integers

In Croqtile, *mdspan* can be used to represent dimensional shapes. However, as introduced, *mdspan* stands for **Multi-Dimensional Span**. The elements inside are *Dimensional Spans*, which imply ranges of values.

For example, an *mdspan* `[7, 14]` represents a group of two ranges `[0, 7), [0, 14)`. Therefore, an *mdspan* does not represent a list of integer values. However, grouping integer values is useful in certain scenarios:

- When considering the delta of two mdspans, multiple integers are needed. In high-level semantics, the delta may reflect padding differences when adjusting shape dimensions.
- When tiling a shape, the multiple tiling factors are integers, not ranges.

These situations motivate Croqtile to have a type for grouping integer values.

Look-Ahead: Bounded I-Tuples

A bounded tuple attaches ranges to an ordinary integer tuple. For example, a position `{1, 2}` together with bounds `[4, 8]` identifies one position and also describes the space containing it. Loops use the bounds for iteration, and `chunkat` uses them to derive a partition.

Spanned: Data and Buffers

A function's inputs and outputs are often called data, while its temporary storage is called a buffer. Croqtile represents both with spanned types: stored elements associated with a shape. The terminology describes how the storage is used, not a different type system.

The term spanned data is used for either case. An `mdspan` describes the shape; a spanned object combines that description with an element type and access to storage. Naming or deriving a shape alone does not create a copy of the data.

Buffer Lifetime Management

The Croqtile compiler manages the lifetime of storage allocated inside a function. It attempts to reuse buffers whose lifetimes do not overlap, so programmers do not need to manage those allocations themselves.

Automatic allocation does not replace synchronization. If an asynchronous transfer or computation still uses a buffer, that use must finish before the storage is overwritten or released. In a pipeline, the waits and completion operations make these lifetimes explicit enough for the program to reuse the same slots safely.

Bounded Variable

In a `parallel-by` block, the variables defined after the `parallel` keyword are referred to as **Bounded Variables**. Specifically:

- If the variable is an *integer*, it is called a **bounded integer**.
- If the variable is an *ituple*, it is called a **bounded ituple**.

This terminology indicates that the variable not only has a value but also has defined bounds. For example, in the statement `parallel p by 6`, `p` is a *bounded integer* with a bound `[0, 6)`, where `6` is the exclusive upper bound.

The concept of bounded variables will be explored further in later sections, where their usage will be detailed.

Values of Bounded Variables

A bounded variable carries a range and a current value. For an index declared by `with-in`, the following rules describe how `foreach` supplies that current value:

- **Current Value**:
  - Within a `foreach` statement, the current value is determined by the loop iteration, as the bounded variable serves as the iteration variable.
  - Outside of a `foreach` statement, the bounded variable always has a value of zero.
- **Upper-Bound Value**: This is specified in the `with-in` or `parallel-by` statements.

After a `foreach` loop, a `with-in` index has its outside-the-loop value again; it is not a C++ loop variable left at the terminating value. A parallel index follows a different rule: its value identifies the current parallel instance. The example below concerns the sequential `with-in` case.

```
with x in 6 {
  // x's current value is 0
  foreach x {
    // x's current value is either 0, 1, 2, ..., 5
  }
  // x's current value is 0, NOT 6
}
```

Listing 3.10: Values of Bounded Variables

Memory in Croqtile

Storage becomes part of the algorithm when different parallel instances need different views of a tile. The same `[M, N]` shape may describe a global input, a shared staging tile, or a private working buffer. The shape controls indexing; the storage specifier determines where the buffer belongs.

Spanned Types in Croqtile

A spanned type combines an element type, such as `f32`, with a shape. The same shape can be reused with another element type, and a derived shape can describe a tile without changing what each element stores.

The spanned type by itself does not allocate storage for the data. Instead, it serves as a blueprint for defining how the data will be organized in memory, allowing for runtime flexibility and optimizations.

Storage Qualifiers in Croqtile

The storage specifiers `global`, `shared`, and `local` distinguish device-wide storage, storage shared within a block, and thread-private storage on the targets used here. Section 2.6 gives declaration examples. Choose a location from the intended readers and writers of the buffer, then arrange transfers between locations as needed.

3.3 Symbolic Shapes, Shape Evaluation, and Value Numbering

First-Class Citizen: Shape

A raw pointer does not tell a caller whether its elements form a vector, a matrix, or a tile cut from a larger tensor. That missing information has to be supplied elsewhere whenever code computes an offset or checks a transfer. Croqtile keeps shape information alongside the data so declarations, tile selections, and movement statements can refer to the same dimensions.

In contrast, Croqtile enforces code safety and simplifies the programming of shaped data by requiring that **any data declared or used must be associated with a shape**. This motivates Croqtile to treat shape as a first-class citizen.

Evaluation of *mdspan*

An mdspan with constant values is sufficient for many scenarios, as high-performance device kernels often require fine-tuning based on fixed input data shapes. In these cases, the `mdspan`s are evaluated at compile-time, meaning their values do not incur extra execution time or storage overhead.

When a dimension comes from an input, generated host code evaluates the required shape expressions on entry to the Croqtile function. Constant parts can still be simplified beforehand. The amount of runtime work depends on the remaining expressions and checks.

Evaluation of Integers and I-Tuples

Integer and tuple expressions are evaluated at compile time when their operands are known. Expressions depending on runtime indices or input sizes remain computations in the generated program. Immutability helps the compiler reuse and simplify them; it does not make every value a constant.

A kernel often needs to preserve its structure while accepting different input sizes. Anonymous dimensions leave a size unspecified; symbolic dimensions also give that size a name. The name matters when two inputs must agree or when an output dimension is derived from an input.

Fixed and Dynamic Shape

Data with fixed shapes is commonly used in high-performance computing kernels, allowing for aggressive optimization with known dimensions. However, certain scenarios require handling shapes with runtime dimensions, known as dynamic shapes. This necessitates kernel generality to manage inputs of varying shapes, a feature highlighted by many machine learning frameworks.

Improved Code Safety

The reason for improved code safety of using *symbolic dimension* is that more comprehensive code checks can be applied by Croqtile compiler. For example, consider an *anonymous dimension* version of the `Matmul` function:

```
__co__ auto Matmul(s32 [?, ?] lhs, s32 [?, ?] rhs) {
  int tile_m = 32, tile_n = 8, tile_k = 16;
  s32 [lhs.span / {tile_m, tile_k}] tiled_lhs;  // 'mdspan' and 'ituple': rank
     must be same
  s32 [rhs.span / {tile_k, tile_n}] tiled_rhs;  // 'tiled_rhs' can not have a
    zero dimension
  // ...
}
```

Listing 3.11: A kernel with anonymous dimensions

Croqtile performs both compile-time and runtime checks to ensure safety:

- **Compile-time Checks**: Croqtile verifies rank consistency. In this example, `lhs.span` has a rank of `2`, matching the rank of `{tile_m, tile_k}`, so there are no issues.
- **Runtime Checks**: Croqtile generates code to validate dimensions at runtime. For instance, in the declaration of `tiled_lhs`, its shape must not have a dimension of `0`, which would result in an invalid zero-sized buffer. Croqtile's *transpilation* process generates the following *target host code*:

```
void __croqtile_transpiled_Matmul(const choreo::spanned_view<choreo::s32, 2> &
    lhs,
                                  const choreo::spanned_view<choreo::s32, 2> &
    rhs) {
  choreo::runtime_check(lhs.shape()[0] / 32 > 0,
                        "the 0 dimension of 'lhs' may result in spanned data "
                        "with a dimension value of 0.");
  // ...
}
```

Listing 3.12: The transpiled signature

Here, the `croqtile_abort` function aborts program execution if the condition is not met. This early check helps Croqtile detect issues promptly.

For the *symbolic dimension* version, Croqtile compiler generates additional checks in the *target host code*:

```
void __croqtile_transpiled_Matmul(const choreo::spanned_view<choreo::s32, 2> &
    lhs,
```

```
                                    const choreo::spanned_view<choreo::s32, 2> &
      rhs) {
  // additional check happens only when using symbolic dimensions
  choreo::runtime_check(lhs.shape()[1] == rhs.shape()[0],
                        "dimension 'K' is not consistent.");
  choreo::runtime_check(lhs.shape()[0] / 32 > 0,
                        "the 0 dimension of 'lhs' may result in spanned data "
                        "with a dimension value of 0.");
  // ...
}
```

Listing 3.13: The generated shape check

In addition to the checks performed in the *anonymous dimension* version, the *symbolic dimension* version verifies the consistency of dimensions between `lhs` and `rhs`, resulting in safer code.

Therefore, it is recommended that programmers use symbolic dimensions not only for ease of use but also to enhance code safety.

Cost and Limitations

Dynamic dimensions leave less information available during compilation. Some constraints must therefore be checked at runtime. Croqtile places shape checks in generated host code where possible; application-specific requirements may still need assertions.

Anonymous and symbolic dimension syntax is restricted to spanned-data parameters. An internal buffer can derive its shape from an input, but it cannot introduce a new unspecified dimension with `?` or a new symbolic name. The following declaration is rejected:

```
__co__ void foo() { f32 [M] d;} // compile-time error: can not applied to the
    spanned buffer
```

Listing 3.14: Cost and Limitations

In short Anonymous dimensions express an unknown size. Symbolic dimensions express an unknown size together with a relationship the program can name. Reusing `K` in two operands records that their contraction dimensions must agree. Croqtile checks the parts it can establish statically and generates checks for relationships that depend on actual input dimensions.

a. Improved Readability and Clarity

Using symbolic dimensions like `M` and `N` explicitly in the shape definition makes the program **easier to read** and understand. Instead of seeing a `?` or `-1` and wondering what the dimension is supposed to represent, the reader can immediately grasp that `M` and `N` are variables that depend on runtime values. This improves **maintainability** and reduces the cognitive load for developers who are reading and modifying the code.

For example, two question marks identify two unknown extents; they do not state that those extents are equal. A repeated symbolic name records that equality directly in the function signature:

```
// Using symbolic dimensions
mdspan sp : [M, N];
mdspan spn : sp [1, M / 2, N / 4];
```

```
// Clear how the new shape is derived
// Using '?' or '-1'
mdspan sp : [? , ?];
```

Listing 3.15: a. Improved Readability and Clarity

The anonymous form remains useful when the dimensions are independent. Choose names when the relationship between them is part of the operation.

b. Type Safety and Compile-Time Checking

Symbolic dimensions let the compiler reason about expressions without knowing every numerical extent. It can check ranks and simplify relationships such as repeated uses of `M / 2`. This does not eliminate runtime validation: whether caller-supplied dimensions agree or satisfy a required constraint may only be known when the function is entered.

The expression below keeps the derived shape tied to the named input dimensions. That connection is available to the compiler and visible to the reader, instead of being maintained through separate size arguments:

```
mdspan sp : [M, N];
// M and N are symbolic dimensions
mdspan spn : sp [1, M / 2, N / 4];
// Compiler ensures validity of shape expressions'
```

Listing 3.16: b. Type Safety and Compile-Time Checking

This ensures type consistency and reduces the potential for errors during runtime.

c. Expressiveness and Flexibility

A named dimension can appear in several shape expressions. For example, a buffer may use half of M along one axis and a padded N along another. These expressions show how the new shape depends on the input dimensions.

An anonymous dimension can still be read through the input's shape. A symbolic name is useful when that same value occurs in several expressions: changing one derivation then leaves its relationship to the other dimensions visible.

For example, you can easily express relationships between dimensions using symbolic variables:

```
mdspan sp : [M, N]; // Symbolic dimensions
mdspan spn : sp [1, M / 2, N / 4]; // Derived shape based on symbolic
    expressions
```

Listing 3.17: c. Expressiveness and Flexibility

The benefit is that the dependency is written where the shape is declared. The program still has to satisfy the resulting size and divisibility requirements for each call.

d. Compile-Time Optimization

Symbolic expressions give the compiler opportunities to reuse a dimension calculation and simplify expressions that share known relationships. Constant parts can be evaluated during compilation; parts depending on input sizes remain runtime calculations. A symbolic name

preserves information for optimization, but it does not turn an unknown input size into a constant or guarantee a particular performance result.

Introduction to Croqtile Value Numbering

In Croqtile, the value numbering process serves to facilitate Shape Inference by either directly inferring shapes known at compile time or generating shape computation expressions.

While many value numbering approaches focus on eliminating redundant computations (expressions that consistently produce the same value), Croqtile's value numbering aims to establish a progressive method for simplifying the Croqtile domain-specific construct 'mdspan'. In this context, the application of constant propagation, algebraic simplification, and other techniques becomes more essential.

In terms of the implementation method, Croqtile utilizes a simple **scope-based** value numbering approach, an extension of the local value numbering method, complemented by a lexical-scoped value number table[1]. Unlike other value numbering methods, Croqtile does not necessitate the construction of basic blocks, pruned-SSA, or a compiler backend, as it applies value numbering directly onto the Abstract Syntax Tree (AST). However, to facilitate this approach, certain restrictions must be imposed on Croqtile's syntax rules, particularly to prevent value mutation.

Quick Example

The value numbering process assigns a value number to all entities containing a list of values, including for mdspan and ituple. For example,

```
mdspan a : {1, b, c + 3}
```

Listing 3.18: A shape expression

The value numbering process would produce values including:

```
   VN #0:  const_1
   VN #1:  b
   VN #2:  c
   VN #3:  const_3
   VN #4:  +:#2:#3
   VN #5:  #0,#1,#4
```

Listing 3.19: Its value-number graph

A number after "VN #" identifies an expression's value. It may represent a constant integer, a symbolic value, or a combination of other value numbers. Here, #0 and #3 refer to constants; #1 and #2 refer to the integer variables `b` and `c`. Value #4 represents the sum of #2 and #3, which expands to `c + 3`. Value #5 represents the list of values in `mdspan a`. Indexing `a(0)` selects #0, the constant 1.

From Expressions to Value Numbers

The current value-numbering implementation follows the syntax tree. It traverses the abstract syntax tree and assigns a value number whenever a node contains an expression.

The method of generating value numbers is simple. For each expression, it takes the expression as the input, and returns corresponding value number.

More specifically, it:

- Generates a signature from an expression.
- If the signature already exists, return the associated value number by looking up the value number table.
- Or else, it associates the signature with a new value number in the value number table, and return it.

For simplicity, the current implementation represents an expression signature as a string. A hash-based representation could replace it later, as in many value-numbering implementations.

In addition, to handle the symbol definition and symbol reference, it follows the following rules:

- When an expression is to initialize a symbol (variable or partial type), the symbol is assigned the same value number of the expression. More specifically, it associates the signature of the *scoped* symbol name with the value number in the value number table.
- When an expression references a symbol, it looks up its corresponding signature in the value number table, and return it.

Note the symbol must use its *scoped name* for signature generation, since identical names are allowed to be defined in different scopes.

Simplifications

Every time the value numbering process generates a signature, it tries to simplify the expression. For example,

```
1    int a = 0 + 1;
```

Listing 3.20: A constant expression

Given the simple integer expression, the value numbering process could generate signatures and associated value number as illustrated:

```
1    VN #0:  const_3
2    VN #1:  const_8
3    VN #2:  +:#0:#1
4    Alias a -> #2
```

Listing 3.21: The simplified graph

Here, without optimizations, two constant values #0 and #1, and a new value number #2, which is an addition of #0 and #1 are generated. #2 has the signature of "+:#0:#1". However, considering that addition of constants can be evaluated at compile time, simplification could be performed. The following output showcases the optimized value numbering process:

```
1    VN #0:  const_3
2    VN #1:  const_8
3    VN #2:  const_11
4    Alias a -> #2
```

Listing 3.22: The folded graph

The signature of #2 now represents a constant, which implies the algebraic simplification is applied successfully.

From Value Numbers to Expression

After value numbering and simplification, each expression refers to its simplified value number. Croqtile uses these results to infer `mdspan` shapes.

An `mdspan` describes the number of elements in each dimension. Together with a fundamental element type, it determines the storage size of a spanned object.

Some dimensions are determined at runtime; others can be fully evaluated at compile time. Known dimensions allow storage requirements to be determined in advance. Otherwise, the compiler generates code to calculate the shape at runtime.

For nonconstant values, the compiler must reconstruct expressions from the value numbers.

Lexical Scopes and Restriction

Croqtile applies value numbering globally across the abstract syntax tree. This syntax-directed approach does not construct basic blocks or perform data-flow analysis, so it cannot infer shapes whose dimensions change through reassignment.

For a reader of generated code, the consequence is straightforward: a constant shape may disappear into constants, while a symbolic shape becomes a calculation in the generated program. Value numbering tries to preserve and simplify those relationships. It does not prove every mathematical identity, so an expression that seems equivalent by hand may still need a clearer formulation.

Limitations

More specifically, it has to set two limitations:

1. Any mdspan cannot have element with bounded integer type, or produced from bounded ituple.

2. Simple integer type and ituple type cannot be re-defined.

Reference

[1] Briggs P, Cooper K D, Simpson L T. Value numbering[J]. Software: Practice and Experience, 1997, 27(6): 701-724.

3.4 Parallelism, Space Specifiers, and Masking

Parallelism: How Croqtile Represents Parallel Work

The previous two chapters kept things simple: `parallel {i, j} by [4, 8]` created 32 instances, each handling one element, and you never asked where those instances actually ran. That was fine for element-wise addition and for understanding data movement — the compiler picked sensible defaults.

How You Think About Parallel Tasks

Before you touch any Croqtile syntax, let's talk about what "parallel" actually means.

Suppose you have eight independent tasks — eight tiles to add, eight rows to process, whatever. You can describe all eight at once and call them "parallel." But that statement says nothing about how many actually run at the same time. On a single CPU core, they execute one after another. On a 4-core CPU, maybe four run simultaneously.

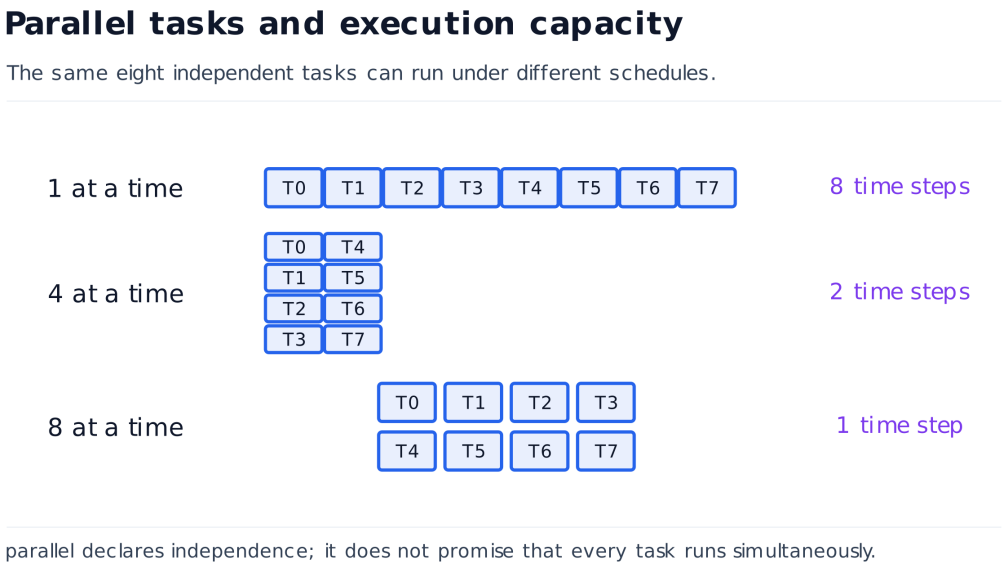


Figure 3.1: The same eight tasks take eight, two, or one time steps when one, four, or eight tasks can run at once.

Parallelism describes which tasks can be independent. The execution resources determine how many of those tasks can make progress at once.

When tile counts and launch dimensions are written as the same set of constants, changing the tiling also forces a change to the resource mapping. Keeping the two descriptions separate makes those choices easier to revise.

Croqtile uses `parallel` for the independent work and space specifiers for its mapping. The work can therefore be read before deciding whether an axis belongs to blocks, threads, or cooperating groups.

Matrix Multiply: Putting It Together

With `parallel`, `foreach`, `dma.copy`, `chunkat`, `#`, and space specifiers, a tiled matrix multiply is within reach.

The plan: multiply a `[128, 256]` matrix by a `[256, 256]` matrix to get a `[128, 256]` result.

Control Structures

Croqtile differs from C++ in how it expresses parallelization and loops, but it follows C++ in using `if-else` blocks to handle branches inside a program.

Sequentialism and Parallelism

Repetitive tasks are common in programming.

Currently, the most widely used parallelization paradigm is **Single Program, Multiple Data (SPMD)**. In this paradigm, code is written once but executed as multiple instances simultaneously. Each instance operates independently unless synchronization is explicitly implemented. Croqtile adopts this paradigm and abstracts parallelism using the **parallel-by** block.

A sequential loop is needed when later work depends on earlier work, such as accumulating successive K slices. Croqtile separates the bounded range declared by `with-in` from the traversal performed by `foreach`. This also permits work before and after the traversal, as used in buffered pipelines.

The following sections explain how Croqtile constructs loops and expresses parallel work.

A `parallel-by` block declares a set of independent instances; the space specifier says how those instances are grouped.

Parallel Execution Block

A **parallel execution block** contains code executed by multiple instances. Each instance runs the same code with its own parallel index.

In short A `parallel-by` block declares independent instances and a space specifier groups them; masking then decides which of those instances actually execute a statement.

Loops remain necessary for work that has to happen in order, and Section 2.7 introduces their Croqtile forms.

Parallelization and Masking

SPMD programs often launch a fixed number of parallel instances even when the work does not divide evenly among them. Masking limits execution to the instances with valid work and leaves the others inactive.

Croqtile supports both explicit masks and masking implied by the surrounding code structure.

Explicit Masking/Condition

A thread mask describes which SPMD instances execute a region. Within the code for a single instance, it appears as a Boolean condition, usually a comparison involving a thread identifier.

In the examples that follow, Croqtile experimentally supports explicit masking of the `foreach` block, which masks off loop code as a whole. For example:

```
__co__  void foo() {
  parallel p by 6 {
    with q in [2] {
      foreach q if (p == 0) {
        // this iteration only work for parallel thread 0
      }
    }
  }
}
```

Listing 3.23: Explicit Masking/Condition

Implicit Masking

Some code structures in Croqtile imply implicit masking. One typical situation is the multi-level `parallel-by` block. For example:

```
__co__  void foo() {
  parallel p by 6 {
    // parallel-level-0:
    //   code here is as if implicitly masked 'if (q == 0)'
    parallel q by 2 {
      // parallel-level-1:
      //   code here is for all 'q' threads
    }
    // parallel-level-0:
    //   code here is as if implicitly masked 'if (q == 0)'
  }
}
```

Listing 3.24: Implicit Masking

This example has two parallel levels. Code inside `parallel p by 6` but outside `parallel q by 2` belongs only to the outer level. If the target launches two inner instances for that region, the outer-only code behaves as though guarded by `if (q == 0)`.

A condition can also guard a `foreach` loop, controlling execution of the loop as a whole.

Experimental: Optional *Where-Binding* for `within`

You can append a *where-clause* to impose **Where-Binding** constraints among different bounded variables defined in a `with-in` statement. This allows different bounded variables to be treated as aliases for each other. Below is an example:

```
with {m, n} in [M, N], {n_p, k} in [N_P, K]
where n_p <-> n {
  // matmul implements with m, n, K. 'n_p' always refers to the same value as
    'n'.
}
```

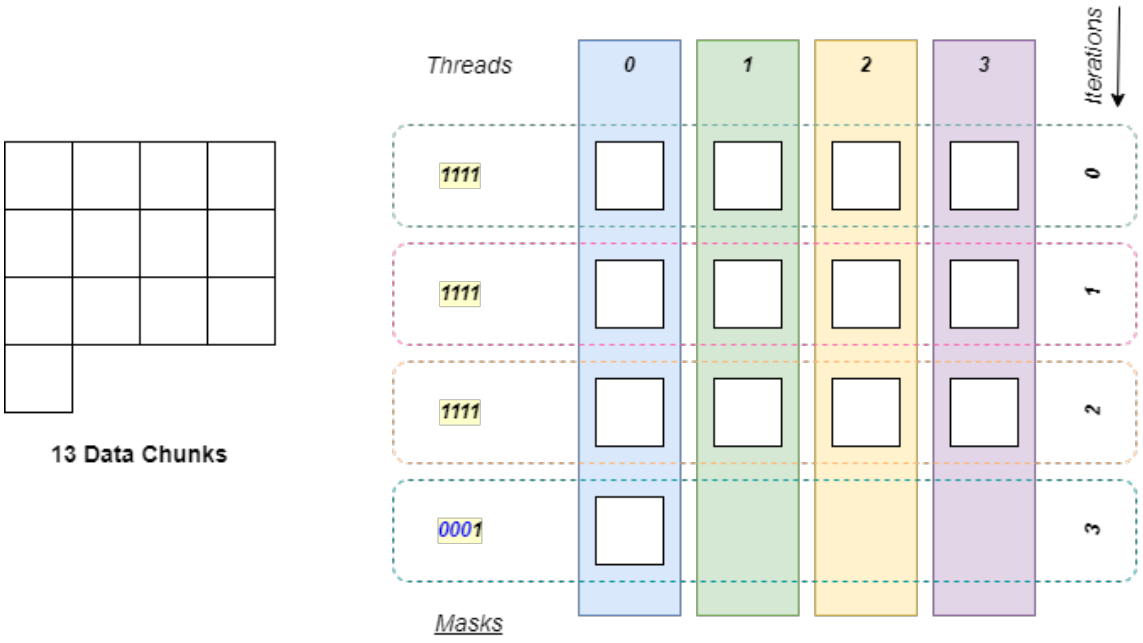
Listing 3.25: Experimental: Optional Where-Binding for within

Syntactically, the `<->` operation establishes the *where-binding* between two bounded variables. Within the `with-in` block, any reference to `n_p` refers to `n`, and vice versa.

Croqtile primarily uses SPMD (Single Program, Multiple Data), but thread masking also allows different instances to run different code, expressing MPMD (Multiple Program, Multiple Data) behavior. The `inthreads` block selects these divergent regions.

Masking in SPMD

It is typically necessary to mask off some *parallel threads* in SPMD programming. For example, programmers often divide data into chunks for parallel or iterative processing. However, there may be a remainder that cannot be evenly distributed across all threads.



The figure distributes 13 data chunks across four parallel instances in four iterations. All instances are active for the first three iterations, giving mask `1111`. Only instance 0 has work in the final iteration, giving mask `0001` when the least significant bit represents instance 0. A `1` denotes an active instance and a `0` an inactive one, preventing unnecessary processing of nonexistent chunks.

Uniform and Divergence

From the SPMD programmer's perspective, code **diverges** when masking causes different instances to follow different paths. An SPMD program can therefore contain:

- **Uniform Code**: *All threads* execute the same code path.
- **Divergent Code**: Only the *masked threads* execute the *divergent* code path.

It is necessary to have the *divergent* code in SPMD code to handle different real-world requirements. And from a perspective of different programming paradigm, applying *divergent* code in SPMD programs can even mimic MPMD (Multiple Program Multiple Data) behavior. In MPMD, different threads execute different code. Therefore, *divergence* can extend SPMD to MPMD scope conceptually.

Use an `inthreads` block to select the instances that execute a divergent region.

Masking for Multi-Levels

Thread masking can be applied at different parallel levels. The following example demonstrates this:

```
__co__  void foo() {
  parallel p by 3, q by 4 {
    inthreads (p < 2 && q == 0) {
      // divergent code for thread (0, 0), (1, 0)
      // thread-mask: 011-0001
    }
```

```
    inthreads (q == 1) {
      // divergent code for thread (0, 1), (1, 1), (2, 1)
      // thread-mask: 111-0010
    }
  }
}
```

Listing 3.26: Masking for Multi-Levels

This code uses two levels of parallelization:

- Outer level (`p`): Parallel count of 3
- Inner level (`q`): Parallel count of 4

The first `inthreads` predicate (`p < 2 && q == 0`) restricts execution to:

- Outer parallel IDs: `0, 1` (excluding 2)
- Inner parallel ID: `0` only

The thread mask (`011-0001`) reflects these constraints, where the two levels are separated by a hyphen (-).

The second predicate (`q == 1`) imposes no restriction on the outer level (`p`), allowing all outer IDs (`0, 1, 2`) while restricting the inner level to `q == 1`.

In short Masking is what allows divergent execution inside an otherwise uniform parallel region. Developers can use either explicit thread masking via `inthreads` blocks or implicit control through `parallel-by` constructs to restrict execution to specific threads.

Masking identifies who executes a region. Whether the other instances wait at its end depends on the synchronization form. Section 3.8 develops asynchronous regions, where the program must express data dependencies with events rather than rely on a join after each region.

Worked Example: Tiled Matrix Multiplication

Scalar Matmul (No DMA)

The first version uses `parallel` to divide the output and `.at()` to read individual elements. It does not yet use explicit data movement.

```
__co__ s32 [128, 256] matmul(s32 [128, 256] lhs, s32 [256, 256] rhs) {
  s32 [lhs.span(0), rhs.span(1)] output;
  parallel {p, q} by [16, 64] {
    foreach {m, n} in [128 / #p, 256 / #q]
      output.at(p#m, q#n) = 0;
    foreach index = {m, n, k} in [128 / #p, 256 / #q, 256]
      output.at(p#m, q#n) += lhs.at(p#m, k) * rhs.at(k, q#n);
  }
  return output;
}
```

Listing 3.27: Scalar Matmul (No DMA)

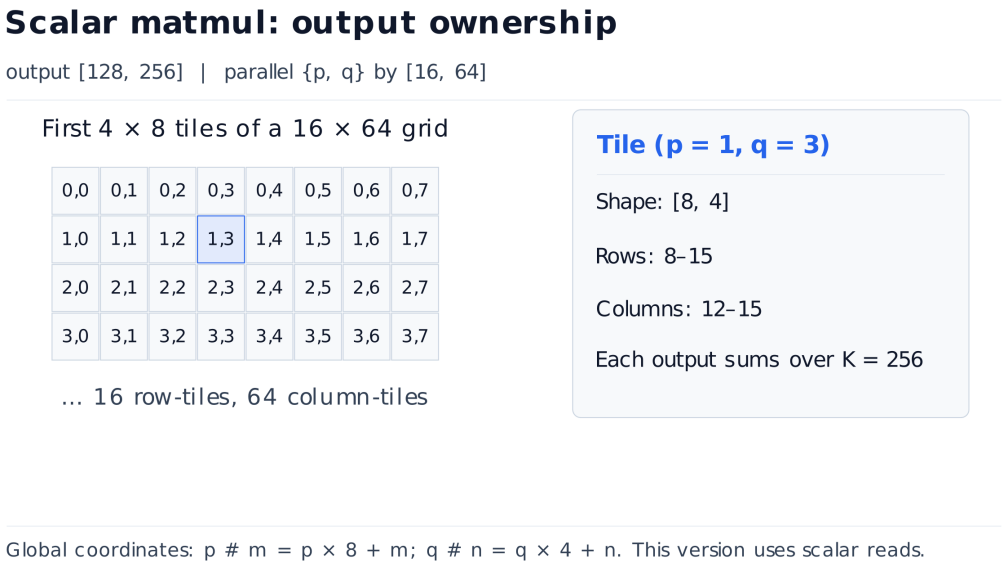


Figure 3.2: The [128, 256] output has a [16, 64] tile grid. Each tile has shape [8, 4]; only the first few tiles are drawn.

The tile dimensions follow from the parallel bounds:

The output declaration derives 128 rows from `lhs.span(0)` and 256 columns from `rhs.span(1)`. The contraction dimension appears in both inputs but does not become an output dimension.

The declaration `parallel {p, q} by [16, 64]` gives one parallel level two coordinate axes. Its 1,024 index pairs partition the output into tiles of eight rows and four columns.

The loop binds `m`, `n`, and `k` as components of `index`. The first two visit positions inside the output tile; the third traverses the contraction dimension. Their trip counts follow directly from the output partition:

- `128 / #p = 128 / 16 = 8` rows per tile
- `256 / #q = 256 / 64 = 4` columns per tile
- `256` for the full contraction dimension

The composed row `p#m` has value `p * 8 + m`. As p visits the 16 row tiles and m visits the eight rows within each tile, it covers rows 0 through 127. The column expression `q#n` works the same way with four columns per tile.

For each output position, the body sums `lhs.at(p#m, k) * rhs.at(k, q#n)` over K. All contributions go to the same output element. The zeroing loop runs in the same parallel instance, so each accumulator element is cleared by the instance that owns it.

Tracing One Output Element

Trace output position `(37, 50)` using the block-and-thread decomposition of the DMA matmul in Section 3.6.1. That example uses an 8 by 16 block grid, with each block handling a 16 by 16 output tile. This differs from the 16 by 64 logical grid in the scalar example immediately above; both cover the same output shape.

Blocks partition 128 rows into 8 tiles of 16: `px = 37 / 16 = 2`, offset `qx = 37 % 16 = 5`. Columns partition 256 into 16 tiles of 16: `py = 50 / 16 = 3`, offset `qy = 50 % 16 = 2`. So block `(2, 3)`, thread `(5, 2)`.

For that thread, the K-loop runs 16 iterations. On iteration `tile_k = 0`, `dma.copy` loads `lhs` rows 32..47 and K columns 0..15 into shared `lhs_load`, and `rhs` K rows 0..15 and columns 48..63 into shared `rhs_load`. The inner `foreach k in [16]` accumulates `lhs_load.data.at(5, k) * rhs_load.data.at(k, 2)` for k = 0..15 into `output.at(37, 50)`. Then `tile_k = 1` loads the next K-strip, and so on. After all 16 iterations, `output.at(37, 50)` holds the complete dot product — same answer as the scalar reference.

3.5 Tile Selection and Data Expressions

`chunkat`, `subspan`, and `view().from()`: Three Ways to Select a Tile

A tile expression answers two questions: how large is the region, and where does it begin? With `chunkat`, bounded indices provide the partition counts and select a piece. With `subspan(...).at(...)`, the tile shape is explicit and the coordinates count tiles. With `view(...).from(...)`, both the shape and the starting element coordinates are explicit. The distinction matters whenever a window starts between regular tile boundaries.

`#`: Compose and Extent

The unary form `#p` retrieves an index's upper bound. The binary form `p # i` combines an outer index with an inner one: its value is `p * #i + i`, and its bound is `#p * #i`. For eight tiles containing 16 elements each, tile 2 and inner element 5 identify element 37 in a range of 128 elements.

Handling irregular access

Regular tiling works well when the dimensions divide evenly and the origin follows a fixed grid. Other kernels need a different description: expert batches can start at runtime offsets, convolution windows can overlap, and a final tile can contain fewer valid elements. A flat input may also need a multidimensional view before it can be consumed.

The following operations adjust the selected region or its shape while leaving the surrounding algorithm recognizable. The resulting view still has to satisfy the chosen transfer operation's requirements, including shape, layout, and any target-specific alignment constraints.

Tile and Flow

It is possible to use the `chunkat` expression for either the source or the destination of a *DMA statement*. When `chunkat` is used only for the source, it typically divides the input or buffer into smaller chunks to move it to storage closer to the processor. In contrast, when `chunkat` is used only for the destination, it typically moves the buffer to compose part of the output. For example:

```
1 __co__ auto foo(f32 [36] input) {
2   f32 [36] output;
3   parallel p by 6 {
4     f0 = dma.copy input.chunkat(p) => shared;  // tile the source
5     dma.copy f0.data => output.chunkat(p);     // un-tile the destination
6   }
```

```
  return output;
}
```

Listing 3.28: Tile and Flow

In this example, the statement applies `chunkat` to the destination, moving the data block in `shared` and "un-tiling" each block to the output. Note that the Croqtile compiler **requires the source data and destination data to have the same shape and element type** if not using *storage* as the destination *data expression*.

Using the `chunkat` expression for both the source and destination is rarely supported by any target platform at this time.

Compose Two Bounded Variables for Tiling with

The RGB-to-grayscale example in Section 3.10.2 combines a worker index q with a sequential image index n. The expression `q # n` assigns consecutive images to each worker. For N = 18 and six workers, the inner bound is 3, so worker 1 processes images 3, 4, and 5. The details below use that same partition.

With six workers, q has bound 6 and n has bound N / 6. Their composition has bound N and value `q * (N / 6) + n`. The remaining indices h and w choose spatial tiles; the channel dimension stays whole. Thus the worker assignment and the local traversal jointly identify the source tile and its output position.

Changing the operand order changes the assignment. For N = 18, `n # q` would give worker 1 the images 1, 7, and 13 as n advances, instead of the consecutive images 3, 4, and 5 selected by `q # n`. The operator makes this scheduling choice explicit in the data expression; its performance depends on how the resulting accesses behave on the target.

Arbitrary-Offset Windows: view and from

A mixture-of-experts kernel often packs different numbers of tokens for different experts. Each expert's data begins at an offset known only at runtime, so a fixed tile grid may not describe the required slice. A window gives that slice its own origin before the inner tiling begins.

The expression `view(M, N).from(row, col)` describes an M by N rectangle beginning at the supplied element coordinates. The selection need not begin on a tile-grid boundary. A transfer using that view must still satisfy its own alignment and shape requirements:

```
patch = matrix.view(16, 16).from(37, 50);
```

Listing 3.29: A window at a constant offset

The expression selects a `[16, 16]` slice starting at row 37, column 50. The origin can be any runtime value.

The power of `view` / `from` is composability: once you carve out a window, all downstream operations — `subspan`, `chunkat`, `dma.copy`, `dma.copy` — apply to the window as if it were a standalone tensor. The pipeline stays unchanged; only the origin shifts:

```
expert_lhs = lhs.view(expert_M, K).from(expert_offset, 0);
dma.copy expert_lhs.subspan(WARP_M, TILE_K).at(block_m, iv_k) => shared;
```

Listing 3.30: A window at a runtime offset

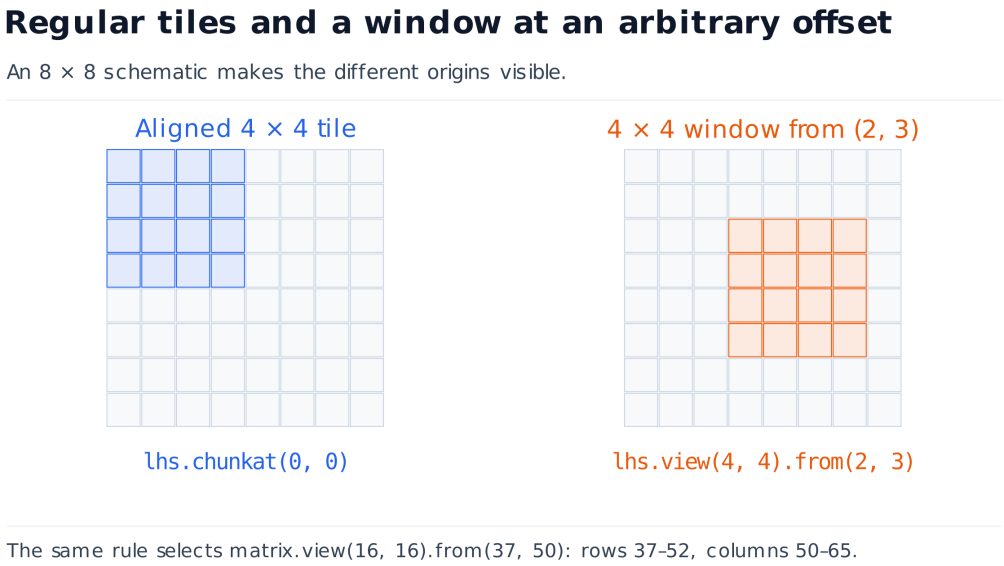


Figure 3.3: The small example uses a [4, 4] window from (2, 3); the same rule selects the [16, 16] window from (37, 50) in the code.

In an MoE stack, each expert's token batch starts at a dynamic row `expert_offset`. The `view` / `from` call rewires the operand at the top of the pipeline; everything below — DMA, staging, MMA loop — runs the same code for every expert.

Use `chunkat` when the partition counts already describe the desired region. Use `view` and `from` when the origin is an explicit element offset, then apply any regular tiling within that window. This keeps the irregular outer selection separate from the repeated inner work.

Strided Tiles: subspan, step, and at

A `subspan(M, K).at(i, j)` expression uses tile coordinates. Without an explicit step, its element origin is `(i * M, j * K)`. Specifying a step changes the distance between successive tile origins while leaving the selected tile extent unchanged.

Adding `.step(sM, sK)` overrides the stride: tiles are spaced `sM` rows and `sK` columns apart instead of being packed:

```
1 matrix.subspan(16, 16).step(32, 32).at(i, j);
```

Listing 3.31: Strided tiles: .subspan, .step, and .at

Tile `(0,0)` starts at `(0,0)`, but tile `(1,0)` starts at row 32 (not row 16), leaving a 16-row gap. Omitting `.step` uses a step equal to the tile size — the packed case.

A step smaller than the tile extent creates overlapping windows; a larger step leaves gaps. For example, a 16-row window advanced by eight rows shares half its rows with the next window. The step controls the origin sequence, so it should not be confused with a stride between elements inside one tile.

Layout Reinterpretation with span_as

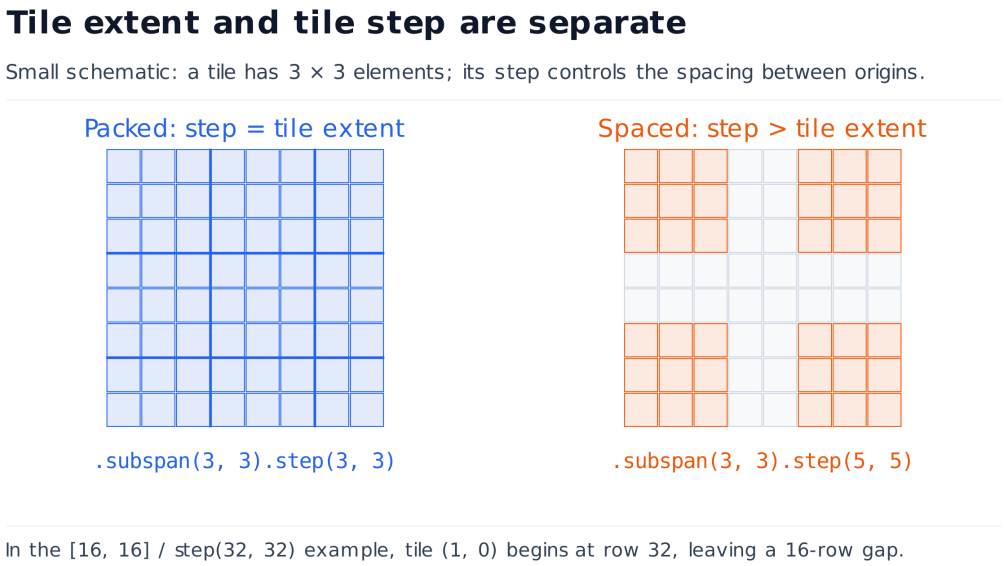


Figure 3.4: The small diagram contrasts packed [3, 3] tiles with step(5, 5). In the code, [16, 16] tiles with step(32, 32) leave 16-element gaps.

Layout reinterpretation: `span_as`

Some kernels load data as a flat 1D strip but need to feed it to MMA as a 2D matrix. A naive approach would copy the data into a reshaped buffer — `span_as` avoids this by reinterpreting the existing buffer's shape in place:

```
flat_buffer.span_as([rows, cols])
```

Listing 3.32: A flat strip reinterpreted as a matrix

Element count is preserved; only the logical rank changes. No data is moved.

```
strip_load = dma.copy data.chunkat(tile) => shared;
tile_2d = strip_load.data.span_as([tile_m, tile_k]);
ma = mma.load tile_2d.chunkat(_, iv_warp);
```

Listing 3.33: A movement statement using the reinterpretation

The product `rows * cols` must equal the number of elements in the underlying region. A reshape preserves linear element order, so it cannot transpose a matrix or remove padding between rows. When the dimensions are constants, the compiler can check the element count before execution; dimensions derived from inputs must satisfy the same invariant for the actual call.

Tile Selection: Summary and Syntax

In short

Syntax at a glance Section 3.6 takes the tile expressions above and uses them as the source and destination of movement statements. For the cases where a program needs plain

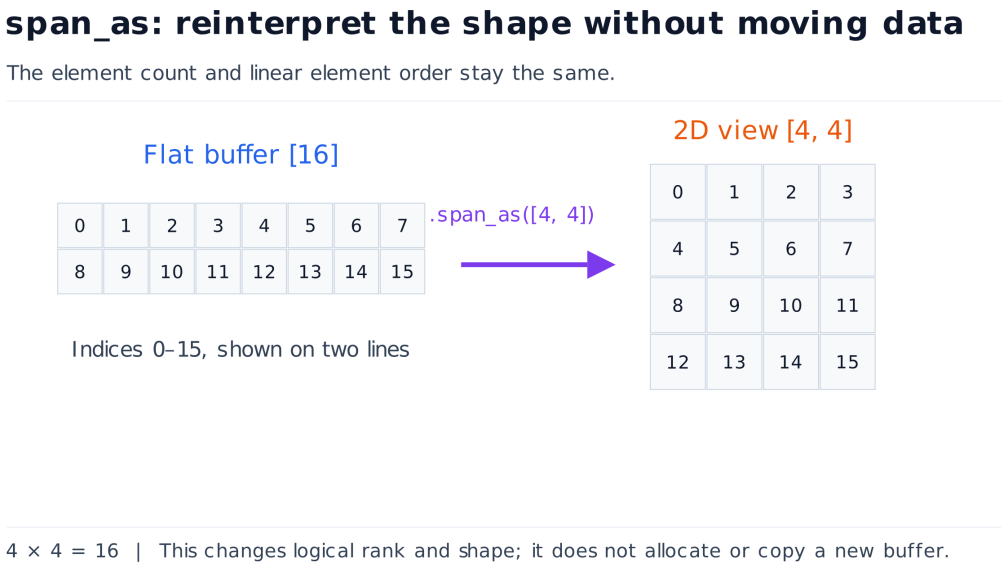


Figure 3.5: The 16-element buffer becomes a [4, 4] view with the same linear order. The 1D strip is displayed on two lines for readability.

Concept	Syntax	Role
Software DMA	`dma.copy` / `dma.copy.swiz<N>`	Thread-cooperative tile transfer;
Swizzle	`.swiz<N>` on copy + `mma.load.swiz<N>`	Shared-memory remapping; matching modes on copy and load
Arbitrary windows	`view(M,N).from(r,c)`	Ragged or runtime-positioned slices
Strided tiling	`.subspan().step().at()`	Non-packed layouts, overlapping stencils
Partial tiles	`.zfill`	Zero-fill out-of-bounds elements
Shape reinterpretation	`span_as([dims])`	Zero-copy reshape for staging buffers

Syntax	Meaning
`dma.copy.swiz<N> src => dst`	Copy with swizzle width
textttN in bytes	
`mma.load.swiz<N> src`	MMA operand load consistent with swizzle `N`
`.view(M, N).from(r, c)`	Arbitrary-offset `M x N` window
`.subspan(M, K).step(sM, sK).at(i, j)`	Strided tile selection
`.zfill`	Zero-fill out-of-bounds elements on copy source
`span_as(dims)`	Reinterpret linear storage as shaped tensor

C++, Section 2.13 collects the `__device__` functions, preprocessor guards and injected source text.

3.6 Data Movement: Operations, Configuration, and Async Patterns

What You Have So Far

The constructs introduced so far assemble into one skeleton: a `__co__` function with typed, shaped parameters and a return value; `parallel` and `.at()` for element-wise computation; `lhs.span` for deriving output shapes from inputs; and the host-side `make_spandata` / `.view()` API for creating and passing tensors.

Section 2.9 introduces `dma.copy` and `chunkat` to express block-level data movement.

Data Movement: The Tile Movement Engine

Section 1.3 used scalar indexing: pick one position, read inputs, compute, write output.

A tiled implementation instead selects a region, stages it in the storage used by the computation, and reuses it there. Croqtile expresses the transfer directly, including transformations that belong to the movement itself:

- `dma.copy` moves a tile as-is.
- `dma.transp<...>` permutes dimensions while moving it.
- `dma.pad<...>` enlarges a tile and fills the border.

This part of the guide introduces that model, shows the core syntax, and gives the rules you need to write DMA code that is both correct and likely to lower well.

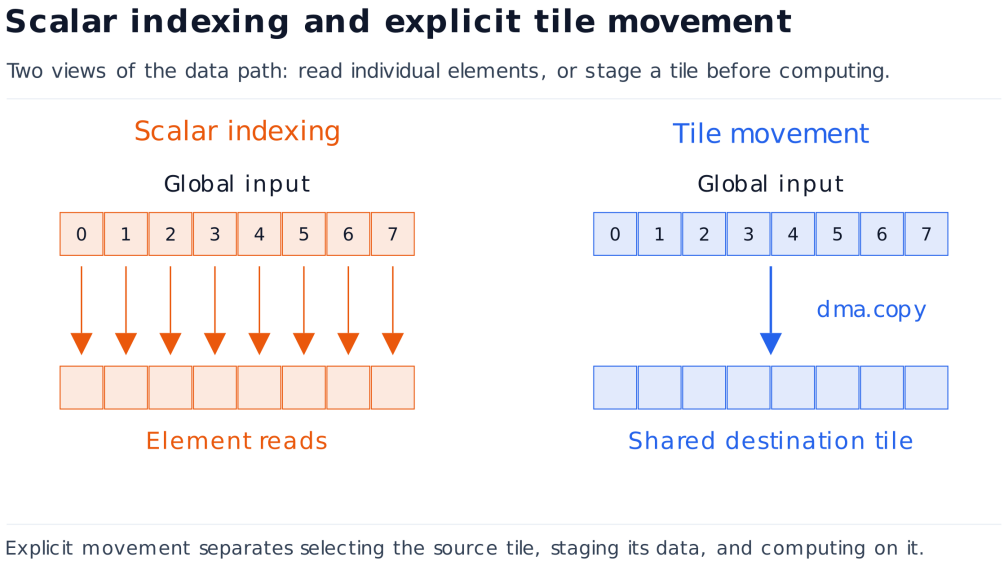


Figure 3.6: Scalar indexing reads individual elements; explicit tile movement stages a selected tile into shared storage.

Other Operations: `transp` and `pad`

A plain copy preserves the source's shape and element order. Use `transp` when the destination needs a different dimension order, or `pad` when it needs extra positions filled with a specified value. Treat the destination shape as part of the operation: later tile selection and computation must use the transformed shape, not the original one.

Async Operation: the DMA Statement

Alongside parallel execution, Croqtile provides asynchronous DMA statements for data movement.

An asynchronous DMA operation runs alongside the SPMD code. Like an asynchronous CPU thread, it allows other work to continue, but its behavior is restricted to the configured data transfer.

The following code showcases one basic DMA statement.

```
1 global f32 [10] g_data;
2 local f32 [10] l_data;
3 f = dma.copy.async g_data => l_data;
4 // ... async operations
5 wait f;          // explicit wait
```

Listing 3.34: An asynchronous DMA declaration

The assignment names `f` as the **future** of the asynchronous DMA operation. Use `wait f` to pause the current instance until the transfer completes. Until then, the instance can continue executing alongside the transfer.

Explicit temporary-buffer declarations can be repetitive. Croqtile also allows the following shorter form:

```
1 global f32 [10] data;
2 f = dma.copy data => local;
3 ... f.data;  // retrieve the 'local' data from the future
```

Listing 3.35: The same declaration with storage

Here, you do not need to specify the exact target location where the DMA transfers data. Instead, you only specify the destination memory type. This is advantageous in many scenarios. And to retrieve the transferred data, you simply invoke the 'data' member function of 'future'. Besides, the dma operation is not marked with '.async', so the wait operation on 'f' is not needed and cannot be performed.

The shorter declaration leaves allocation to the compiler, but the data dependency is unchanged. Identify the destination through the future, wait if the transfer is asynchronous, and check that the next operation expects the resulting shape and storage. The advanced patterns below preserve these requirements while allowing several transfers to be in flight.

Data Movement Statement

Croqtile aims to address multiple levels of data movement. Croqtile adopts this terminology and abstracts data movement as **DMA Operations**.

Operation Type

Croqtile maps these functionalities at the software level as different **DMA Operation Types**. The supported operations include:

- `dma.copy`: Copies flat memory directly.
- `dma.pad`: Pads shaped data while transferring the data.
- `dma.transp`: Transposes shaped data while transferring the data.

The configurations, such as padding and transposing details, are programmed as parameters of the DMA operation. For example:

```
global f32 [32, 16, 9] input;
dma.transp<0, 2, 1> input => shared;  // Result shape [32, 9, 16]
dma.pad<{1, 0, 3}, {0, 1, 2}, {0, 0, 0}, 0.1f> input => shared; // Result
    shape [33, 17, 14]
```

Listing 3.36: Operation Type

DMA configurations appear in angle brackets, and their contents depend on the operation. The supported syntax and constraints also depend on the target platform.

Support for operations other than `dma.copy` varies by platform. A transfer may map to a dedicated copy engine or to a sequence of load instructions issued by cooperating threads.

The data movement is essential for *tileflow programs*. The *DMA statement* has a form for each of those levels, from a single copy to a chained non-blocking transfer.

Full Non-Blocking DMA Mode (Chain Mode in Croqtile)

A transfer that uses the result of another transfer must start after its predecessor completes. The `after` clause records that dependency without making the issuing code wait at that point. The example chains a copy from a staging buffer to an earlier operation:

```
out_store = dma.copy.async l2_out => output.chunkat(m_tile, n_tile) after
    out_store_s;
```

Listing 3.37: Full Non-Blocking DMA Mode (Chain Mode in Croqtile)

In this example:

- `out_store` is the asynchronous DMA operation that transfers data from l2_out to output.chunkat(m_tile, n_tile).
- `out_store_s` is another DMA operation or event that must complete before out_store can proceed.
- The after out_store_s syntax specifies that `out_store` should only start after the completion of `out_store_s`, which ensures that there is no blocking in the main thread.

The issuing code may continue while the dependency is pending. This does not make the destination immediately safe to read, and it does not release the source for reuse. A later consumer still needs the completion of the final transfer. Read a chain from source to destination and check each handoff, including the last one back to ordinary computation.

Worked Example: DMA Matmul

DMA Matmul: Tiles in shared storage

```
__co__ s32 [128, 256] matmul(s32 [128, 256] lhs, s32 [256, 256] rhs) {
  s32 [lhs.span(0), rhs.span(1)] output;
  parallel {px, py} by [8, 16] : block {
    parallel {qx, qy} by [16, 16] : thread
      output.at(px#qx, py#qy) = 0;
    foreach {tile_k} in [16] {
      lhs_load = dma.copy lhs.chunkat(px, tile_k) => shared;
      rhs_load = dma.copy rhs.chunkat(tile_k, py) => shared;
      parallel {qx, qy} by [16, 16] : thread {
        foreach k in [256 / #tile_k]
          output.at(px#qx, py#qy) += lhs_load.data.at(qx, k) * rhs_load.data.
    at(k, qy);
      }
    }
  }
  return output;
}
```

Listing 3.38: DMA Matmul: Tiles in Shared Storage

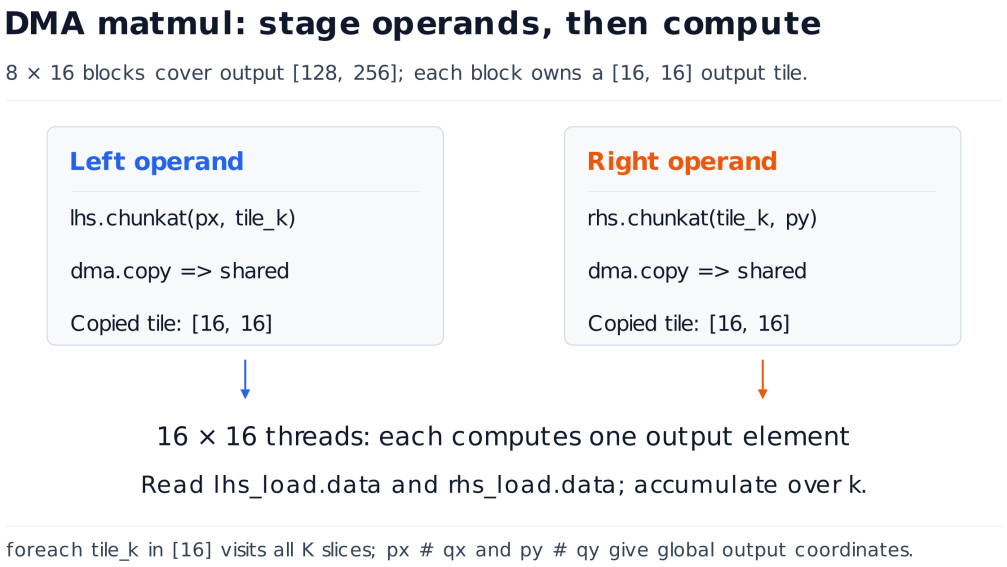


Figure 3.7: Each block stages [16, 16] operand tiles for one K slice; 256 threads accumulate the block's [16, 16] output tile.

The scalar version indexed one element at a time. The tiled version first copies a tile into shared storage, then computes on that tile:

1. The outer `parallel` now uses `: block` with brace-form indices `{px, py}`. An 8×16 grid of blocks covers the output.

2. A `foreach` over `tile_k` walks the K dimension in 16 steps. Each step copies one strip of `lhs` and one strip of `rhs` into `shared` storage with `dma.copy ... => shared`.

3. An inner `parallel {qx, qy} by [16, 16] : thread` creates 256 threads within each block. Each thread owns one output element within the block's tile.

4. The arithmetic reads from `lhs_load.data` and `rhs_load.data` — the shared-memory copies — instead of global `lhs` and `rhs`.

The composed indices work in two layers: `px#qx` composes block index `px` with thread index `qx` to form the global row; `py#qy` does the same for columns.

The block grid `[8, 16]` gives each block 16 rows and 16 columns of output. Its `[16, 16]` threads assign one output element to each thread. Sixteen K tiles, each containing 16 elements along K, cover the full contraction dimension of 256.

The reuse occurs inside the block. A loaded left-hand tile supplies several output columns, and a loaded right-hand tile supplies several output rows. The K loop visits every slice once and adds its contribution to the same output positions. As in the scalar sketch, the output is cleared before the K loop runs. Staging changes the source of the inner-loop reads, while the dot product remains the same.

Multi-Buffering: dma.any, swap, and rotate

Double buffering with `swap`

Give the K-loop **two** logical buffers. While the math drains buffer 0, DMA fills buffer 1 with the next tile. After the math step, **swap** the handles: what was "next" becomes "current," and the freed slot is ready for the following load.

Croqtile spells this with `dma.copy.async` (non-blocking copy), `dma.any` (a placeholder future), `swap` (exchange future handles), and a three-phase loop:

```
__co__ auto matmul(s32 [M, K] lhs, s32 [K, N] rhs) {
  s32 [lhs.span(0), rhs.span(1)] output;
  parallel {px, py} by [8, 16] : block
    parallel {qx, qy} by [16, 16] : thread {
    output.at(px#qx, py#qy) = 0;
    with tile_k in 16 {
      // Prologue: start loading tile 0
      lf0 = dma.copy.async lhs.chunkat(px, tile_k) => shared;
      rf0 = dma.copy.async rhs.chunkat(tile_k, py) => shared;
      // Placeholder futures for buffer 1
      lf1 = dma.any;
      rf1 = dma.any;
      // Steady state: load next tile while computing on current
      foreach tile_k(1:) {
        lf1 = dma.copy.async lhs.chunkat(px, tile_k) => shared;
        rf1 = dma.copy.async rhs.chunkat(tile_k, py) => shared;
        wait lf0, rf0;
        foreach k in [256 / #tile_k]
          output.at(px#qx, py#qy) += lf0.data.at(qx, k) * rf0.data.at(k, qy);
        swap(lf0, lf1);
        swap(rf0, rf1);
      }
      // Epilogue: compute on the last loaded tile
      foreach k in [256 / #tile_k]
        output.at(px#qx, py#qy) += lf0.data.at(qx, k) * rf0.data.at(k, qy);
```

```
    }
  }
  return output;
}
```

Listing 3.39: Double buffering with swap

The three phases

The prologue issues loads for tile 0 into `lf0` and `rf0`. No previous tile exists to compute on yet, so these first transfers establish the state needed by the loop.

In steady state, the loop issues the next loads into `lf1` and `rf1`, waits for the current loads, and computes from `lf0` and `rf0`. The two pairs refer to different buffers, so the next transfer can overlap the current computation. The next loads need not finish first. After the current computation, `swap` makes the next pair current for the following iteration.

The epilogue consumes the final loaded tile after the loop has stopped issuing transfers. It still requires that tile's load to be complete before reading it. The listing above illustrates handle rotation but omits a final wait and output initialization; a runnable accumulation must supply both, just as it does for the steady-state reads.

`swap`: names, not bytes

The operation `swap(lf0, lf1)` exchanges the handles that name the current and next buffers; it does not copy their elements or wait for a transfer. For three buffers, `rotate(f0, f1, f2)` cycles the handles. In either case, follow the handle after the exchange to identify which transfer must complete before a read.

`dma.any`: placeholder futures

A `dma.any` declaration reserves a future name before there is a transfer to assign to it. This lets both handles be declared in the scope containing the loop and its epilogue. The placeholder contains no ready tile: a real transfer must replace it before its data is consumed.

Multi-Buffering Optimization with Croqtile

Overlapping computation with DMA data movement can improve kernel performance. This **multi-buffering** pattern requires multiple buffers so one can be filled while another is used for computation.

In Croqtile, this method requires two key programming primitives: the 'dummy' future **dma.any**, and the **swap/rotate** function. Let's illustrate by an example:

```
__co__ s32 [6, 17, 128] ele_add(s32 [6, 17, 128] lhs,
                                s32 [6, 17, 128] rhs) {
  s32[lhs.span] output;
  parallel p by 6 {
    with index = {x, y} in [17, 4] {
      local s32 [rhs.span / {#p, #x, #y}] l_out;
      foreach x { // first bunch
        lfA = dma.copy.async lhs.chunkat(p, x, y) => local;
        rfA = dma.copy.async rhs.chunkat(p, x, y) => local;
        lfB = dma.any;  // dummy DMA used for multi-buffering
        rfB = dma.any;
        foreach y(1:) {
```

```
13            lfB = dma.copy.async lhs.chunkat(p, x, y) => local;
14            rfB = dma.copy.async rhs.chunkat(p, x, y) => local;
15            wait lfA, rfA;  // wait another bunch
16            call kernel(lfA.data, rfA.data, l_out, |lfB.span|);
17            dma.copy l_out => output.chunkat(p, x, y - 1);
18            swap(lfA, lfB); // exchange futures
19            swap(rfA, rfB);
20          }
21          lf = select(#y % 2, lfB, lfA);
22          rf = select(#y % 2, rfB, rfA);
23          wait lf, rf;  // handle the last bunch
24          call kernel(lf.data, rf.data, l_out, |lfB.span|);
25          dma.copy l_out => output.chunkat(p, x, y(-1));
26        }
27      }
28    }
29    return output;
30  }
```

Listing 3.40: Multi-Buffering Optimization with Croqtile

The addition kernel keeps two local buffer sets for each input. The A handles name the current operands, while the B handles receive the next loads. After computing and storing the current result, the swaps advance those roles. The two inputs travel together through this schedule, so each addition uses matching left and right chunks.

The implementation divides data movements and computations into three stages, including:

- the **prologue**(line 8-11). It preloads the first chunk of data. In this example, buffer A is pre-loaded with data.
- the **body**(line 12-20). It processes the data preloaded in the previous iteration or in prologue, and preloads the data for next iteration.
- and the **epilogue**(line 21-25). It processes the last data chunk only.

The placeholders `lfB` and `rfB` are declared before the inner loop so their names remain available in the epilogue. The loop replaces them with real transfer results before using their data. Declaring those names only inside the loop would put them out of scope when the final chunk is consumed.

The **swap** statements at lines 18-19 exchange futures to ensure the **wait** statement at line 15 functions correctly. Note that only futures with identical DMA operations can be swapped. Additionally, the iteration range of the *foreach* statement at line 12 is adjusted. `foreach y(1:)` means the loop over the *bounded variable* 'y' starts from the second value up to the upper bound. In line 17, the *chunkat* expression is also tuned. `y - 1` results in the current value of 'y' minus 1, setting up the body stage to work properly with the prologue stage. This adjustment is necessary because the iteration count must be reduced by 1, and the pre-load operation should fetch the 'next' chunk while computation is conducted on the 'current' chunk of data. Alternatively, you can use `foreach y(:-1)` and adjust the chunkat expression of the pre-load statements at lines 13-14 to achieve the same effect.

In line 25, the expression `y(-1)` retrieves the 'last value' of 'y' within its bound.

The **select** expressions in line 21-22 handle both odd and even upper-bound cases for the bounded variable 'y'. In either case, it obtains the future of the last chunk.

To use more than two buffers, replace **swap** with **rotate**.

3.7 Contraction and Layout Contracts

Tensor Contraction: the `mma` Syntax

Section 3.4's tiled matmul computes the inner contraction the way a CPU would: a `foreach k` loop where each iteration multiplies two scalars and accumulates into one output element. That works.

The job is **2D tensor contraction** — the operation C += A × B where A, B, and C are small, fixed-shape matrix tiles. It is the inner kernel of every GEMM, every convolution expressed as an im2col matmul, every attention head's QK^T.

The difficulty is mapping matrix elements to the distributed operand fragments. Get the layout wrong — say, load a column-major tile into a row-major fragment — and the result is silently incorrect.

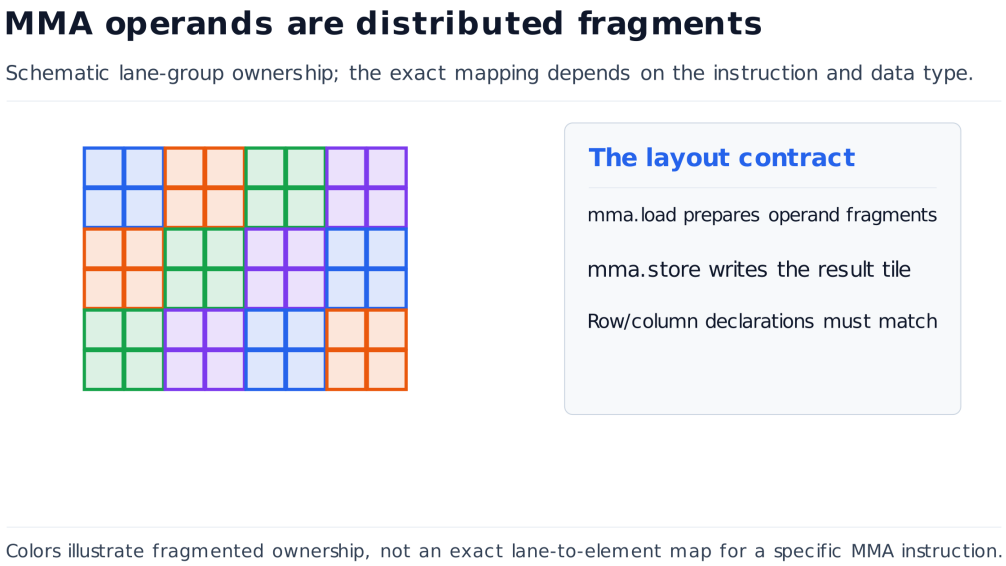


Figure 3.8: Operand fragments are distributed across cooperating lanes. The color pattern is schematic, not an exact instruction-level register map.

Croqtile exposes the contraction through `fill`, `load`, multiply-accumulate, and `store`. The compiler handles the distribution of fragment elements across the chosen cooperation scope. The programmer still chooses compatible tile shapes and states the operand layouts; the abstraction depends on those choices being correct.

Scaling the cooperation scope

The four-step syntax stays the same as the number of cooperating threads changes. The following examples vary the **cooperation scope** while preserving those operations.

Partial Tiles, Guarding, and `.zfill`

Real kernels often end with a partial tile. The important distinction is whether the **source tile shape** and **destination tile shape** are the same.

For example:

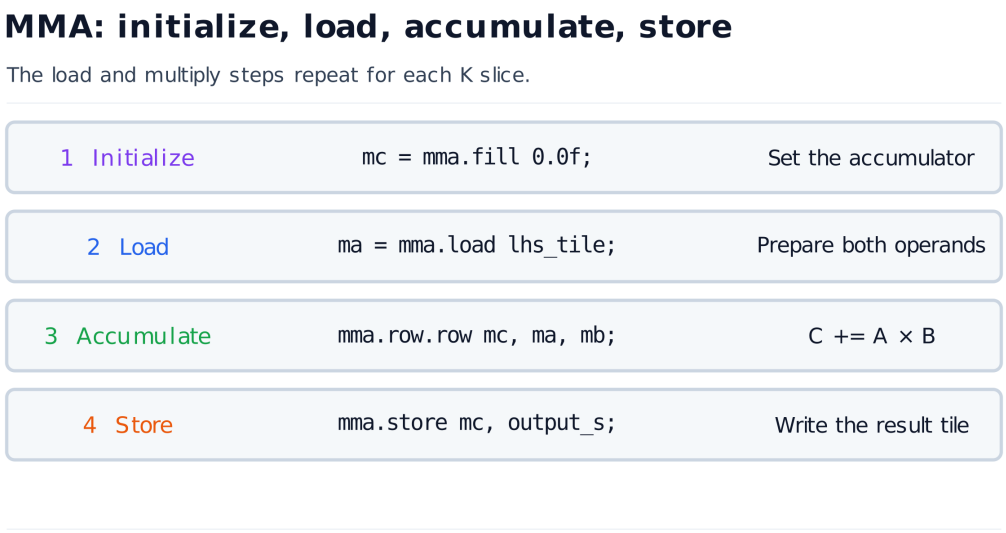


Figure 3.9: Initialize the accumulator, load both operands, accumulate, and store; repeat the load and accumulation steps over K slices.

```
dma.copy.swiz<32>.zfill
  lhs.view(ROWS_M, TILE_K).from(base_m, base_k)
    => lhs_load_s;
```

Listing 3.41: Partial Tiles, Guarding, and .zfill

Here `lhs_load_s` might have shape `[group_M, TILE_K]`, while the source view has shape `[ROWS_M, TILE_K]` and `ROWS_M` may be smaller than `group_M` on the last tile. That is a real shape mismatch, so `.zfill` means: copy the valid source rows and write zero into the remaining rows of the destination tile.

That is the right use for `.zfill`.

What happens when the shapes are the same:

- If the source view and destination tile have the same shape, the compiler already guards runtime tails automatically.
- In that case it generates masking so out-of-bounds elements are not read or written.
- This is enough for cases like tiling a `[100]` tensor into `[64]` tiles: the last tile simply has 36 valid elements.

The `.zfill` form is **not** the general mechanism for "any partial tile." It is specifically for cases where the destination tile is larger than the valid source region and the extra part of the destination must be filled with zeros.

Why `.zfill` matters:

- If the consumer reads only the valid region, the compiler's automatic masking is usually enough.

- If the consumer reads the whole destination tile, especially an MMA fragment, the invalid region must hold a known value.
- `.zfill` makes that extra region zero.

Use `.zfill` when the valid source region is smaller than the destination tile and the consumer will treat the destination as a full dense tile. This is the common case for GEMM-style pipelines with dynamic row counts or padded edge tiles.

If you add `.zfill` when the source and destination shapes are already the same, the compiler can warn that the `.zfill` is redundant. Conversely, when the shapes differ, the compiler can also warn that `.zfill` is needed to avoid using undefined values in the uncovered region.

`.swiz<N>`: Shared-Memory Layout for the Consumer

Typical GEMM staging code looks like this:

```
1 dma.copy.swiz<128> lhs.subspan(WARP_M, TILE_K).at(block_m, iv_k) => lhs_load_s
      ;
2 ma = mma.load.swiz<128> lhs_load_s.chunkat(_, iv_warp);
```

Listing 3.42: .swiz N : Shared-Memory Layout for the Consumer

The load-side and compute-side swizzles need to agree. If you stage a tile with `.swiz<128>`, the consumer should usually load it with the matching swizzle.

This part of the guide only introduces the idea. The MMA chapters go deeper into which swizzle values pair well with which matrix layouts.

Swizzle and layout conflicts

A swizzle changes how elements of a staged tile are arranged in shared storage. The logical matrix shape stays the same, but the reader must use the matching address mapping. Croqtile exposes this choice on the copy operation:

```
1 dma.copy.swiz<128> src => dst;       // software DMA with swizzle
2 dma.copy.swiz<128> src => dst;
```

Listing 3.43: A swizzled copy

The MMA read path must use the same `swiz<N>` so addresses match the staged layout:

```
1 ma = mma.load.swiz<128> lhs_load_s.chunkat(_, iv_warp);
```

Listing 3.44: A swizzled mma load

Swizzle is a property of the copy, not of a particular engine: `dma.copy.swiz<N>` produces the same shared-memory layout regardless of how the data gets there.

Swizzle width. The template argument is the swizzle width in bytes, and the examples use `swiz<128>`. A wider swizzle defeats wider conflict patterns but requires tile extents that line up with that width.

Matching rule. The `<N>` on the copy must match `mma.load.swiz<N>`. If you load with plain `mma.load` from `swiz<128>` data, addresses disagree and you read garbage. The compiler does not enforce the pairing — it is a correctness invariant you maintain. (As introduced in Section 3.7, `mma.load.swiz<N>` is part of the MMA load family.)



Figure 3.10: Eight-bank schematic: different addresses in one bank serialize, while remapping spreads accesses. Copy and MMA load must use matching swizzle.

Beyond dense FP16: what else the four steps express

The examples above use `mma.row.row` on dense FP16 tiles. The same four-step pattern extends to workloads the basic form cannot reach.

A sparse contraction also needs a **metadata operand** `me` that identifies the nonzero elements:

```
1 mma.row.row.sp mc, ma, mb, me;
```

Listing 3.45: A sparse contraction

The `.sp` suffix adds the metadata operand; everything else is the same fill-load-multiply-store rhythm. Any layout combination works: `mma.row.col.sp`, etc. You load `me` from a separate metadata tensor alongside A and B.

For quantized operands with per-tile scales, the contraction also needs the scale values associated with the selected A and B tiles. The `.scale` form expresses this scaling alongside the matrix operation:

```
1 mma.row.row.scale mc, ma, mb, sc_a, sc_b;
```

Listing 3.46: A scaled contraction

Each result element is scaled by `sc_a` and `sc_b` after the contraction — no separate dequant kernel needed. Alternatively, scaling can be a **standalone statement** when the scale source differs from the standard fused path:

```
1 mma.row.row mc, ma, mb;
2 mma.scale mc, sc_a, sc_b;
```

Listing 3.47: Another contraction layout

The standalone `mma.scale` appears in some MoE and mixed-precision kernels.

The `<N>` must agree between `dma.copy.swiz<N>` and `mma.load.swiz<N>` — a mismatch reads garbage. For output, `mma.store.transp mc, dst` writes the accumulator with rows and columns swapped, useful when the next stage expects column-major data.

The table below collects every variant for reference.

Worked Example: One Cooperative Group

One group, one tile: the simplest MMA matmul

The smallest example assigns one output tile to a cooperative group, written `: group`. It accepts left and right operands plus an output buffer supplied by the caller. Follow one output tile through the code: initialize its accumulator, visit every K slice, store the accumulated tile in shared storage, and copy it to its output position.

```
#define TILE_M 16
#define TILE_N 16
#define TILE_K 16
#define WARP_M 16
#define WARP_N 16
#define WARP_K 16
#define MMA_M 16
#define MMA_N 16
#define MMA_K 16
__co__ void matmul_wmma(global f16 [M, K] lhs, global f16 [N, K] rhs, global
    f16 [M, N] output) {
  parallel {block_m, block_n} by [cdiv(M, TILE_M), cdiv(N, TILE_N)] : block {
    shared f16 [TILE_M, TILE_N] output_s;
    parallel {warp_m, warp_n} by [cdiv(TILE_M, MMA_M), cdiv(TILE_N, MMA_N)] :
    group {
      mc = mma.fill 0.0;
      foreach {iv_k} in [cdiv(K, TILE_K)] {
        lhs_load_s = dma.copy lhs.subspan(TILE_M, TILE_K).at(block_m, iv_k) =>
     shared;
        rhs_load_s = dma.copy rhs.subspan(TILE_N, TILE_K).at(block_n, iv_k) =>
     shared;
        foreach iv_warp_k in [cdiv(TILE_K, MMA_K)] {
          ma = mma.load lhs_load_s.chunkat(warp_m, iv_warp_k);
          mb = mma.load rhs_load_s.chunkat(warp_n, iv_warp_k);
          mma.row.row mc, ma, mb;
        }
      }
      mma.store mc, output_s.subspan(MMA_M, MMA_N).at(warp_m, warp_n);
    }
    dma.copy output_s => output.subspan(TILE_M, TILE_N).at(block_m, block_n);
  }
}
```

Listing 3.48: One group, one tile: the simplest MMA matmul

The function returns `void` because it writes into the `output` parameter. The caller supplies that buffer and reads it after the operation completes. Inside the function, `output_s` is the staging buffer used between `mma.store` and the final copy.

The ceiling division `cdiv(M, TILE_M)` counts the tiles along M, including a possible partial tile. The pair `(block_m, block_n)` identifies the output tile assigned to the current block.

The operand selections use the same output coordinates with the current K-slice coordinate.

When the matrix dimensions and the tile dimensions are all 16, the block grid is 1 by 1. Larger matrices produce more output tiles. The `MMA_M`, `MMA_N`, and `MMA_K` names in the listing must be defined consistently with the intended fragment geometry before compiling it.

Section 3.5 explains how `subspan` and `chunkat` select tiles.

Quick 'subspan' syntax reminder

`subspan(h, w)` declares a fixed-size rectangular view shape, and `.at(i, j)` selects which view instance you want from the parent tensor. In other words, `lhs.subspan(TILE_M, TILE_K).at(block_m, iv_k)` means "take the `[TILE_M, TILE_K]` tile at `(block_m, iv_k)`." Use `subspan` when you want an explicit tile shape; use `chunkat` when you want to slice by equal chunks of an existing dimension.

The accumulator spans the entire K loop. Initializing it inside that loop would discard earlier contributions, while storing inside the loop would expose only a partial result. Keeping initialization before the loop and the store after it makes the output tile's lifetime visible.

MMA Syntax Summary

Syntax	Meaning
`mc = mma.fill 0.0`	Initialize accumulator tile to zero
`mma.fill.f16 0.0f` / `mma.fill.f32 0.0f`	Accumulator with explicit precision
`mma.load.swiz<N> src`	Load with swizzle mode (Section 3.7)
`mma.row.row mc, ma, mb`	C += A × B (both row-major)
`mma.row.col mc, ma, mb`	C += A × B (A row-major, B col-major)
`mma.row.row.sp mc, ma, mb, me`	Sparse MMA with metadata operand
`mma.row.row.scale mc, ma, mb, sc_a, sc_b`	Fused MMA + per-tile dequantization
`mma.scale mc, sc_a, sc_b`	Standalone post-MMA scaling
`mma.store.transp mc, dst`	Write accumulator transposed
`cdiv(a, b)`	Ceiling division
`__co__ void fn(...)`	Kernel that writes results in-place

Syntax at a glance Read the modifiers together with the data path. A swizzled load must match the layout produced by the copy; a sparse operation needs metadata for the same operand tile; scaling values must belong to the tile being accumulated. The short statement names do not remove these relationships.

MMA Design Summary

In short The contraction is fast, but loads and compute still take turns — Section 3.8 introduces **role specialization** so different thread groups can play different roles: one group loading data while another computes, overlapping memory and math in time.

3.8 Coordination, Pipelines, and Persistent Scheduling

Topic	Takeaway
2D tensor contraction	C += A × B on tile-shaped operands — the universal inner kernel
Four-step abstraction	**fill** → **load** → **multiply** → **store**; the compiler handles fragment layouts for each target
Scaling cooperation	Syntax stays invariant
Layout contract	`mma.row.row`, `mma.row.col`, etc.
Sparse and quantized	`.sp` adds a metadata operand; `.scale` fuses per-tile dequantization
Swizzle and pipeline	`mma.load.swiz<N>` matches swizzled shared layouts

Control Flow: Divergence, Concurrency, and Coordination

Every programming language that targets a parallel machine must answer one question: **what happens when different threads need to do different things?**

On a CPU, each core has its own program counter. When threads share an instruction stream, however, divergent paths execute in sequence while inactive threads are masked.

All threads evaluate the condition, and those for which it is false are masked. This **predicated execution** is simple and widely used, but divergent paths can be expensive.

A tiled matrix multiplication also has work with different roles: one group can load the next operands while another computes with the current operands. The groups need to progress independently between well-defined handoffs. A runtime condition describes which branch an instance takes; a pipeline also needs a way to express those separate roles and their dependencies.

Croqtile introduces two additional control-flow primitives to fill this gap:

- `inthreads.async` — **structured concurrent regions**: compile-time partitioning of threads into groups that run **different programs simultaneously**. The compiler generates separate instruction streams;
- `shared event` / `wait` / `trigger` — **inter-region signaling**: lightweight synchronization tokens that let concurrent regions communicate safely.

Structured concurrent regions with `inthreads.async`

The form `inthreads.async` assigns different thread groups to separately progressing regions. Its predicate identifies the group responsible for each region. This is useful for a producer and consumer whose roles are fixed even though their progress depends on which buffers are ready.

```
parallel p1 by 2 : thread {
  inthreads.async (p1 == 0) {
  }
  inthreads.async (p1 == 1) {
  }
}
```

Listing 3.49: Structured concurrent regions with inthreads.async

The distinction from `if` is structural, not just performance:

The regions are lexically scoped, so the compiler can identify their participants from the program structure. The code does not create an unbounded number of tasks at runtime. Each region operates within the declared parallel scope.

	`if` (predicated execution)	`inthreads.async` (structured concurrency)
Resolution	Runtime — every thread evaluates the condition	Compile time — thread assignment is fixed
Instruction streams	One program; divergent threads masked	Separate programs per region
PL analogy	`if`/`else` in any language	`async`/`spawn` in structured concurrency (Trio, Go goroutines, Cilk)

Without `.async`, the region rejoins the surrounding execution before later code proceeds. Adding `.async` allows other regions to advance independently. Any data handoff between them then needs an explicit synchronization protocol.

The figure below shows the effect. — The producer's DMA and the consumer's MMA overlap in time:

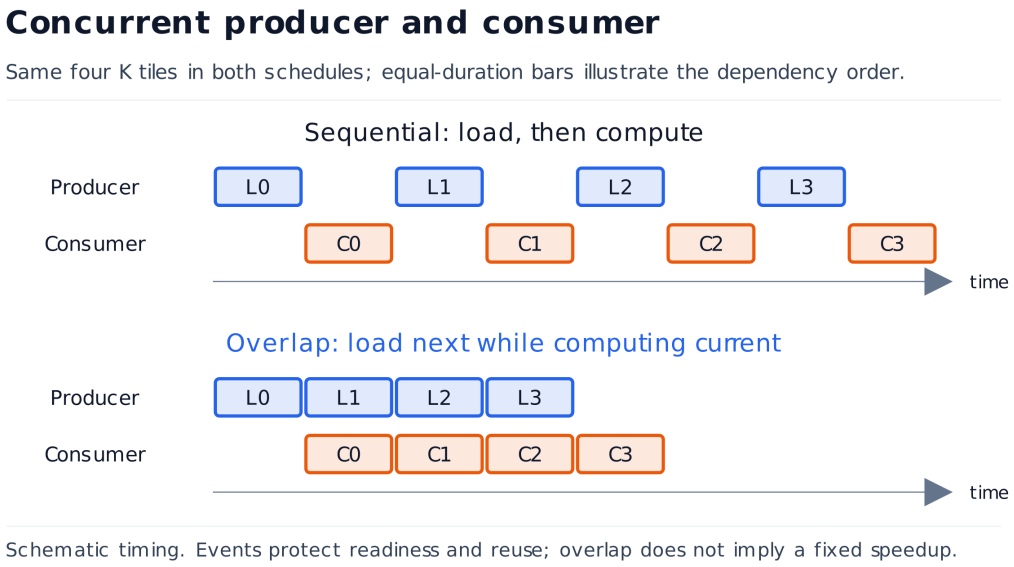


Figure 3.11: Four load/compute pairs are shown in both schedules. Concurrent producer and consumer regions overlap work while preserving dependencies.

Inter-region signaling with events

When `inthreads.async` creates concurrent regions, those regions need a way to communicate. Croqtile provides **events**.

```
shared event full;
shared event empty;
```

Listing 3.50: Inter-region signaling with events

Events have two operations:

- **`trigger name`** — signal that a condition is met (e.g., "data is ready")

- `wait name` — block until the corresponding `trigger` fires

The producer calls `trigger full` after writing a tile to signal "data ready." The consumer calls `wait full` before reading, blocking until the signal arrives. Symmetrically, the consumer triggers `empty` after finishing its read (the buffer can be reused), and the producer waits on `empty` before writing the next tile.

This is a **credit-based bounded buffer** protocol — `full` is the "data available" credit; `empty` is the "buffer free" credit.

Synchronization in Practice: Pipelines, Buffers, and Events

A buffer pipeline has two ordering requirements. A consumer must wait until the producer finishes filling its slot, and the producer must wait until the consumer finishes using that slot before overwriting it. A fast load does not remove the second requirement, and a fast consumer does not remove the first.

The double-buffering pattern in Section 3.6.2 uses one group to issue transfers, wait for current inputs, and rotate handles. A producer-consumer pipeline assigns those roles to separate asynchronous regions. Events then carry the readiness and reuse signals that the single-group version expressed through its own instruction order.

Both patterns solve the same underlying problem: **you cannot read a buffer while someone is writing to it**. They differ in how they structure the solution.

Why you cannot "just overlap" without events

The `swap` pattern uses one thread group to control both loading and computation in a known order. Separate producer and consumer groups need a signaling mechanism to coordinate their independent schedules.

The picture below contrasts a strict load-then-compute staircase with double-buffered overlap: same logical work, less idle time.

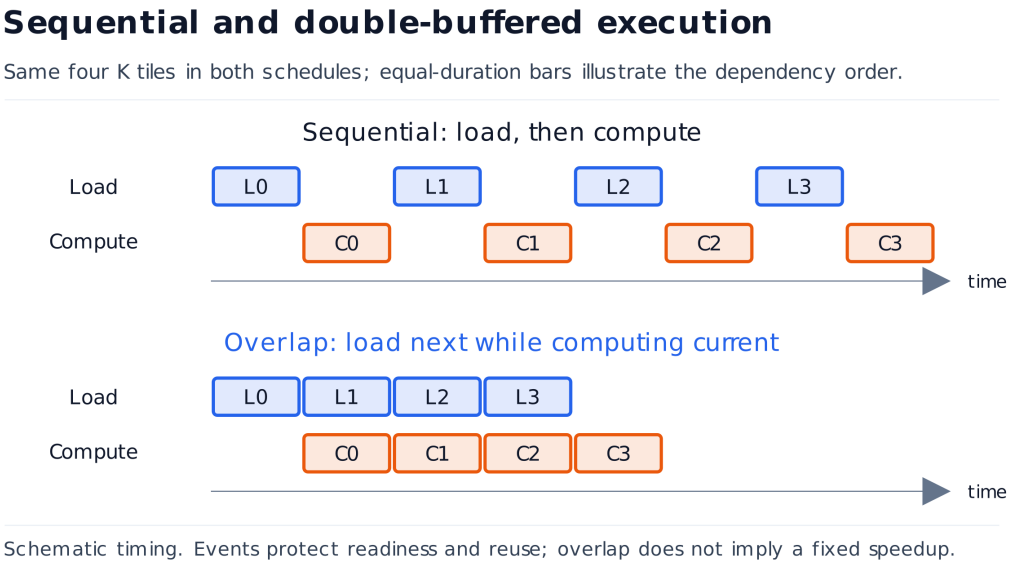


Figure 3.12: The same four K tiles run sequentially or with two-buffer overlap. Timings are schematic and include pipeline startup and drain.

Make it Asynchronous

Like `parallel-by`, `inthreads` can execute asynchronously. This lets instances running a divergent region proceed independently of those outside it, expressing MPMD behavior within an SPMD program:

```
__co__  void foo() {
  parallel p by 6 {
    inthreads.async (p <= 2) {
      // divergent code for thread 0, 1, 2 (thread-mask: 000111)
    }
    inthreads.async (p > 2) {
      // divergent code for thread 3, 4, 5 (thread-mask: 111000)
    }
    sync.shared; // sync point
                 // all 'inthreads' paths must get executed
  }
}
```

Listing 3.51: Make it Asynchronous

The example divides the parallel instances into two groups without synchronizing after each `inthreads` block, allowing the groups to execute concurrently. The subsequent `sync.shared` establishes a synchronization point that both paths must reach before execution continues.

Only the outer `inthreads` can be declared as asynchronous. Programmers will get a compile-time error when using `inthreads.async` for any inner `inthreads` block.

Event Instances

In Croqtile, events are associated with the storage specifier, either `global`, `shared`, and `local`. Such storage specifier specifies the event instances.

```
__co__ void foo() {
  global event ge;     // single instance in the tileflow program
  parallel p by 2 {    // shared level
    shared event se;   // two instances, 1 for each shared instance
    parallel q by 6 {  // local level
      local event le;  // 12 instances in total, 6 for each shared instance
    }
  }
}
```

Listing 3.52: Event Instances

The comments identify the event instances. As with spanned-data declarations, the storage specifier determines the level at which an event is stored and shared. An implementation may represent events with Boolean state at that storage level; the number of instances follows the corresponding parallel scope.

Deadlocks

Events are lock-free by default to support efficient synchronization between asynchronous blocks. Multiple instances updating the same event require care, particularly when using `wait`.

For example,

```
parallel p by 1 {
  shared event e;
  trigger e;   // one thread set e;
  parallel q by 2 {
    wait e;    // two threads can reset e;
  }
}
```

Listing 3.53: A single-instance deadlock

Both threads wait on the same event `e`. Because `wait` resets the event, `thread-0` may reset it before `thread-1` observes the trigger. In that case, `thread-1` remains blocked and the program deadlocks.

```
qc = 2;
parallel p by 1 {
  shared event e[qc];
  trigger e;   // one thread set e[0] and e[1];
  parallel q by qc {
    wait e[q]; // no dead lock
  }
}
```

Listing 3.54: A credit-count deadlock

In this code, you utilize an event array to guarantee each event is waited and reset by a single thread.

Canonical Pattern: One Producer and One Consumer

The canonical pattern: 1 producer + 1 consumer

The most common use of `inthreads.async` is the **1P1C** (one producer, one consumer) split for matmul:

```
parallel p1 by 2 : thread {
  inthreads.async (p1 == 0) {
  }
  inthreads.async (p1 == 1) {
  }
}
```

Listing 3.55: The canonical pattern: 1 producer 1 consumer

The declaration `parallel p1 by 2 : thread` creates two cooperating groups in the same surrounding block. The value of `p1` selects a role; it does not identify a tile along K.

The region for `p1 == 0` is the producer. It owns the writes that fill the shared staging buffers. The other group can proceed with computation while those writes target a different, available slot.

The region for `p1 == 1` is the consumer. It reads completed tiles and updates its accumulator. The two regions share staging storage, so the event protocol must identify when a slot changes from writable to readable and back again.

Because both regions share a buffer, their reads and writes must be coordinated. Without synchronization, the consumer could read before the producer finishes writing. Events express the required ordering.

Worked Example: A 1P1C Kernel Skeleton

A 1P1C matmul skeleton

Event-based synchronization is left out of this skeleton; Section 3.8 adds the full pipeline protocol. Focus on the program structure:

```
__co__ auto matmul(global f16 [M, K] lhs, global f16 [N, K] rhs) {
  f16 [M, N] output;
  parallel {block_m, block_n} by [cdiv(M, WARP_M), cdiv(N, WARP_N)] : block {
    shared f16 [WARP_M, TILE_K] lhs_load_s;
    shared f16 [WARP_N, TILE_K] rhs_load_s;
    shared f16 [WARP_M, WARP_N] output_s;
    parallel p1 by 2 : thread {
      inthreads.async (p1 == 0) {
        foreach {iv_k} in [cdiv(K, TILE_K)] {
          dma.copy lhs.subspan(WARP_M, TILE_K).at(block_m, iv_k) => lhs_load_s
    ;
          dma.copy rhs.chunkat(block_n, iv_k) => rhs_load_s;
        }
      }
      inthreads.async (p1 == 1) {
        mc = mma.fill.f16 0.0f;
        foreach {iv_k} in [cdiv(K, TILE_K)] {
          foreach {iv_warp} in [cdiv(TILE_K, WARP_K)] {
            ma = mma.load lhs_load_s.chunkat(_, iv_warp);
            mb = mma.load rhs_load_s.chunkat(_, iv_warp);
            mma.row.row mc, ma, mb;
          }
        }
        mma.store mc, output_s;
        dma.copy output_s => output.subspan(WARP_M, WARP_N).at(block_m,
    block_n);
      }
    }
  }
  return output;
}
```

Listing 3.56: A 1P1C matmul skeleton

The producer loop visits `cdiv(K, TILE_K)` input slices. For each slice it fills the left and right staging buffers. Both operands must be ready before the consumer starts the corresponding contraction.

The consumer initializes one accumulator for its output tile, then loads and multiplies the operand fragments for each K slice. It writes the output after the last contribution. Because the two loops run independently, matching loop counts alone does not guarantee that the consumer sees the intended slice.

The skeleton omits the handoff between these loops. Before use as a working pipeline, add a readiness signal after loading and a reuse signal after consumption. Section 3.8.5 supplies both with per-slot events.

Persistent Scheduling

In the earlier kernels, the grid grew with the problem: roughly one block per output tile. For large matrices that means large launch counts, and the last wave of blocks often leaves SMs partially idle — **tail underutilization**.

A **persistent kernel** fixes the launch size (often near the SM count) and lets each block iterate over multiple tiles:

```
__co__ auto matmul(global f16 [M, K] lhs, global f16 [N, K] rhs) {
  f16 [M, N] output;
  int total_tiles = cdiv(M, WARP_M) * cdiv(N, WARP_N);
  parallel block_id by NUM_GROUPS : block {
    shared f16 [WARP_M, TILE_K] lhs_load_s;
    shared f16 [WARP_N, TILE_K] rhs_load_s;
    shared f16 [WARP_M, WARP_N] output_s;
    foreach {tile_iter} in [cdiv(total_tiles, NUM_GROUPS)] {
      tile_id = tile_iter # block_id;
      if (tile_id < total_tiles) {
        block_m = tile_id / cdiv(N, WARP_N);
        block_n = tile_id % cdiv(N, WARP_N);
        mc = mma.fill.f16 0.0f;
        foreach {iv_k} in [cdiv(K, TILE_K)] {
          dma.copy lhs.subspan(WARP_M, TILE_K).at(block_m, iv_k) => lhs_load_s
    ;
          dma.copy rhs.subspan(WARP_N, TILE_K).at(block_n, iv_k) => rhs_load_s
    ;
          foreach {iv_warp} in [cdiv(TILE_K, WARP_K)] {
            parallel p by 1 : thread {
              ma = mma.load lhs_load_s.chunkat(_, iv_warp);
              mb = mma.load rhs_load_s.chunkat(_, iv_warp);
              mma.row.row mc, ma, mb;
            }
          }
        }
        mma.store mc, output_s;
        dma.copy output_s => output.subspan(WARP_M, WARP_N).at(block_m,
    block_n);
      }
    }
  }
  return output;
}
```

Listing 3.57: Persistent scheduling and the if guard

The declaration fixes the number of workers at `NUM_GROUPS`. Work size changes the number of loop iterations rather than the number of launched blocks.

The expression `tile_iter # block_id` has value `tile_iter * NUM_GROUPS + block_id`. It assigns the next consecutive set of tiles to the workers on each iteration.

The condition `tile_id < total_tiles` excludes indices beyond the final valid tile. All work for that tile belongs inside the guard, including any producer-consumer protocol whose participants must agree whether the tile exists.

For four workers and ten output tiles, the composed tile indices are 0–3 on the first iteration, 4–7 on the second, and 8–11 on the third. The guard accepts only 8 and 9 in the last iteration. This covers each valid tile once, but it does not guarantee that every worker

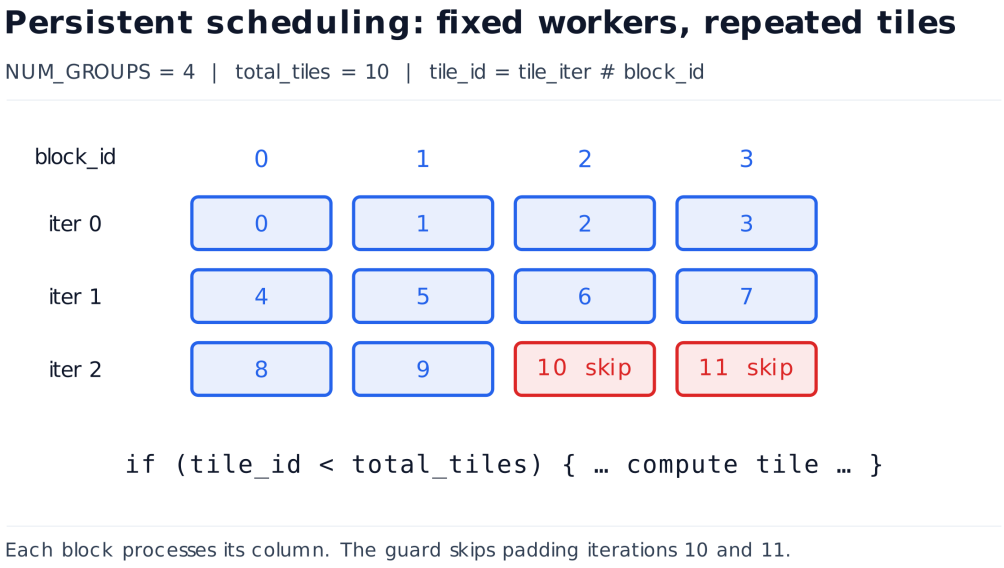


Figure 3.13: Four fixed blocks process ten tiles in three iterations; the final two padded tile IDs are skipped by the if guard.

stays busy until the end; tile cost and the remaining work still determine utilization.

Data-dependent vs persistent grids

Aspect	One block per tile	Persistent (`NUM_GROUPS` blocks)
Grid size	Grows with problem	Fixed
Tail utilization	Last wave may leave SMs idle	Workers can process further tiles; a tail may remain
Extra constructs	Minimal	`total_tiles`, `tile_iter # block_id`, `if`
Complexity	Lower	Higher

Host-Level Concurrency and Stream Binding

`parallel.async`: host-level concurrency

Everything above runs inside a kernel. Sometimes you need concurrency at the **host level**: launch a grid without blocking the CPU.

```
1 parallel.async {px, py} by [grid_m, grid_n] : block {
2   // kernel body
3 }
```

Listing 3.58: parallel.async: host-level concurrency

An asynchronous launch returns control to the host after submitting the kernel. Completion happens later. Stream binding selects which stream receives the work; without an explicit binding, the default stream is used.

This is **host orchestration**, orthogonal to in-kernel control flow. It does not replace `inthreads.async` for thread partitioning or `if` for runtime predicates — it decides *when* and *where* a grid runs relative to other grids.

Asynchronous program launch and Stream Binding

By default, the host waits for a `parallel-by` kernel to complete before continuing. To launch kernels asynchronously, use `parallel.async`.

```
__co__ void foo(stream s, f32[M] a) {
  parallel.async<s> p by M : block {
    a.at(p) = 1.0f;
  }
  // host continues immediately (no synchronization)
}
```

Listing 3.59: Asynchronous program launch and Stream Binding

Section 3.8 covers this in detail.

Synchronization and Asynchronous `parallel-by`

Similar to `inthreads`, `parallel-by` can also be made asynchronous by appending `.async`:

```
parallel.async p by 6 : block {
}
```

Listing 3.60: An asynchronous launch

Without `.async`, the host waits for the kernel to finish before continuing:

```
parallel p by 6 : block {
}
// host continues only after the kernel finishes
```

Listing 3.61: The synchronous form

With `.async`, the host code continues immediately after launching the kernel. This is essential for overlapping computation with data transfers or launching independent kernels concurrently.

In Croqtile, you can bind a `parallel-by` block to a specific stream using angle-bracket syntax:

```
__co__ void foo(stream s0, stream s1, f32[M] a, f32[N] b) {
  parallel.async<s0> p by M : block {
    a.at(p) = 1.0f;
  }
  parallel.async<s1> q by N : block {
    b.at(q) = 2.0f;
  }
}
```

Listing 3.62: Stream Binding

In this example:

- `stream s0, stream s1`: Stream parameters passed from the host.
- `parallel.async<s0>`: Launches the kernel asynchronously on stream `s0`.
- `parallel.async<s1>`: Launches a second kernel on stream `s1`.

Stream Binding without `.async`

Stream binding also works with synchronous `parallel-by`. In this case, the host launches the kernel on the specified stream and then waits for that stream to complete:

```
__co__ void bar(stream _s, f32[M] a) {
  parallel<_s> p by M : block {
    a.at(p) = 1.0f;
  }
  // host synchronizes on stream _s before continuing
}
```

Listing 3.63: Stream Binding without .async

Rules

- Stream binding is only allowed on **block-level** parallel (`: block`).
- The identifier inside `<...>` must be a `stream`-typed variable.
- Without stream binding, synchronous `parallel-by` uses device-wide synchronization.

Without stream binding, asynchronous `parallel-by` uses the default stream.

A stream selects an ordering context for launches. It does not make two kernels independent if the second reads data written by the first. When using separate streams, keep that dependency explicit and retain the buffers until their queued users finish. Host-side return from an asynchronous launch only confirms that execution was submitted.

Worked Example: A Full Event Pipeline

The full 1P1C event pipeline

This kernel combines `inthreads.async` (from Section 3.8) with event arrays to build a complete multi-stage pipeline. The producer and consumer run as separate concurrent programs, coordinated entirely through `wait` / `trigger`:

```
#define TILE_M 128
#define TILE_N 128
#define TILE_K 128
#define WARP_M 64
#define WARP_N 64
#define WARP_K 16
#define STAGES 2
__co__ auto matmul(global f16 [M, K] lhs, global f16 [N, K] rhs) {
  f16 [M, N] output;
  parallel {block_m, block_n} by [cdiv(M, WARP_M), cdiv(N, WARP_N)] : block {
    shared event full[STAGES], empty[STAGES];
    shared f16 [WARP_M, TILE_K] lhs_load_s[STAGES];
    shared f16 [WARP_N, TILE_K] rhs_load_s[STAGES];
```

```
    shared f16 [WARP_M, WARP_N] output_s;
    parallel p1 by 2 : thread {
      inthreads.async (p1 == 0) {
        foreach {iv_k} in [cdiv(K, TILE_K)] {
          stage = iv_k % STAGES;
          wait empty[stage];
          dma.copy lhs.subspan(WARP_M, TILE_K).at(block_m, iv_k)
            => lhs_load_s[stage];
          dma.copy rhs.chunkat(block_n, iv_k)
            => rhs_load_s[stage];
          trigger full[stage];
        }
      }
      inthreads.async (p1 == 1) {
        mc = mma.fill.f16 0.0f;
        foreach {s} in [STAGES]
          trigger empty[s];
        foreach {iv_k} in [cdiv(K, TILE_K)] {
          stage = iv_k % STAGES;
          wait full[stage];
          foreach {iv_warp} in [cdiv(TILE_K, WARP_K)] {
            ma = mma.load lhs_load_s[stage].chunkat(_, iv_warp);
            mb = mma.load rhs_load_s[stage].chunkat(_, iv_warp);
            mma.row.row mc, ma, mb;
          }
          trigger empty[stage];
        }
        mma.store mc, output_s;
        dma.copy output_s => output.subspan(WARP_M, WARP_N).at(block_m,
    block_n);
      }
    }
  }
  return output;
}
```

Listing 3.64: The full 1P1C event pipeline

Walking through the kernel

The stage index is `iv_k % STAGES`. With two slots, tiles 0 and 2 use slot 0, while tiles 1 and 3 use slot 1. These are different logical tiles sharing physical storage at different times; the event pair belongs to the slot, not to one permanent tile.

The producer waits on `empty[stage]` before writing either operand into that slot. Both copies in this example are synchronous, so triggering `full[stage]` after them means both operands are ready. Replacing the copies with asynchronous operations would require completing those operations before sending the same signal.

The consumer first triggers every `empty` event. All events begin unset, so this bootstrap grants the producer permission to fill each slot once. No `full` event is triggered here because no input has been loaded yet.

Consumer path. Each `iv_k`: `wait full[stage]` blocks until the producer has filled that slot, then MMA over the tile and `trigger empty[stage]` to release the slot for reuse.

The consumer signals `empty[stage]` only after the tile's matrix operations have completed; releasing a stage early lets the producer overwrite operands the MMA is still reading.

Credit flow for one stage

The diagram matches the code: bootstrap grants empty credits; the producer waits on `empty`, fills, signals `full`; the consumer waits on `full`, computes, signals `empty`. When `iv_k` wraps modulo `STAGES`, the same physical stage re-enters the cycle.

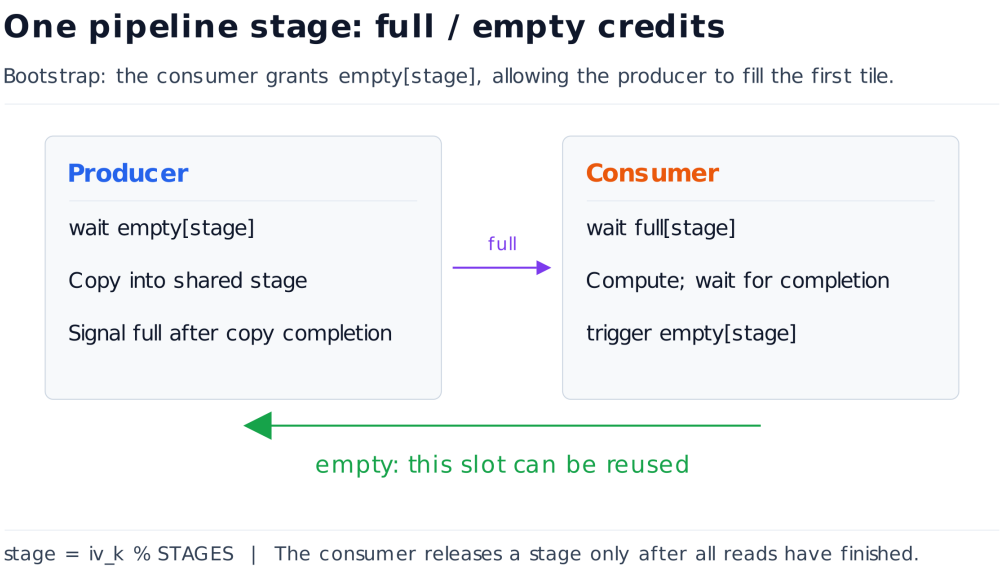


Figure 3.14: An empty credit permits filling a stage; a full credit permits reading it. The consumer releases the stage only after computation completes.

Coordination Pattern Summary

Concept	Primitive	When to use
Predicated execution	`if`	Data-dependent decisions (bounds, conditions)
Structured concurrency	`inthreads.async`	Compile-time thread partitioning (producer/consumer, heterogeneous roles)
Inter-region signaling	`shared event` / `wait` / `trigger`	Coordination between concurrent regions
Host concurrency	`parallel.async` / `stream s`	Multi-kernel overlap, non-blocking launch
Persistent scheduling	`if` + `foreach` + `#`	Fixed grid size, tile striping with padding guard

In short The completed pipeline adds two dependencies to the earlier skeleton: input readiness before computation, and completion of operand use before slot reuse. When adapting it, trace one slot through a full cycle and then through its next reuse. This exposes missing signals and mismatched iteration counts more clearly than looking at either loop alone.

Pattern	Primitives used	When to use
`swap` double buffering	`dma.any`, `swap`	Single thread group interleaving load and compute
Event pipeline	`inthreads.async`, events	Separate producer and consumer groups

In short Handle rotation is enough when one group controls both the transfer schedule and the computation. Separate producer and consumer regions need events because neither can infer the other's progress from its own instruction order. More buffering can give them room to overlap, but it cannot repair a missing dependency.

The same synchronization patterns can be combined with the richer data-movement primitives described in this guide.

3.9 C++ Interoperability and the Preprocessor

C++ Interop: Device Functions, Inline Code, and the Preprocessor

A kernel may need a reduction, a sorting routine, or an instruction that Croqtile does not express directly. Existing device C++ can supply that computation while Croqtile continues to arrange its inputs, outputs, and parallel scope. The interface needs to preserve the assumptions on both sides of the call.

Croqtile's interoperability story has three parts:

1. **`__device__` functions** — The compiler passes them through unchanged.
2. **`__cpp__`**
3. **The preprocessor** — `#define`, `#if`, `#ifdef`, `#error` for compile-time configuration, the same role `#define` plays in C/C++ codebases.

The preceding sections introduced tiles, parallelism, MMA, pipelines, and events. C++ interoperability extends these abstractions when a kernel needs additional device functions or target-specific code.

`__device__` vs ordinary C++ inline

Both `__device__` functions and ordinary C++ functions are passed through unchanged, but they land in different compilation paths — and the distinction matters.

A `__device__` function compiles to device code. The `__co__` body invokes it with `call`, and the generated `__global__` kernel calls it directly on the device.

An ordinary C++ function (no `__device__` or `__co__` annotation) compiles to **CPU code**. It runs on the host — setting up buffers, launching kernels, printing results. You cannot call a host function from inside a `__co__` body.

The boundary is strict: `__device__` lives on the device side of the `c++` split, host C++ lives on the host side. The `__co__` function is the bridge — the compiler transforms it into a `__global__` kernel (device side) and generates a host-side wrapper that launches it.

Difference with C++ pre-processing

Croqtile preprocessing runs much earlier than c++ preprocessing. The workflow of croqtile compilation is as shown below:

```
chore-preprocessing -> croqtile compilation -> c/c++ preprocessing -> c/c++
    compilation
```

Listing 3.65: Differences from C++ Preprocessing

Croqtile preprocessing coordinates macros across the host and device portions of a program. Its implementation substitutes macros and handles conditional compilation within Tileflow functions, while leaving other preprocessing to the C++ preprocessor. This division can produce differences from ordinary C++ preprocessing.

Pre-defined Macros

Croqtile also imports the target's built-in macros to match native compilation. These macros can be used in both Tileflow functions and host code.

Trigger C++ Template Instantiation

It is possible for Croqtile code to make function call to a template function. For example:

```
template<int M, int N, int K>
__device__ void matmul_kernel(int *lhs, int* rhs, int* output) {}
__co__ auto matmul(s32 [96, 72] lhs, s32 [72, 24] rhs) {
  s32 [lhs.span(0), rhs.span(1)] output;
  parallel {p, q} by [6, 6] : block {
    shared s32 [output.span / {#p, #q}] buffer;   // shape: [16, 4]
    lhs_load = dma.copy lhs.chunkat(p, _) => shared;
    rhs_load = dma.copy rhs.chunkat(_, q) => shared;
    call matmul_kernel<buffer.span(0), buffer.span(1), 72>(lhs_load.data,
    rhs_load.data, buffer);
    dma.copy buffer => output.chunkat(p, q);
  }
  return output;
}
```

Listing 3.66: A templated device helper

The call supplies three constant template arguments to `matmul_kernel`. The target compiler instantiates the corresponding specialization, just as it would for an ordinary C++ template call.

Template arguments must be evaluable as constants during Croqtile compilation. An input-dependent value cannot select a C++ specialization at runtime:

```
__co__ void foo(int M) {
  parallel p by 1 {
    call bar<M>();  // error: 'M' is a runtime value
  }
}
```

Listing 3.67: A call that instantiates it

The Croqtile compiler attempts to evaluate template arguments at compile time. If it cannot determine a required constant, compilation stops with an error.

More Questions

Like C++ functions, Croqtile functions can be defined throughout a source file. When linking a module that contains one, its **function signature** determines how it is called.

A Croqtile function follows the C++ calling convention and uses C++ name mangling. It does not support `extern "C"`, because its spanned-data parameters use C++ template-based types.

Some features are not supported yet:

- Croqtile function declaration without definition. This allows calling Croqtile function from another module.
- Croqtile function with `inline`. This allows the Croqtile function to be used in headers.

Factor Target Specific Details

Targets such as *factor* require C linkage for device functions, expressed with `extern "C"`. Function templates cannot have C linkage, so Croqtile provides wrappers to support them:

```
__cok__ {
extern "C" void bar(int v) {};  // v is a compile-time constant
}
__co__ void foo() {
  parallel p by 1 { call bar(3); }
}
```

Listing 3.68: A cok block around device code

Programmers can turn `bar` into a function template, like following:

```
__cok__ {
template <int v> void bar() {};
// compiler generate a explicit function template specialization:
// template<> void bar<3>() {};
}
__co__ void foo() {
  parallel p by 1 { call bar<3>(); }
}
```

Listing 3.69: The block after factoring target-specific details

Croqtile can generate a wrapper for the `bar<3>` specialization. The function template itself has C++ linkage; the wrapper provides the target-facing entry point.

As the implementation differs from other targets, you may utilize `-kt` to enable such functionality for *croqtile-factor* compilation.

Worked Example: Calling a Device Helper

`__device__` functions: target alongside Croqtile

A custom reduction, sorting network, or specialized math operation can be implemented as a device helper.

A `__co__` function can call a `__device__` function using the `call` keyword:

```
template <int K>
```

```
__device__ void warp_topk(f32* vals, s32* idxs);
template <typename T>
__device__ __forceinline__ T SHFL_XOR(T var, int lane_mask, int width) {
  return __shfl_xor_sync(uint32_t(-1), var, lane_mask, width);
}
__co__ void moe_topk(f32 [N_TOKENS, N_EXPERTS] scores,
                     s32 [N_TOKENS, K]& topk_ids,
                     f32 [N_TOKENS, K]& topk_scores,
                     int N_BLOCK) {
  parallel n by N_BLOCK : block {
    foreach m in [ |scores.span| / N_THREAD / N_BLOCK ] {
      shared_scores = dma.copy scores.chunkat(n#m, _) => shared;
      parallel gid by [N_THREAD / 32] : group
        parallel tid by 32 : thread {
          flat_scores = shared_scores.span_as([|shared_scores.span|]);
          score = flat_scores.at(gid # tid);
          local s32 [K] frag_idx{-1};
          local f32 [K] frag_val{-1.0f};
          frag_idx.at(0) = tid;
          frag_val.at(0) = score;
          call warp_topk<8>(frag_val, frag_idx);
          inthreads (tid == 0)
            foreach k in K {
              topk_ids.at(n#m, gid#k) = frag_idx.at(k);
              topk_scores.at(n#m, gid#k) = frag_val.at(k);
            }
        }
    }
  }
}
```

Listing 3.70: device functions: target alongside Croqtile

The helper receives raw pointers to element storage. Its interface must therefore state any sizes or layout assumptions that it needs; the pointer alone does not preserve the Croqtile shape.

The `SHFL_XOR` helper is ordinary device code passed to the target compiler. Its use inside the computation belongs to the helper's implementation; Croqtile does not translate that body into tile operations.

The `call group_topk<8>(...)` statement supplies the helper's compile-time configuration and ordinary arguments. Croqtile converts spanned-data arguments to element pointers, and the target compiler validates the resulting C++ call.

The `inthreads (tid == 0)` region selects one thread for the enclosed work and rejoins afterwards. It has no `.async` modifier, so it does not introduce a separately progressing producer or consumer.

In this pattern, Croqtile handles parallelism, tiling, and data movement, while the device helper performs the specialized computation. Production kernels use it for MoE top-K, custom reductions, and specialized math.

Interoperability Summary

In short Section 3.11 turns to the other side of the workflow: what to do when the kernel compiles but the output is wrong — debugging, verbose modes and systematic narrowing. The two worked programs in Section 3.10 put the interoperability mechanisms into context.

Syntax	What it does	Runs on
`#define NAME value`	Object-like macro; shared across `__co__` and host regions	Preprocessor
`#if` / `#ifdef` / `#error`	Conditional compilation and compile-time assertions	Preprocessor
`-DNAME=value`	Define macros from the command line for tile sweeps	Preprocessor
`call helper(...)`	Invoke a device helper	Device
`__cpp__("...")`	Insert target C++ at the current scope	Target-dependent

3.10 Tileflow Patterns and Design Rationales

Why the restricted interface works: expressiveness vs performance

Writing every load and address calculation separately gives a programmer many choices, but it also leaves the compiler to recover which operations form one tile transfer. Croqtile asks the program to describe that region explicitly. The restriction is useful when the desired movement fits a regular shape, because its geometry is available before individual transfers are generated.

Croqtile makes tile geometry and layout explicit through structured movement operations. This gives the compiler the information it needs to lower a transfer efficiently and avoids repeating per-element address choreography.

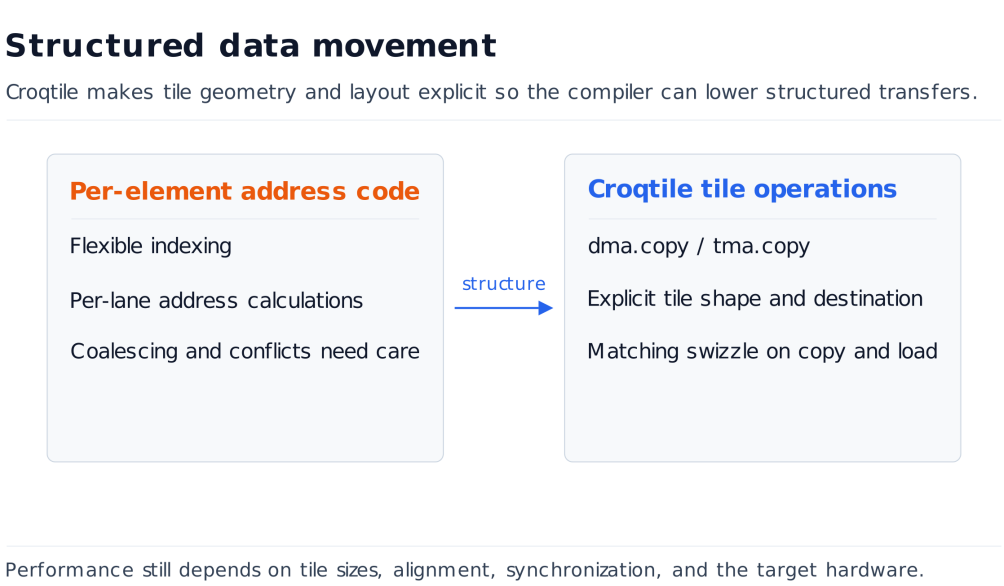


Figure 3.15: Structured tile operations expose geometry and layout for efficient lowering; performance still depends on the chosen tiles and execution configuration.

The trade is explicit: structured operations constrain how a transfer is described. They support efficient lowering, while actual performance still depends on tile sizes, alignment, layouts, and synchronization.

Easy to Use

A tile selection can use the input shape and bounded indices directly. For example, changing the number of chunks changes the derived chunk size without requiring a second set of manually calculated offsets. The programmer still chooses the partition; the language keeps the consequences of that choice connected.

Compile-Time Safety

Shape and rank checks catch incompatible uses before execution when enough information is available. Runtime checks cover constraints that depend on actual inputs, and assertions can state additional application requirements. Together they help localize failures, but they do not replace checking output values or the synchronization protocol.

Dynamic Shapes

Symbolic dimensions let one source program express a family of input shapes. Output sizes and iteration counts can be derived from the named dimensions instead of repeated as independent constants. Each call must still satisfy the kernel's tiling, layout, and target constraints; symbolic shapes do not imply that every shape is supported equally well.

Born for AI Tuning

Compact source gives an automated tuning process room to retain the kernel, compiler diagnostics, and measurement results together. Explicit tile sizes and cooperation scopes also provide concrete choices to vary. Each candidate still needs correctness checks and measurement on the intended workload; a successful compilation only establishes part of that evidence.

Tiling Optimisation in Croqtile

The following examples use the same tile-selection machinery for different computations. Matrix multiplication carries partial sums across K slices; RGB-to-grayscale conversion processes each selected image region independently. Comparing their data paths shows which parts of the structure come from tiling and which come from the computation itself.

Worked Example: Tiled matmul

Example 1: `matmul`

The first example is matrix multiplication on a device program. The code shows how Croqtile's tiling and memory management mechanisms work together to optimize matrix operations, including how DMA (Direct Memory Access) is leveraged for efficient data movement.

Example Code: Matrix Multiplication with Tiling

```
__co__ s32 [512, 1024] matmul(s32 [512, 1024] lhs, s32 [1024, 1024] rhs) {
  // Device program
  s32[lhs.span(0), rhs.span(1)] output; // Use same shape as lhs
  parallel p by 1 {
    with index = {m_tile, k_tile, n_tile} in [1, 1, 1] {
      foreach m_tile, n_tile {
```

```
        shared s32[512, 1024] l2_out;
        foreach k_tile {
          // Initiate DMA operations
          lhs_load = dma.copy.async lhs.chunkat(m_tile, k_tile) => shared;
          rhs_load = dma.copy.async rhs.chunkat(k_tile, n_tile) => shared;
          out_store_s = dma.any;
          parallel q by 1 {
            with index = {m_tile_s, k_tile_s, n_tile_s} in [4, 4, 8] {
              // Local buffers for tiling
              local s32 [128, 256] lhs_load_s_buffer_0 {0},
    lhs_load_s_buffer_1 {0};
              local s32 [256, 128] rhs_load_s_buffer_0 {0},
    rhs_load_s_buffer_1 {0};
              foreach index {
                // Add "after" primitive to chain up paired DMA operations
                lhs_load_s = dma.copy.async lhs_load.data.chunkat(m_tile_s,
    k_tile_s) => select(k_tile_s % 2, lhs_load_s_buffer_0, lhs_load_s_buffer_1
    ) after lhs_load;
                rhs_load_s = dma.copy.async rhs_load.data.chunkat(k_tile_s,
    n_tile_s) => select(k_tile_s % 2, rhs_load_s_buffer_0, rhs_load_s_buffer_1
    ) after rhs_load;
                out_load_s = dma.copy.async l2_out.chunkat(m_tile_s, n_tile_s)
     => local;
                wait out_load_s;
                // Perform the matrix multiplication (dot product) operation
                wait lhs_load_s, rhs_load_s;
                call dot_general_fp32(lhs_load_s.data, rhs_load_s.data, 256,
    256, 256, 0, 8, out_load_s.data);
                // Store the result back to l2_out
                out_store_s = dma.copy.async out_load_s.data => l2_out.chunkat
    (m_tile_s, n_tile_s);
              }
            }
          }
          // Add "after" primitive to chain up DMA operations for the final
    output
          out_store = dma.copy.async l2_out => output.chunkat(m_tile, n_tile)
    after out_store_s;
          wait out_store;
        }
      }
    }
  }
  return output;
}
```

Listing 3.71: Example Code: Matrix Multiplication with Tiling

The outer staging buffers hold full operands in this configuration because the outer tile bounds are all 1. The inner bounds `[4, 4, 8]` divide M, K, and N: a left tile is `[128, 256]`, a right tile is `[256, 128]`, and their product contributes to a `[128, 128]` output tile. The `after` clauses order the inner input copies after the outer copies. This listing is a structural illustration: helper definitions, the handoff that keeps a buffer from being overwritten while still in use, and output initialization must still be supplied and checked before treating it as a runnable kernel.

Memory Tiling with `chunkat`

In the matrix multiplication example, **memory tiling** is achieved using the `chunkat` method, which slices the input matrices `lhs` (left-hand side) and `rhs` (right-hand side) into smaller chunks to be processed in parallel.
Tiling via `chunkat`:

- The `lhs` and `rhs` matrices are tiled using the `chunkat` method. The chunks are defined by `m_tile`, `k_tile`, and `n_tile`, which represent different dimensions of the matrices involved in the multiplication.
- The chunks for both matrices are loaded asynchronously into shared storage (`lhs_load` and `rhs_load`) with the `dma.copy.async` operation. This operation loads a portion of the matrix from a source buffer into a staging buffer, which is a common optimization in large matrix operations to improve data locality.

Bounded Variables:

- The variables `m_tile`, `k_tile`, and `n_tile` are **bounded variables**. They are defined in the context of `parallel-by` and `with-in` constructs, which define the iteration space for the tiling.
- These bounded variables allow the chunks to be computed in parallel, where each thread works on a different slice of the matrix. This parallelization enables efficient computation of matrix multiplications across multiple threads.

Worked Example: rgb2gray

Example 2: rgb2gray

The second example is the RGB to Grayscale conversion algorithm. It converts a color (RGB) image to a grayscale image using a weighted average method. It shows how physical parallelism works together with virtual parallelism.

Example Code

```
__co__ auto rgb2gray(f32 [N, 3, H, W] input) {
  f32 [N, H, W] out;
  parallel q by 6 {
    with index={n, h, w} in [N, H, W]/{#q, 16, 512} {
      foreach n, h, w {
        // _ means no tiling
        input_L1_A = dma.copy input.chunkat(q#n, _, h, w) => local;
        dims : input_L1_A.span[(0), (2), (3)];
        local f32 [dims] out_L1;
        call rgb2gray_kernel_fp32_fp32(input_L1_A.data, out_L1, |out_L1|, 1);
        dma.copy out_L1 => out.chunkat(q#n, h, w);
      }
    }
  }
  return out;
}
```

Listing 3.72: Example Code

The input shape is `[N, 3, H, W]`, with the three color channels in the second dimension. The underscore in `chunkat(q#n, _, h, w)` keeps those channels together while selecting one image and a spatial region. Removing that channel dimension from the loaded shape gives the shape of `out_L1`; the helper writes one grayscale value for each spatial position.

The composition `q # n` assigns a consecutive group of images to each of the six parallel instances. The bounds assume that N divides by 6, H by 16, and W by 512 for this regular partition. Within those assumptions, each input tile contains all three channels for a 16 by 512 region, and the final copy writes its grayscale result to the matching output tile. Other image sizes need an explicit boundary policy.

3.11 Diagnostics and Methodology

Debug and Verbose: Printing, Assertions, and GDB

Debugging works best by **systematically narrowing the search**, starting with inexpensive checks. Static analysis and compile-time assertions catch errors without runtime overhead. Targeted runtime checks then help locate the problem. Use an interactive debugger once these checks have narrowed the possible causes.

Croqtile provides tools at each level: **compile-time shape printing** (`print!`, `println!`) to verify tile dimensions without launching a kernel, **runtime assertions** (`assert`) for invariant checks, **guarded device `println`** for runtime inspection of specific threads, **debug RTTI** for `debugger`, and a **verbose mode** (`-v`) to see what the compiler is actually doing under the hood.

Start from a small input whose expected result is easy to compute. For a wrong matrix element, recover its tile and local coordinates, check the selected input slices, and compare one partial sum before examining the full output. If the program hangs, trace one event slot instead: record its initial state, each trigger, and the instance expected to consume it. This distinguishes an indexing error from a missing handoff.

When the failure occurs during compilation, identify the stage before changing the kernel. Preprocessed output reveals macro expansion, generated source reveals Croqtile's translation, and verbose output identifies the target compiler command. Once the issue is fixed, rerun the reference comparison with diagnostic output disabled before measuring performance. Sections 1.4 and 2.14 give the corresponding commands and inspection syntax.

Where to go next

Appendix A collects the syntax in one place. For the compiler design behind the language, see [LancerLab, 2026].

Appendix A

Syntax Quick Reference

A condensed index of the constructs used in this guide, grouped the way you meet them: declarations first, then iteration, tile selection and movement, then statements, coordination and diagnostics. Chapter 3 explains what each construct means and why it is defined that way; this appendix is for looking up a spelling without re-reading an example.

A.1 Declarations and shapes

Construct	Meaning
`__co__ T [shape] f(T [shape] a)`	Croqtile function with shaped parameters.
`__co__ void` / `__co__ auto`	In-place result / inferred return type.
`T [a.span] b;`	Buffer with a copied shape.
`a.span` / `a.span(i)`	Whole shape / one dimension.
`x : [7, 8]`	Named `mdspan` declaration.
`a = 1, 2, 3`	`ituple` declaration.
`tensor.at(i, j)`	Position access.

A.2 Iteration

Construct	Meaning
`parallel p by N` / `parallel p, q by [M, N]`	Independent instances.
`: block` / `: thread` / `: group`	Annotations on a parallel axis.
`foreach idx in [N]` / `foreach t(1:)`	Ordered / sliced iteration.
`with idx in [a, b]`	Scoped index binding.
`p # m` / `#p`	Compose indices / axis extent.
`cdiv(a, b)`	Ceiling division.

A.3 Tile selection and movement

A.4 Statement modifiers and contraction

Construct	Meaning
`f = dma.copy src => shared`	Tile movement statement.
`a.chunkat(t)`	One tile of a regular partition.
`a.subspan(M, K).at(i, j)`	Explicit extents and tile coordinates.
`a.subspan(M, K).step(sM, sK).at(i, j)`	Strided tile selection.
`a.view(M, N).from(r, c)`	Window with explicit element offsets.
`a.span_as([dims])`	Shape reinterpretation without moving data.
`f.data` / `f.span` / `dma.any`	Moved tile / its shape / placeholder handle.
`.async` / `wait f` / `swap` / `rotate`	Async transfer and handle rotation.
`global` / `shared` / `local`	Storage specifiers.

Construct	Meaning
`dma.transp<...>` / `dma.pad<...>`	Transform while moving.
`.zfill` / `.swiz<N>`	Fill and layout modifiers.
`mc = mma.fill 0.0`	Zero an accumulator.
`ma = mma.load src.chunkat(...)`	Load an operand.
`mma.row.row mc, ma, mb`	Contraction with a layout contract.
`mma.store mc, dst`	Store the accumulator.
`.sp` / `.scale` / `mma.commit` / `mma.scale`	Contraction variants and stage fence.

Construct	Meaning
`if (expr)  ...`	Runtime predicate.
`inthreads.async (cond)` / `inthreads (cond)`	Concurrent region / sequential filter.
`shared event e[N]` / `trigger e` / `wait e`	Synchronisation tokens.
`parallel.async ...` / `stream s` / `yield`	Host scheduling and early return.
`__device__ fn()` / `call fn(args)`	Interoperating C++ function and its invocation.
`__cpp__("...")` / `__cpp__(R"(...)")`	Injected source text.
`#define` / `#if` / `#ifdef` / `#error` / `-D`	Preprocessor.
`print!` / `println!` / `print` / `println`	Compile-time and runtime output.
`assert` / `croq_assert`	Device and host assertions.

A.5 Coordination, interop and diagnostics

Where to go next

For the meaning behind a construct, return to the section that introduced it: Chapter 2 for the first reading and Chapter 3 for the design. To see a construct in a complete program, start with the worked kernels in Section 3.4, 3.6 and 3.8.

Bibliography

LancerLab. Croqtile compiler repository. https://github.com/LancerLab/croqtile, 2026.

Croqtile Programming Guide, version 1.0, November 2026.